

INFORMÁTICA Y COMUNICACIONES
G.S. Desarrollo de Aplicaciones Web



2.^a edición

Desarrollo web en entorno cliente

Juan Carlos Moreno Pérez



Contenidos digitales
www.sintesis.com


EDITORIAL
SÍNTESIS

Desarrollo web en entorno cliente

2.ª Edición

El módulo de Desarrollo Web en Entorno Cliente está relacionado con el certificado de profesionalidad IFCD0210 (cualificación profesional de referencia IFC154_3: Desarrollo de aplicaciones con tecnologías web):

— Unidad de competencia UC0491_3: Desarrollar elementos software en el entorno cliente.

Consulte nuestra página web: **www.sintesis.com**
En ella encontrará el catálogo completo y comentado



Queda prohibida, salvo excepción prevista en la ley, cualquier forma de reproducción, distribución, comunicación pública y transformación de esta obra sin contar con autorización de los titulares de la propiedad intelectual. La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (arts. 270 y sigs. Código Penal). El Centro Español de Derechos Reprográficos (www.cedro.org) vela por el respeto de los citados derechos.

Desarrollo web en entorno cliente

Juan Carlos Moreno Pérez

2.ª Edición



ASESOR EDITORIAL:

Juan Carlos Moreno Pérez

© Juan Carlos Moreno Pérez

© EDITORIAL SÍNTESIS, S. A.
Vallehermoso, 34. 28015 Madrid
Teléfono 91 593 20 98
<http://www.sintesis.com>

ISBN: 978-84-135770-9-8

Impreso en España - Printed in Spain

Reservados todos los derechos. Está prohibido, bajo las sanciones penales y el resarcimiento civil previstos en las leyes, reproducir, registrar o transmitir esta publicación, íntegra o parcialmente, por cualquier sistema de recuperación y por cualquier medio, sea mecánico, electrónico, magnético, electroóptico, por fotocopia o por cualquier otro, sin la autorización previa por escrito de Editorial Síntesis, S. A.

Índice

PRESENTACIÓN	11
1. SELECCIÓN DE ARQUITECTURAS Y HERRAMIENTAS DE PROGRAMACIÓN	13
Objetivos	13
Mapa conceptual	14
Glosario	14
1.1. Introducción	15
1.2. Front-end y back-end	15
1.3. Lenguajes de programación en entorno cliente	16
1.3.1. ReactJS	17
1.3.2. AngularJS	17
1.3.3. Vue.js	17
1.4. Características de los lenguajes de script	18
1.5. Integración de JavaScript dentro de HTML	19
1.5.1. Ejemplo de JavaScript en ficheros js separados	20
1.5.2. Ejemplo de JavaScript con código dentro del HTML	21
1.6. Herramientas de programación en JavaScript	22
1.6.1. Herramientas online	22
1.6.2. Utilización de IDE y sistemas de control de versiones	22
1.7. Posibilidades que ofrece JavaScript	24
1.7.1. Modificación del contenido de una página web	24

1.7.2. Cambiar atributos de objetos HTML	24
1.7.3. Cambiar el estilo CSS	26
1.8. Comunicación de JavaScript con el exterior	26
1.8.1. Escribir en la consola del navegador utilizando console.log()	27
1.8.2. Escribir en cualquier elemento HTML utilizando el atributo innerHTML	27
1.8.3. Generar directamente HTML utilizando el método document.write()	27
1.8.4. Generar un mensaje de alerta utilizando el método window.alert()	28
Resumen	28
Ejercicios propuestos	29
Actividades de autoevaluación	30

2. SINTAXIS DEL LENGUAJE JAVASCRIPT	33
Objetivos	33
Mapa conceptual	34
Glosario	34
2.1. Sintaxis del lenguaje. Operadores y palabras reservadas	35
2.1.1. Las 10 reglas básicas de la sintaxis del lenguaje	35
2.1.2. Las palabras reservadas de JavaScript	36
2.1.3. Uso de variables en JavaScript	37
2.1.4. Operadores en JavaScript	38
2.2. Tipos de datos. Asignaciones y expresiones	40
2.2.1. El valor null	40
2.2.2. Conversiones entre tipos de datos en JavaScript	40
2.3. Introducción a las funciones	41
2.4. Introducción a los objetos en JavaScript	42
2.5. Variables y ámbitos de utilización	43
2.5.1. Las diferencias entre var y let	44
2.6. Eventos	44
2.6.1. Tipos de eventos en JavaScript	45
2.6.2. Los listener	46
2.7. Objeto string o cadena de caracteres	47
2.8. Números	47
2.8.1. Convertir un número en un string	47
2.8.2. Ajuste de decimales. Número de decimales de un número	48
2.8.3. Precisión de un número	48
2.8.4. Convertir cualquier variable en un número	48
2.9. Fechas	48
2.9.1. Creación de un objeto de tipo fecha	49
2.9.2. Formatos de tipo fecha	49
2.9.3. Métodos del tipo fecha	49
2.10. Arrays	50
2.10.1. Operaciones con arrays	51
2.10.2. Arrays multidimensionales	51
2.11. Sentencias condicionales	52
2.12. Bucles	54
2.12.1. Recorrido de un array	55
2.13. Práctica guiada	56
Resumen	59
Ejercicios propuestos	60
Actividades de autoevaluación	61

3. UTILIZACIÓN DE LOS OBJETOS PREDEFINIDOS DEL LENGUAJE	63
Objetivos	63
Mapa conceptual	64
Glosario	64
3.1. Introducción	65
3.2. Manejo del tiempo en JavaScript	65
3.2.1. Las funciones setTimeout y setInterval	67
3.3. Cookies	69
3.3.1. ¿Para qué se utilizan las cookies?	69
3.3.2. ¿Cómo crear una cookie?	70
3.3.3. ¿Cómo leer las cookies?	70
3.4. Almacenamiento local	71
3.4.1. El objeto localStorage	71
3.5. Objetos en JavaScript	72
3.5.1. Recorrer la información de un objeto	73
3.5.2. Constructores de JavaScript	74
3.6. Funciones en JavaScript	75
3.6.1. La recursividad en JavaScript	76
3.6.2. Los parámetros de las funciones	76
3.6.3. Funciones y métodos en objetos de tipo array	77
3.6.4. Funciones y métodos en objetos y variables de tipo string	82
3.6.5. Funciones globales del lenguaje JavaScript relativas a números	84
3.6.6. Propiedades globales de JavaScript	85
3.6.7. Algunas funciones globales de JavaScript	86
3.7. Prácticas guiadas	87
3.7.1. Uso de estructuras JavaScript: creación de un generador automático de historias	87
3.7.2. Utilización avanzada de arrays: generación de un sudoku aleatorio	91
Resumen	97
Ejercicios propuestos	97
Actividades de autoevaluación	101
4. EL DOM Y EL BOM	103
Objetivos	103
Mapa conceptual	104
Glosario	104
4.1. Introducción	104
4.2. Formularios	105
4.2.1. La validación de campos	105
4.3. Expresiones regulares	106
4.4. DOM	108
4.4.1. El acceso al DOM. El método document.querySelector()	113
4.5. Eventos del DOM	115
4.6. Manejadores de eventos	117
4.7. Nodos del DOM	118
4.7.1. ¿Cómo se accede a los nodos (elementos HTML)?	118
4.7.2. ¿Cómo se crea un nuevo nodo?	119
4.7.3. ¿Cómo se elimina un nodo?	120
4.7.4. Práctica guiada: creación y eliminación de elementos	121

4.8. Propiedades del DOM	123
4.8.1. El acceso a los nodos	123
4.8.2. Capturar ciertos eventos	123
4.9. Modificando el DOM	124
4.9.1. Cambiar el tamaño del texto	125
4.9.2. Mostrar y ocultar elementos de una página web	126
4.9.3. Deshabilitar objetos	128
4.9.4. Comprobar que se introduce información en un campo de texto	129
4.10. BOM	130
4.10.1. El objeto window	131
4.10.2. El objeto location	132
4.10.3. El objeto history	132
4.10.4. El objeto navigator	132
Resumen	133
Ejercicios propuestos	135
Actividades de autoevaluación	135
 5. MECANISMOS DE COMUNICACIÓN ASÍNCRONA	139
Objetivos	139
Mapa conceptual	140
Glosario	140
5.1. Introducción	140
5.2. Los web workers	141
5.3. JSON	142
5.3.1. Sintaxis JSON	143
5.3.2. Práctica guiada: instalación de un servidor JSON	144
5.4. Comunicación asíncrona. AJAX	146
5.4.1. Práctica guiada: sistema de localización de ciudades	148
5.4.2. Ejemplo con AJAX y JSON	151
5.4.3. Ejemplo con AJAX y XML	153
5.5. Comunicación asíncrona con el servidor. AJAX y jQuery	154
5.5.1. Carga simple de datos load()	155
5.5.2. Llamada asíncrona utilizando POST	156
5.6. Práctica guiada: encuestas interactivas con AJAX	158
Resumen	162
Ejercicios propuestos	163
Actividades de autoevaluación	165
 6. REACTJS: UN FRAMEWORK DE JAVASCRIPT	167
Objetivos	167
Mapa conceptual	168
Glosario	168
6.1. Introducción a React	169
6.1.1. Creación de la primera aplicación React	170
6.1.2. Ejecución de la aplicación React	171
6.2. Análisis del contenido de la primera aplicación	172
6.2.1. El fichero index.html	172
6.2.2. El fichero index.js	172

6.2.3. El fichero src/App.js	173
6.3. Introducción a JSX	174
6.4. Componentes React	175
6.4.1. Class components y function components	176
6.4.2. Componentes contenedores y presentaciones	178
6.4.3. Creando el componente principal. Conversor euro-dólares	179
6.5. Los componentes. El state	180
6.5.1. El state y los métodos	180
6.5.2. Los componentes hijo y la comunicación padre-hijo: las props	182
6.6. El DOM virtual de React	184
6.6.1. Renderización y cambios	184
6.7. Los callbacks	185
6.8. Prácticas guiadas	187
6.8.1. Los reyes godos	187
6.8.2. Conversor de dólares a euros y viceversa	191
Resumen	192
Ejercicios propuestos	193
Actividades de autoevaluación	201



RECURSOS DIGITALES

Archivo libro_cliente.rar, que incluye los recursos necesarios para trabajar los ejemplos que aparecen en el libro

Presentación

Desarrollo Web en Entorno Cliente es un módulo fundamental del ciclo formativo de grado superior Desarrollo de Aplicaciones Web, dado que los contenidos que se trabajan en él se van a utilizar profusamente durante la vida profesional de cualquier programador web.

Es un hecho que, actualmente, la programación web del lado del cliente está teniendo cada vez más importancia, dada la potencia de los nuevos frameworks existentes en el mercado, como ReactJS o Angular. Dentro del contenido del libro, se ha reservado el último capítulo para trabajar el framework ReactJS, el más utilizado por los programadores a nivel mundial.

Además de para preparar este módulo, este manual servirá para completar el temario de los certificados de profesionalidad relacionados con la tecnología web, como son Confección y Publicación de Páginas Web, Sistemas de Gestión de Información y otros. En concreto, se cubren los contenidos de la unidad de competencia UC0491_3, perteneciente al certificado de profesionalidad Desarrollo de aplicaciones con tecnología web (código IFCD0210, cualificación profesional de referencia IFC154_3).

Igualmente, el contenido de esta obra sirve como colofón para cualquier tipo de programación web, ya que se tratan conceptos que complementan la formación de cualquier programador como:

- Las características de los lenguajes de programación del lado del cliente. Poniendo énfasis en la diferencia entre back-end y front-end.
- La sintaxis de JavaScript.
- Los objetos predefinidos, las funciones, etc., fundamentales a la hora de crear una web. Estos conceptos harán que la web sea más eficiente y dinámica y se adapte mejor al objetivo que se pretenda conseguir.
- El DOM y el BOM. Conceptos básicos en la programación web del lado del cliente que todo programador debe manejar de forma eficiente.
- Los mecanismos de comunicación asíncrona, los cuales permitirán al programador realizar una web fluida sin tener que refrescar toda la información cada vez que ocurra un evento en ella.

- ReactJS. Como se ha citado anteriormente, es el framework más utilizado en la actualidad y permitirá al lector obtener una ventaja competitiva a la hora de su incorporación al mercado de trabajo.

Para todo aquel profesional que tenga conocimientos de programación web, este libro será una ayuda indispensable si quiere ampliar sus conocimientos sobre la programación web en el front-end.

En esta edición se han potenciado aspectos del lenguaje muy utilizados en frameworks como Angular y ReactJS, como son las funciones flecha y los métodos `map` o `reduce` de arrays. Asimismo, para darle un enfoque más cercano al ámbito laboral, se ha profundizado en el capítulo de ReactJS añadiendo ejemplos y eligiendo ejercicios más progresivos para reducir su curva de aprendizaje.

A lo largo de los seis capítulos, se presentan muchos ejemplos y fragmentos de código para practicar, que permitirán al lector adquirir los conocimientos que se ofrecen. En www.sintesis.com, usando el código que aparece en la primera página de este manual, se pueden descargar los recursos necesarios para trabajar con ellos.

La filosofía seguida en esta obra es *aprende haciéndolo*, por lo que, además de las prácticas guiadas, se proponen más de setenta ejercicios que permitirán, una vez realizados, tener la certeza de que se maneja la materia con solvencia.

En www.sintesis.com puedes descargar el archivo [libro_cliente.zip](#), que incluye los recursos necesarios para trabajar los ejemplos. Se puede descargar a través del código y las indicaciones incluidas en la primera página del libro.

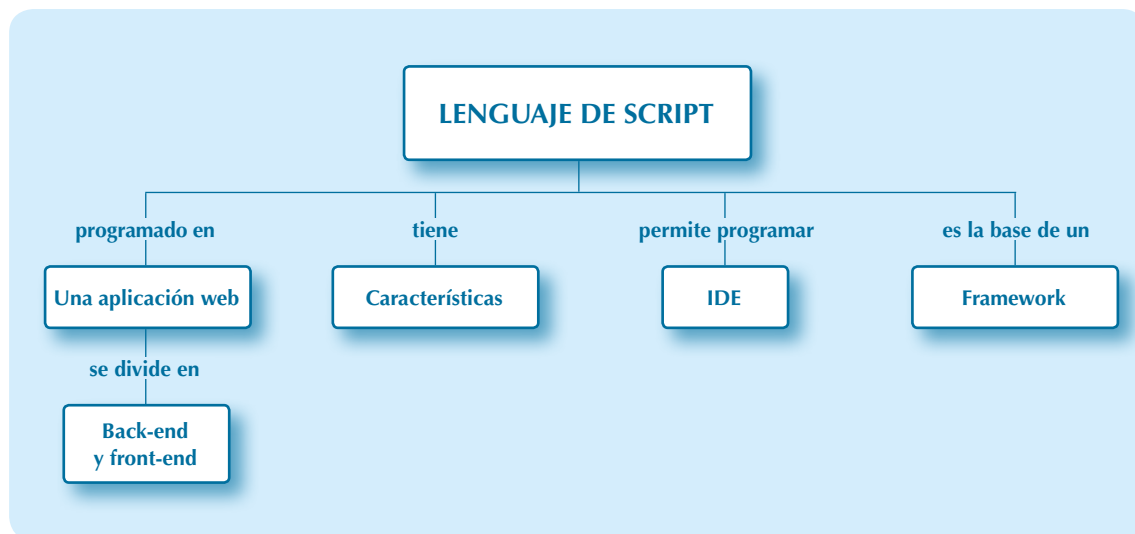
No queremos terminar esta presentación sin animar al lector a que trabaje con ahínco todos los temas y conceptos que se exponen a lo largo de sus páginas. La contraprestación que conseguirá será alta, puesto que el conocimiento que se adquiera se va a utilizar durante toda su carrera profesional.

Selección de arquitecturas y herramientas de programación

Objetivos

- ✓ Diferenciar los lenguajes y la distribución de la lógica entre back-end y front-end.
- ✓ Conocer las características de los lenguajes de script y framework más utilizados.
- ✓ Aprender a integrar JavaScript en el código HTML.
- ✓ Entender las posibilidades de un lenguaje de programación en entorno cliente.

Mapa conceptual



Glosario

DOM (Document Object Model). Conjunto de los elementos de los que consta una página web (enlaces, botones, textos, etc.).

DOM virtual. Concepto de programación que consiste en que el framework guarde en memoria una copia del DOM original y sirve para reducir al máximo las renderizaciones a nivel de navegador para incrementar el rendimiento de la aplicación.

JSX. Extensión de JavaScript. Aunque JSX parece un lenguaje de plantillas, se puede integrar JavaScript en él para darle una mayor potencia.

Package manager. Gestor de paquetes que permite registrar el software que está instalado en el equipo, actualizarlo o eliminarlo de una forma eficiente y sencilla.

Patrón de diseño. Técnica para resolver un problema de programación de una forma estándar.

Renderizar. Anglicismo que hace referencia a la visualización de un elemento o una página web en el navegador.

Slack. Herramienta de trabajo colaborativo en la cual se pueden crear canales de comunicación e intercambiar no solamente texto, sino archivos, URL, etcétera.

TypeScript. Lenguaje open-source mantenido y desarrollado por Microsoft que es un superconjunto de JavaScript. Se diseñó para grandes aplicaciones y compilarse a JavaScript posteriormente.

1.1. Introducción

Cuando se creó la web allá por 1989 en el laboratorio europeo de partículas (CERN) por Tim Berners-Lee, nadie se podría imaginar en lo que se convertiría décadas más tarde.

Actualmente, el W3C es el consorcio que se encarga de desarrollar los estándares para que el desarrollo de internet funcione a largo plazo. El W3C es un organismo abierto y cualquiera puede unirse a sus grupos y participar en los blogs u otras discusiones. En España, el sitio web del W3C es www.w3c.es.

Hoy en día, gran parte del comercio mundial se realiza de forma electrónica y no solamente Amazon, Alibaba u otros gigantes, sino cualquier pequeño y mediano comercio. Es más, dada la deslocalización de sus clientes o trabajadores, muchas empresas están migrando sus sistemas tradicionales dentro de los servidores de la empresa a aplicaciones alojadas en servidores de internet (computación en la nube). Por ello, empresas como AWS (Amazon Web Services) están creciendo de forma exponencial. Por estas y otras razones, el puesto de desarrollador web está siendo muy demandado en el mercado.

El desarrollo web es muy diverso y, por ello, los trabajadores se suelen especializar solo en un aspecto de este. Existen diseñadores web en los que se valora la originalidad o el buen gusto para elegir colores, formas y disposición de elementos en una página web, programadores web expertos en algún lenguaje del lado del cliente o servidor, administradores de sistemas y bases de datos, arquitectos web, etc.

1.2. Front-end y back-end

Actualmente, se suele catalogar el desarrollo web en dos partes back-end (la parte no visible de la web, como las bases de datos o los scripts que se ejecutan en el servidor) y front-end (la parte visible de una web, como las hojas de estilo, el código HTML, los scripts que se ejecutan en el lado del cliente). A continuación, se describirá con más profundidad este modelo cliente servidor.

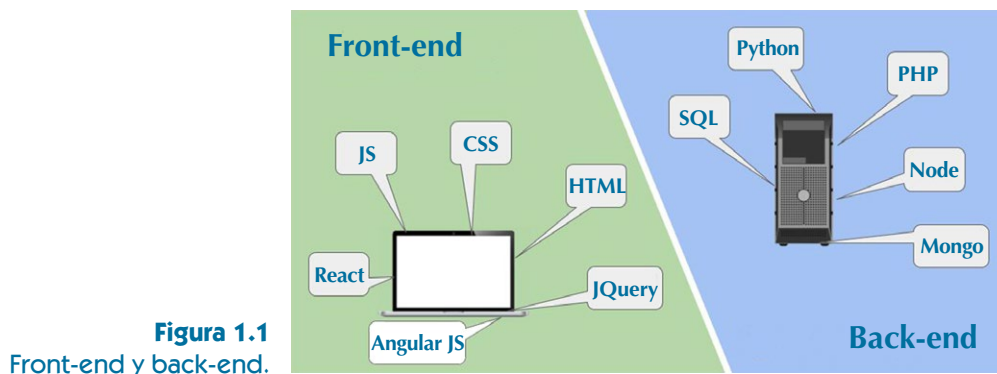


Figura 1.1
Front-end y back-end.

En las empresas desarrolladoras de aplicaciones web, suele haber técnicos especialistas en back-end y front-end. Los técnicos de back-end se encargan de todo el proceso en el lado del servidor, como el acceso a la base de datos (MySQL, MaríaDB, PostgreSQL, MongoDB, Oracle), creación de servicios, etc. Estos técnicos programarán en lenguajes como PHP, Ruby on Rails, Django, Node.js, .NET, etc.



PARA SABER MÁS

Los frameworks nacieron como librerías, más o menos completas, que tenían una serie de estructuras que permitían al programador tener una base para la creación y el desarrollo de sus proyectos. Hoy, son mucho más que eso, puesto que pueden utilizar lenguajes como TypeScript o JSX, los cuales luego se compilan a JavaScript.

Entre las ventajas que aportan los frameworks, están:

- El *coste*. Muchos de estos frameworks son de código abierto, con lo cual no hay que realizar ninguna inversión.
- Están suficientemente *probados* y su código suele carecer de errores, puesto que muchos programadores lo utilizan. Además, suelen tener un alto nivel de seguridad y rendimiento.
- Permite desarrollar mucho más *rápido*, puesto que muchas de las estructuras, clases, patrones de diseño, etc., vienen ya incorporadas en el framework.
- Cualquier persona que maneja un framework determinado *puede entender* e incorporarse de forma eficiente y efectiva a un equipo de desarrollo que lo esté utilizando en un proyecto determinado.

En la parte del front-end, prima la parte creativa y la originalidad, puesto que el perfil es más cercano al diseñador, aunque también se trabaja en el código. La programación de la interfaz se llevará a cabo en lenguajes como HTML, CSS, JavaScript. Con frameworks como AngularJS, ReactJS y otros, la parte cliente o front-end está ganando terreno a la parte servidora. En muchas ocasiones, estos frameworks están diseñados para almacenar estructuras de datos relegando al back-end o simplemente para asegurar la persistencia de estos.

1.3. Lenguajes de programación en entorno cliente

La programación web en el lado del cliente se basa en tres pilares fundamentales que se citan a continuación:

1. *El lenguaje HTML*. No es un lenguaje de programación, sino un lenguaje de marcado. El HTML define el contenido que va a tener el documento. La función del navegador web será la de leer e interpretar todo este contenido, junto con las etiquetas, y visualizarlo al cliente. La ventaja del código HTML es que cualquier navegador debería visualizar el contenido de la página web de la misma forma y con el mismo aspecto.
2. *El lenguaje CSS*. Define la presentación del documento. CSS es un lenguaje de diseño gráfico y su objetivo es que la página web sea atractiva al usuario. No modifica el comportamiento ni el contenido, sino el aspecto de la página web.
3. *El lenguaje JavaScript*. El código o lenguaje JavaScript agrega el contenido dinámico a las páginas web. JavaScript es un verdadero lenguaje de programación, a diferencia de los lenguajes HTML y CSS.

Actualmente, las empresas no programan directamente sobre JavaScript, sino basándose en un framework de este. A continuación, se citarán algunos de los frameworks más utilizados para JavaScript.

1.3.1. ReactJS

ReactJS (<https://reactjs.org/>) es un framework creado por Facebook que permite a los programadores realizar aplicaciones web de una forma rápida y eficiente renderizando (dibujando) los componentes del front-end de una manera sencilla y eficaz.

React utiliza programación orientada a componentes (que no objetos). Los componentes gestionan su propio estado y, cuando se agrupa una serie de componentes, los programadores son capaces de ir creando las interfaces de usuario.

Una de las características de React es que utiliza un DOM virtual que mapea los objetos desde este hasta el DOM del navegador.



Actividad propuesta 1.1

Averigua qué es la programación reactiva.

1.3.2. AngularJS

Angular (<https://angular.io>) fue creado y es mantenido por Google. La primera versión de Angular se denominó *AngularJS* y todavía hay una comunidad utilizando este framework por su fácil integración con JavaScript. Las versiones sucesivas de Angular se denominan *Angular* a secas y ya ha dejado de ser una simple librería para pasar a ser una plataforma de desarrollo.

El problema con este y otros frameworks es que su curva de aprendizaje es bastante pronunciada, dado que no son fáciles de aprender.

Actualmente, en Angular, se programa en TypeScript, que es un superconjunto de JavaScript desarrollado por Microsoft y utiliza el patrón reactivo RxJS.

1.3.3. Vue.js

Una de las características de este framework (<https://vuejs.org>) frente a otros es la ligereza y la velocidad de ejecución. El objetivo que se plantearon sus desarrolladores y diseñadores fue el crear un framework con las mejores ventajas de los existentes. A diferencia de Angular, su curva de aprendizaje no es tan pronunciada y los desarrolladores de Laravel (un framework de back-end) lo utilizan para usarlo en el front-end de sus aplicaciones.

Al igual que React, utiliza un DOM virtual, dadas las ventajas que ofrece este tipo de implementaciones.

**TOMA NOTA**

Existen otros frameworks como EmberJS, BackboneJS, MeteorJS, Aurelia.js, Polymer o Mithril.js. Escoger un framework es un proceso importante, puesto que el objetivo es elegir el más popular para que tenga un soporte más amplio, el que tenga más futuro, el más rápido de ejecución, el más fácil de aprender. Dicha elección muchas veces es complicada, ya que la tecnología cambia de forma muy rápida.

WWW**Recurso web**

A través del siguiente código QR, puedes acceder a la página web Raygun y encontrar información sobre los nueve frameworks más utilizados:



1.4. Características de los lenguajes de script

Hubo un tiempo en el que los lenguajes de programación se utilizaban para crear programas de nóminas, contabilidad, edición de texto, hojas de cálculo, gestión de almacenes, etc., en los que se realizaba un análisis, un diseño de la aplicación y luego se pasaba a codificar todo lo desarrollado en fases anteriores. Los programas se ejecutaban en un equipo standalone o en entornos cliente servidor.

Desde hace mucho tiempo, ese tipo de necesidades han cambiado, puesto que la conectividad total y las nuevas necesidades de las empresas y clientes son distintas. Ahora se necesitan aplicaciones que se puedan ejecutar sobre un navegador web o aplicaciones para dispositivos móviles como smartphones o tabletas.

Ya no se desarrolla todo desde cero, sino que, en muchos casos, se utiliza un sistema host—como puede ser el navegador— para integrar pequeños fragmentos de código que aporten el aspecto dinámico a las páginas web visualizadas.

Los scripts nacieron como fragmentos de código que realizaban ciertas tareas o rutinas concretas. En los sistemas operativos, los scripts se usan para automatizar tareas y siempre van a ser ejecutadas por un intérprete de comandos (por lo tanto, los scripts siempre van a ser interpretados).

Actualmente, los scripts no son pequeños fragmentos de código, sino que pueden considerarse auténticos programas.

Véanse algunas de las diferencias entre los lenguajes de script y los lenguajes de programación:

- Los lenguajes de script son interpretados, mientras que, muchas veces, los lenguajes de programación se compilan.
- Los lenguajes de script utilizan componentes ya preexistentes, mientras que los lenguajes de programación, en ocasiones, empiezan a desarrollarse desde cero.
- A veces, los scripts se incrustan dentro de otros programas (como el JavaScript se integra con el HTML).

- Los programas de lenguajes de programación se pueden ejecutar de forma independiente, mientras que un lenguaje de script se ejecuta muchas veces dentro de otro programa.
- Los scripts se ejecutan línea a línea, con lo cual pueden producirse muchos errores en ejecución.
- Los lenguajes de script no generan un fichero ejecutable.
- Los lenguajes de script no necesitan ser compilados.
- Los lenguajes de script han sido diseñados para que ser fáciles de utilizar y programar.

Algunos lenguajes de programación son: Java, C, C++, Swift, Pascal, etc.

Algunos lenguajes de scripting son: JavaScript, Shell, Perl, PHP, Python, Ruby, etc.



SABÍAS QUE...

El lenguaje de scripting Python es uno de los que tienen más proyección porque es ámpliamente utilizado en inteligencia artificial.

1.5. Integración de JavaScript dentro de HTML

Como ya se ha explicado, JavaScript se combina o complementa al código HTML de una página web. Existen dos opciones:

1. Integrar el código JavaScript dentro de los archivos HTML.
2. Tener separado del HTML el código JavaScript en archivos con extensión js.

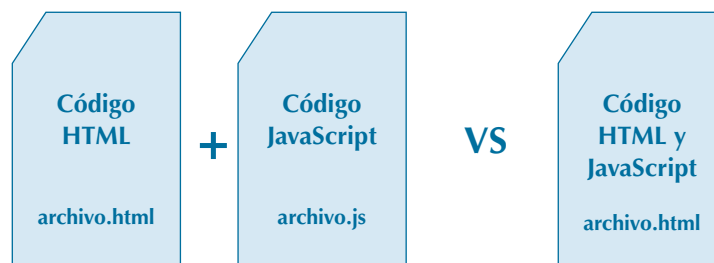


Figura 1.2

Código JavaScript dentro y fuera de un archivo HTML.

TOMA NOTA



Lo más limpio y eficaz es tener el código JavaScript fuera de los archivos HTML por las ventajas que ofrece.

A continuación, en el apartado 1.5.1, se desarrolla un ejemplo de ambas posibilidades.

1.5.1. Ejemplo de JavaScript en ficheros js separados

Los archivos JavaScript en un proyecto profesional suelen estar en ficheros separados del código HTML. Generalmente, las empresas y programadores con experiencia así lo aconsejan. Seguidamente, se ofrece al lector un ejemplo de página web en el que los códigos HTML y JavaScript residen en ficheros diferentes.

RECUERDA

- ✓ El código JavaScript está contenido dentro de las etiquetas `<script>` y `</script>`.
El código JavaScript se puede colocar tanto dentro de la etiqueta `<head>` como dentro de la etiqueta `<body>`. Nuestro consejo es que se sitúe en el mismo sitio. No es buena práctica diseminar el código por toda una página porque luego es imposible de entender y mantener.
Es posible ver versiones antiguas de JavaScript con etiquetas del tipo: `<script type="text/javascript">`.

Contenido del archivo *index.html*:

```
<!DOCTYPE html>
<html>
<title>Myfpschool</title>
<head>
<script src="script.js"></script>
</head>
<body>
</body>
</html>
```

La etiqueta `<script src="script.js"></script>` indica que el código JavaScript está contenido en un archivo aparte llamado *script.js*, que se encuentra en la misma carpeta que este archivo *index.html*.

A continuación, se muestra el contenido del archivo *script.js*:

```
function diAlgo()
{
    alert("hola");
}
diAlgo();
```

Actividad propuesta 1.2



Crea los dos archivos de este apartado y ejecútalos comprobando que la página web muestra un mensaje al cargarse.

1.5.2. Ejemplo de JavaScript con código dentro del HTML

Como se ha explicado anteriormente, el código JavaScript también puede estar embebido dentro del código HTML. A continuación, se muestra el mismo ejemplo del apartado 1.5.1, pero con los códigos HTML y JavaScript en un mismo fichero.

Detalle del archivo *index.html*:

```
<!DOCTYPE html>
<html>
<title>Myfpschool</title>
<head>
<script>
  function diAlgo()
  {
    alert("Hola");
  }
</script>
</head>
<body>

<script>
diAlgo();
</script>

</body>
</html>
```



Actividad propuesta 1.3

Si creas y ejecutas las actividades propuestas 1.1 y 1.2, podrás comprobar que el funcionamiento es igual en ambos. Observa cómo ambos códigos muestran un mensaje de alerta al navegante.

Como se puede observar, aunque el resultado es el mismo, la forma de distribuir el código HTML y el JavaScript no es igual. El mejor consejo es utilizar archivos separados para HTML y JavaScript, salvo que las líneas de código de este último sean mínimas y no vayan a ser modificadas prácticamente nunca.

Las ventajas de tener el JavaScript en un fichero separado son que las páginas cargarán mucho más rápido, se independiza el HTML del código y, como se puede adivinar, el JavaScript será mucho más fácil de mantener.

TOMA NOTA



Los proyectos suelen tener los scripts en una carpeta aparte de nombre `script` o `js` para tener los archivos más ordenados.

1.6. Herramientas de programación en JavaScript

Existen múltiples alternativas a la hora de elegir una herramienta de programación. Una de las ventajas de programar en JavaScript es que puede servir un simple editor de texto como Notepad o gedit. Esto no es lo deseable cuando se desarrolla un proyecto y, por lo tanto, las empresas y profesionales suelen utilizar otras alternativas como los entornos de programación online o herramientas de programación con sistemas de control de versiones.

A continuación, se mostrarán dos alternativas de herramientas de programación.

1.6.1. Herramientas online

Las ventajas que ofrece un IDE (entorno integrado de desarrollo) online son evidentes, se puede ejecutar y probar código desde cualquier dispositivo solamente teniendo acceso a internet. Existen muchas herramientas en el mercado y una de ellas es Coding Ground de Tutorialspoint (https://www.tutorialspoint.com/online_javascript_editor.php).

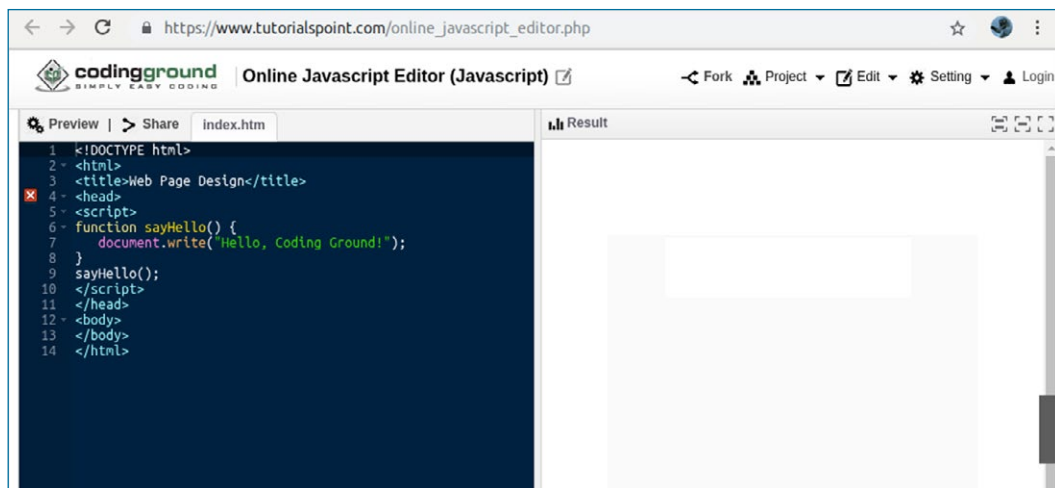


Figura 1.3

Interfaz de Coding Ground, herramienta para compilar JavaScript online.

Esta herramienta no solo permite gestionar los distintos ficheros de un proyecto, sino que se puede descargar o agregar cualquier fichero que haga falta.

1.6.2. Utilización de IDE y sistemas de control de versiones

Otra de las opciones es utilizar un IDE complementado con un sistema de control de versiones. Una alternativa puede ser utilizar Atom (<https://atom.io>).

Atom está disponible para plataformas Linux, Windows y Mac OS X y es una herramienta libre, gratuita, ligera y potente.

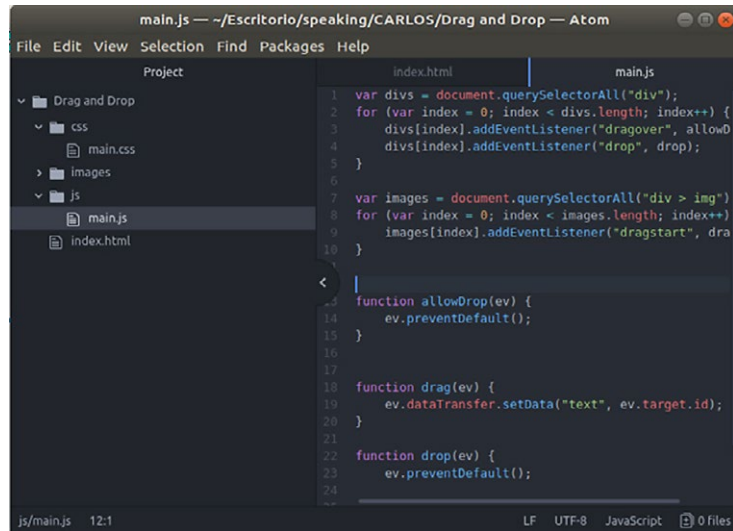


Figura 1.4
Detalle de un IDE
utilizando Atom.

Se citan algunas ventajas de este IDE:

- Es open-source, lo que quiere decir que, si se desea hacer modificaciones o arreglar el código fuente, está disponible para todo el público. Muchas veces los programadores no van a modificar el código, pero al ser software libre se aseguran de que una comunidad de usuarios se encargará de ello, haciendo que los errores y mejoras se corrijan mucho antes que con un software propietario.
- Es gratuito.
- Tiene una gran comunidad detrás de la herramienta, con lo cual cualquier problema o inquietud puede ser resuelta en Slack o incluso en Twitter.
- Es modular. Se pueden deshabilitar ciertas funciones y reemplazar por otras del gusto del programador.
- Tiene integrado un package manager. Se pueden instalar paquetes y temas con un comando `apm` desde el terminal.
- Está integrado con GitHub. Atom está pensado para utilizarlo conjuntamente con este sistema tan famoso de control de versiones.
- El sistema de paneles de Atom es muy apreciado por los programadores.
- Tiene un sistema muy bueno de autocompletado.

Recursos web

www

Como alternativa a Atom, se puede usar Sublime text. (<https://www.sublimetext.com/>) o VisualStudio (<https://code.visualstudio.com>).

1.7. Posibilidades que ofrece JavaScript

A continuación, se mostrarán varios ejemplos básicos de qué se puede hacer con JavaScript.

1.7.1. Modificación del contenido de una página web

En el siguiente código, se muestra cómo se puede modificar el contenido HTML de forma dinámica:

```
<!DOCTYPE html>
<html>
<body>

<h1>Modificando el código HTML</h1>

<p id="prueba">Modificando el contenido.</p>

<button type="button"
onclick="document.getElementById('prueba').innerHTML = 'CAMBIANDO el
contenido!'">
¡Dale!</button>

</body>
</html>
```

El aspecto de la página web creada será el que se muestra en la figura 1.5.



Figura 1.5
Aspecto de la página web creada
modificando el código HTML.

Como se puede observar, ejecutando el código anterior, al pulsar el botón ¡Dale!, cambiará el párrafo “Modificando el contenido” por el párrafo “Cambiando el contenido”. Más adelante se estudiará en profundidad el código JavaScript del ejemplo anterior.

1.7.2. Cambiar atributos de objetos HTML

En el siguiente ejemplo, se va a modificar de forma dinámica el atributo src de un objeto tipo img en un código HTML.

Tras copiar el siguiente código en un archivo de texto, puede observarse cómo funciona:

```

<!DOCTYPE html>
<html>
<body>

<h1>Cambio de imágenes con JavaScript</h1>



<p>Haz click sobre las letras para cambiarlas.</p>

<script>
function cambiaPic() {
    var image = document.getElementById('myFPImage');
    if (image.src.match("green")) {
        image.src = "http://myfpschool.com/wp-content/uploads/2016/06/my-
        black.jpeg";
    } else {
        image.src = "http://myfpschool.com/wp-content/uploads/2016/06/my-
        green.jpeg";
    }
}
</script>

</body>
</html>

```

El aspecto de la página web es el que se muestra en la figura 1.6.

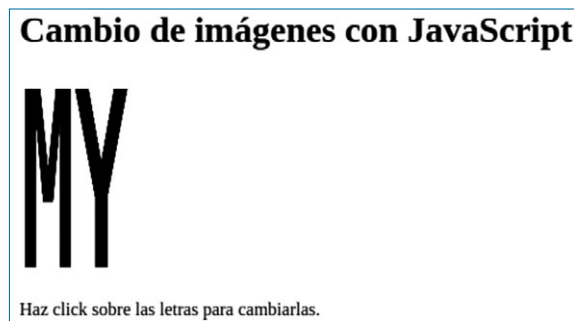


Figura 1.6
Aspecto de la página web del cambio
de imágenes con JavaScript.

Se puede observar que, cuando se hace clic sobre las letras MY, estas cambian de negro a verde, y viceversa, porque, en el script anterior, comprueba qué imagen se está visualizando.



Actividad propuesta 1.4

Copia el código del ejemplo de este apartado y comprueba si funciona en un navegador. En esta ocasión, utiliza tus propias imágenes.

1.7.3. Cambiar el estilo CSS

Con JavaScript no solamente se puede cambiar el contenido, sino que también se puede cambiar el estilo (CSS) de cualquier elemento de la página HTML.

Para ver cómo se puede cambiar el aspecto, se va a generar una página cuyo aspecto sea el de la figura 1.7 y, una vez que se pulse un botón, cambie el aspecto del párrafo para parecerse a la figura 1.8.

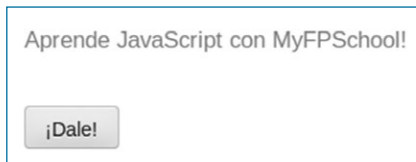


Figura 1.7
Aspecto de la página antes de pulsar el botón.

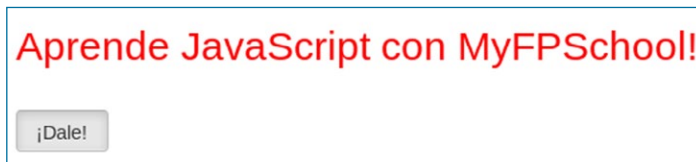


Figura 1.8
Aspecto de la página después de pulsar el botón.

Para ver cómo ha cambiado el aspecto de la página anterior, hay que copiar el siguiente código en un editor como Atom y comprobar si funciona correctamente:

```
<!DOCTYPE html>
<html>
<body>
<p id="mytxt">Aprende JavaScript con MyFPSchool!</p>
<button type="button" onclick="myFunction()">¡Dale!</button>
<script>
function myFunction() {
    var x = document.getElementById("mytxt");
    x.style.fontSize = "25px";
    x.style.color = "red";
}
</script>
</body>
</html>
```

Como se puede observar, en la función myFunction, se selecciona el párrafo que se quiere modificar y se cambia su color a rojo y su tamaño a 25 píxeles.

1.8. Comunicación de JavaScript con el exterior

Existen varias opciones para que el código JavaScript se comunique con el usuario. A continuación, se detalla cada una de ellas.

1.8.1. Escribir en la consola del navegador utilizando `console.log()`

Esta opción la utilizan solamente los desarrolladores. Ningún usuario suele acceder a la consola para ver el contenido escrito en ella.

RECUERDA

- ✓ Para acceder a la consola, basta con pulsar F12 y elegir Consola en su navegador.

Este es un ejemplo de un script que muestra un mensaje por consola:

```
<script>
  console.log("Síntesis, ejemplo de consola");
</script>
```

1.8.2. Escribir en cualquier elemento HTML utilizando el atributo `innerHTML`

A continuación, se muestra un ejemplo de un script que utiliza el atributo `innerHTML` para cambiar el contenido de un elemento HTML:

```
<p id="parrafito"></p>
<script>
document.getElementById("parrafito").innerHTML = 5 + 6;
</script>
```

Nótese que se ha seleccionado previamente el objeto con identificador `parrafito` utilizando el método `getElementById`.

1.8.3. Generar directamente HTML utilizando el método `document.write()`

Con el método `write` se puede generar código HTML directamente. Se pueden también añadir etiquetas en la llamada al método si se desea.

```
<script>
  document.write("<h2>Buenos días</h2>");
</script>
```

1.8.4. Generar un mensaje de alerta utilizando el método `window.alert()`

Con el método `window.alert()` o simplemente `alert()`, se puede mostrar un diálogo emergente en el navegador.

A continuación se presenta un ejemplo de utilización del método `window.alert()` dentro de un código JavaScript:

```
<script>  
    window.alert("BUENAS NOCHES");  
</script>
```

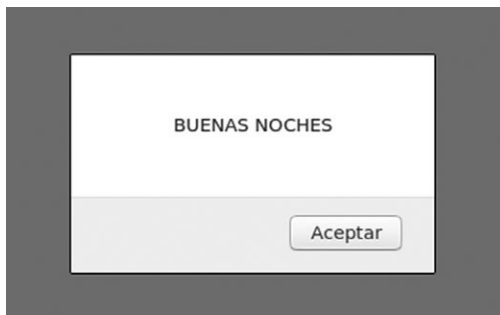


Figura 1.9
Ejemplo del diálogo emergente obtenido usando el método `window.alert()` dentro de un código JavaScript.

Resumen

- W3C es el que se encarga de desarrollar los estándares para que el desarrollo de internet funcione a largo plazo.
- Los frameworks nacieron como librerías, más o menos completas, que tenían una serie de estructuras que permitían al programador disponer de una base para la creación y el desarrollo de sus proyectos.
- Los técnicos de back-end se encargan de todo el proceso en el lado del servidor, como el acceso a la base de datos.
- En la parte del front-end, prima la parte creativa y la originalidad, puesto que el perfil es más cercano al diseñador, aunque también se trabaja en el código.
- ReactJS es un framework creado por Facebook.
- Angular fue creado y es mantenido por Google.
- En Vue.js prima la ligereza y la velocidad de ejecución.
- Los scripts nacieron como fragmentos de código que realizaban ciertas tareas o rutinas concretas.
- En JavaScript se puede colocar el código dentro y fuera del HTML, aunque se aconseja colocarlo fuera.

- Entre las posibilidades que ofrece JavaScript, están:
 - Modificar el contenido de una página web.
 - Cambiar atributos de objetos HTML.
 - Cambiar el estilo CSS.
- JavaScript se puede comunicar con el exterior mediante:
 - `Console.log()`.
 - Mediante el atributo `innerHTML`.
 - `document.write()`.
 - `window.alert()`.

Ejercicios propuestos



1. Crea un segundo botón para el ejemplo del apartado 1.6.1 que modifique el título “Modificando el contenido”.
2. Elabora una secuencia de imágenes con al menos 4 fotogramas. Según el usuario vaya haciendo clic sobre las imágenes, la secuencia se irá reproduciendo.
3. Realiza un test de siete preguntas donde cada pregunta tenga dos botones (verdadero y falso). Dependiendo de si el usuario hace clic en verdadero o en falso la frase se pone verde (acierto) o roja (error).
4. Desarrolla una página web en la que al pulsar un botón se genere un mensaje por consola.
5. Diseña una web con tres botones para que al pulsarlos se genere un mensaje de bienvenida en ruso, español e inglés en la misma página. El mensaje irá en un párrafo (etiqueta `<p>`). Se pide que los mensajes tengan un color diferente dependiendo del idioma.
6. Modifica el código anterior para que los mensajes de bienvenida se generen con alertas.
7. Cambia el código anterior para que los mensajes de bienvenida se generen en consola.
8. Transforma el código anterior para que los mensajes de bienvenida se generen con el método `write()`.
9. ¿En qué se diferencia JavaScript de Java?
10. Describe las ventajas más importantes de usar JavaScript.

ACTIVIDADES DE AUTOEVALUACIÓN

1. ¿Qué es JSX?:
 - ☐ a) Una extensión de JavaScript.
 - ☐ b) Un superconjunto de JavaScript mantenido por Microsoft.
 - ☐ c) Una herramienta de trabajo colaborativo.
2. ¿Cuál es el consorcio que se encarga de desarrollar los estándares para internet?:
 - ☐ a) CERN.
 - ☐ b) W3C.
 - ☐ c) InterNic.
3. ¿Cuál de los siguientes lenguajes o frameworks no pertenece al back-end?:
 - ☐ a) Angular.
 - ☐ b) PHP.
 - ☐ c) Python.
4. De los siguientes softwares, ¿cuál no pertenece al front-end?:
 - ☐ a) ReactJS.
 - ☐ b) JQuery.
 - ☐ c) Mongo.
5. ¿Cuál de los siguientes softwares no es una base de datos?:
 - ☐ a) PostgreSQL.
 - ☐ b) MongoDB.
 - ☐ c) Python.
6. ¿Cuál de los siguientes lenguajes está mantenido por Microsoft?:
 - ☐ a) TypeScript.
 - ☐ b) JavaScript.
 - ☐ c) JSX.
7. ¿Qué es ReactJS?:
 - ☐ a) Es un framework creado por Facebook.
 - ☐ b) Es un framework creado por Google.
 - ☐ c) Es un framework creado por Microsoft.
8. ¿Cuál de los siguientes lenguajes no se considera de scripting?:
 - ☐ a) Shell.
 - ☐ b) Swift.
 - ☐ c) Python.
9. ¿Cuál de las siguientes líneas de código es correcta?:
 - ☐ a) `<script src=script.js /></script>`
 - ☐ b) `<script src=script.js>`
 - ☐ c) `<script src="script.js"></script>`

10. Para obtener la identidad de un elemento del DOM, ¿qué método se utiliza?:

- ☐ a) `document.getElementById()`
- ☐ b) `document.DOMNode()`
- ☐ c) `document.getDOMById()`

SOLUCIONES:

1. ☒ a ☐ b ☐ c

2. ☐ a ☒ b ☐ c

3. ☒ a ☐ b ☐ c

4. ☐ a ☐ b ☒ c

5. ☐ a ☐ b ☒ c

6. ☒ a ☐ b ☐ c

7. ☒ a ☐ b ☐ c

8. ☐ a ☒ b ☐ c

9. ☐ a ☐ b ☒ c

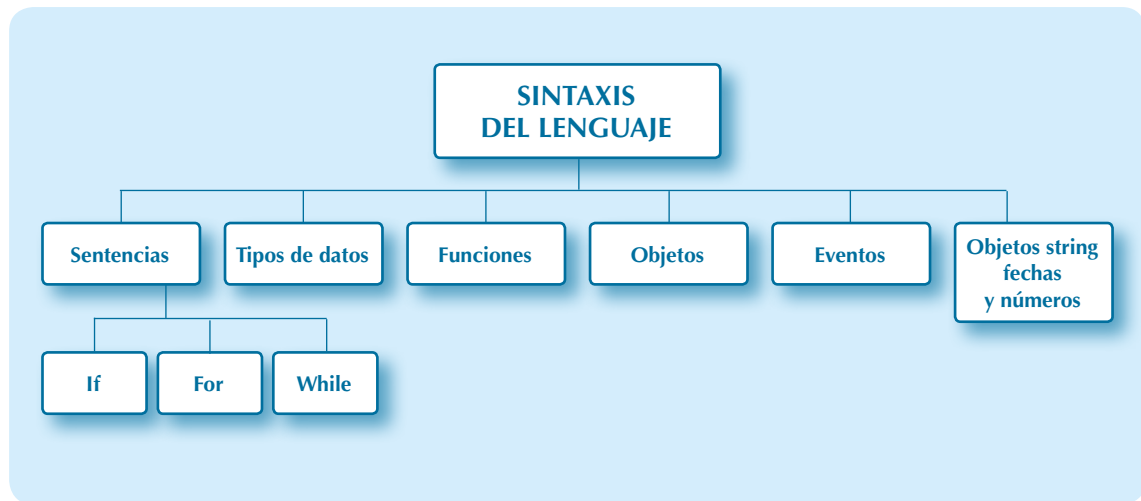
10. ☒ a ☐ b ☐ c

Sintaxis del lenguaje JavaScript

Objetivos

- ✓ Comprender la sintaxis básica de JavaScript para poder realizar pequeños scripts funcionales.
- ✓ Entender las reglas básicas del lenguaje JavaScript.
- ✓ Conocer la utilización de las sentencias básicas de JavaScript.
- ✓ Descubrir elementos propios de JavaScript como los eventos.

Mapa conceptual



Glosario

Case-sensitive. Expresión informática que se aplica a los textos en los que tiene relevancia escribir un carácter en mayúsculas o minúsculas y que significa “sensible a mayúsculas y minúsculas”.

Consola de JavaScript. Herramienta del programador para comunicar su programa JavaScript con el exterior. Se invoca pulsando generalmente la tecla F12.

ES6 o ES2015 (ECMAScript). Especificación de lenguaje JavaScript publicada por ECMA International. Los navegadores utilizan una implementación de ECMAScript y acceso al DOM para manipular las páginas web.

GMT (Greenwich Mean Time). Estándar de tiempo que se refería al tiempo solar medio en el Real Observatorio de Greenwich y que dejó de ser utilizado por la comunidad científica en 1972, cuando se reemplazó por el UTC.

Listener. Código responsable de controlar los eventos. Están a la escucha (de ahí su nombre) y, cuando ocurre un evento, se ejecuta el código que el programador haya implementado.

Operador ternario. Operador que toma tres argumentos. La ventaja que ofrece es que puede reducir varias líneas de código en una sola.

UTC (Coordinated Universal Time). Principal estándar de tiempo, que casi siempre es sinónimo de GMT, y se obtiene a través del tiempo atómico internacional.

2.1. Sintaxis del lenguaje. Operadores y palabras reservadas

La sintaxis de JavaScript es muy parecida a Java y C++. Cualquier programador que sepa programar en Java, PHP u otro lenguaje con sintaxis similar será capaz de comprender la sintaxis de JavaScript de una manera rápida. No obstante, al ser JavaScript un lenguaje de programación del lado del cliente, es importante conocer sus aspectos básicos como pueden ser los eventos, el control de los elementos HTML, etc.

RECUERDA

- ✓ Cualquier programa JavaScript puede comunicarse con la consola del navegador mediante la sentencia:

```
console.log("información para la consola");
```

En la consola, se pueden escribir literales y también valores de variables. Todos los navegadores tienen consola y es muy utilizada por los programadores con fines de depuración.

2.1.1. Las 10 reglas básicas de la sintaxis del lenguaje

A continuación, se presentan las diez reglas básicas del lenguaje JavaScript:

- *Regla 1.* Las instrucciones en JavaScript terminan en un punto y coma. Ejemplo:

```
var s = "hola";
```

- *Regla 2.* Uso de decimales en JavaScript. Los números en JavaScript que tengan decimales utilizarán el punto como separador de las unidades con la parte decimal. Ejemplos de números:

```
var x = 4;  
var pi = 3.14;
```

- *Regla 3.* Los literales se pueden escribir entre comillas dobles o simples. Ejemplo:

```
var s1 = "hola";  
var s2 = 'hola';
```

- *Regla 4.* Cuando sea necesario declarar una variable, se utilizará la palabra reservada *var*.
- *Regla 5.* El operador de asignación, al igual que en la mayoría de lenguajes, es el símbolo igual (=).
- *Regla 6.* Se pueden utilizar los siguientes operadores aritméticos: (+ - * /). Ejemplo:

```
var x = (5*4)/2+1;
```

- *Regla 7.* En las expresiones, también se pueden utilizar variables. Ejemplo:

```
var t = 4;
var x = (5*t)/2+1;
var y;
y = x * 2;
```

- *Regla 8.* Comentarios en JavaScript. Existen dos opciones para comentar el código:
 - a) `//` cuando se desea comentar el resto de la línea a partir de estas dos barras invertidas.
 - b) `/*` y `*/`. todo lo contenido entre ambas etiquetas quedará comentado.
- *Regla 9.* Los identificadores en JavaScript comienzan por una letra o la barra baja (`_`) o el símbolo del dólar (`$`).
- *Regla 10.* JavaScript es sensible a las mayúsculas y minúsculas (case-sensitive). Ejemplo:

```
var nombre = "Julio";
var Nombre = "Ramón";
```



TOMA NOTA

- *Nombre* y *nombre* son dos variables diferentes.
- Ten cuidado al escribir la palabra reservada *var*. Si escribes *Var* o *VAR*, tu código no funcionará.

2.1.2. Las palabras reservadas de JavaScript

Algunas de estas palabras reservadas se han visto ya y otras se trabajarán a lo largo del libro. En el cuadro 2.1, se muestran las más utilizadas.

CUADRO 2.1

Palabras reservadas más comunes de JavaScript

Palabra	Descripción
<code>var</code>	Utilizada para declarar una variable.
<code>if ... else</code>	Estructura condicional.
<code>for</code>	Estructura de repetición. Se ejecutará mientras la condición sea verdadera.
<code>do ... while</code>	Estructura de repetición. Se ejecutará mientras la condición sea verdadera.
<code>switch</code>	Serie de sentencias que van a ser ejecutadas dependiendo de diferentes circunstancias.

[.../...]

CUADRO 2.1 (CONT.)

break	Termina un switch o un bucle.
continue	Sale del bucle y se coloca al comienzo de este.
function	Declara una función.
return	Sale de una función.
try ... catch	Utilizadas para el manejo de excepciones.

2.1.3. Uso de variables en JavaScript

Las variables son la base de la programación. Una variable es una posición de memoria donde se pueden alojar datos. El nombre de la variable permitirá a JavaScript poder ubicar y localizar dicho espacio cuando interprete los scripts. En este apartado, se mostrará el uso de las variables en JavaScript.

FUNDAMENTAL

Constantes en JavaScript

En JavaScript, a partir de ES6 (ES2015), se puede utilizar la palabra reservada *const*. Por lo tanto, en vez de utilizar:

```
var pi = 3.141592;
```

se aconseja emplear:

```
const PI = 3.141592;
```

dado que pi es una constante (valor invariable). Por convención, se suele utilizar mayúsculas cuando se definen constantes.

En JavaScript, se pueden ejecutar las siguientes órdenes:

```
var x;  
x = 2 * x + 1;  
var pi = 3.141592;  
var paginaweb = "Myfpschool";  
var pregunta = '¿cuantos años tienes?', respuesta="veinte";  
paginaweb="Myfpschool.com";  
paginaweb="Myfpschool" + "." + "com";
```

RECUERDA

- ✓ El resultado a la derecha de una asignación se almacena en la variable del lado izquierdo de esta.

2.1.4. Operadores en JavaScript

A continuación, se muestran distintos tipos de operadores utilizados en JavaScript.

A) Operadores aritméticos

Todos los lenguajes de programación tienen operadores y, generalmente, suelen coincidir (salvo los operadores incremento y decremento que aparecieron con C++, siendo uno de sus rasgos más significativos).

CUADRO 2.2
Operadores aritméticos

Operador	Descripción
+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo
++	Incremento
--	Decremento

B) Operadores de asignación

Otros operadores básicos son los operadores de asignación. En el cuadro 2.3, se muestran los operadores de asignación de JavaScript.

CUADRO 2.3
Operadores de asignación

Operador	Ejemplo de uso
=	x = y;
+=	x += y; // igual que (x = x + y)
-=	x -= y; // igual que (x = x - y)
*=	x *= y; // igual que (x = x * y)
/=	x /= y; // igual que (x = x / y)
%=	x %= y; // igual que (x = x % y)

C) Operadores de manejo de strings

En JavaScript, se utilizan los operadores `+` y `+=` para concatenar strings. Véase un ejemplo de uso:

```
var = "hola" + " " + "mundo";
```

O lo que sería igual:

```
var = "hola";
var += " ";
var += "mundo";
```

D) Operadores lógicos y de comparación

Tanto los operadores lógicos como los de comparación son profusamente utilizados por los programadores. En el cuadro 2.4, se muestran los operadores tanto lógicos como de comparación para los programadores JavaScript.

CUADRO 2.4
Operadores lógicos y de comparación

Operador	Descripción
<code>==</code>	Igual que
<code>===</code>	Igual valor y tipo
<code>!=</code>	Distinto
<code>!==</code>	Distinto valor o distinto tipo
<code>></code>	Mayor que
<code><</code>	Menor que
<code>>=</code>	Mayor o igual que
<code><=</code>	Menor o igual que
<code>?</code>	Operador ternario

E) Operadores de tipo

Los operadores de tipo permiten conocer si un objeto es una instancia de un tipo concreto o bien conocer el tipo de una variable. A continuación, se muestran los operadores de tipo disponibles en JavaScript:

- *typeof*. Devuelve el tipo de una variable.
- *instanceof*. Devuelve true si un objeto es una instancia de un tipo de objeto.

2.2. Tipos de datos. Asignaciones y expresiones

JavaScript es un lenguaje bastante simplificado en lo que a tipos de datos se refiere y, por ello, solamente tiene cinco tipos de datos:

1. String.
2. Number.
3. Boolean.
4. Array.
5. Object.

Véase un ejemplo de utilización de cada uno de estos tipos:

```
var edad = 25; // Number
var nombre = "Dimas"; // String
var asignatura = ["lengua", "mate", "cono"]; // Array
var persona = {nombre:"Dimas", apellido:"Moreno"}; // Objeto
```

“Dimas” o “lengua” son literales y, por eso, aparecen entrecomillados. Para JavaScript, las siguientes dos líneas de código son iguales:

```
var dato = "Ronaldo " + 10;
var dato = "Ronaldo " + "10";
```

JavaScript interpretará el número como un string.

2.2.1. El valor null

Null para los lenguajes de programación es *nada* o algo que no existe. Generalmente, cuando se asigna null a una variable, es porque es o será un objeto.

Véase un ejemplo de uso del null:

```
var persona = null;
persona = {nombre:"Dimas", apellido:"Moreno"};
```

2.2.2. Conversiones entre tipos de datos en JavaScript

JavaScript es un lenguaje interpretado, por lo tanto, la conversión entre un tipo de dato y otro es transparente al programador. JavaScript asignará el tipo de datos más acertado a la variable que el programador esté utilizando.

No obstante, existen funciones de conversión como String() y Number() para convertir a cadenas de caracteres y números, respectivamente.

También existen las funciones:

- `parseInt()`. Que convierte una cadena a un número entero.
- `parseFloat()`. Que convierte una cadena a un número decimal.

Véase un ejemplo de uso de estas funciones:

```
alert("Entero: " + parseInt("9.99")); //mostrará Entero: 9
alert("Float: " + parseFloat("9.99")); //mostrará Float: 9.99
```

Es posible también convertir variables de tipo fecha a string con la función `toDateStrin()`. A continuación, se muestra un ejemplo de dicha conversión:

```
var fecha = new Date();
var cadena = fecha.toDateStrin(); // Ahora cadena contendrá la fecha
actual "Sun, 13 Jan 2019".
```

Si lo que se desea es convertir la hora en formato UTC, se puede utilizar alternativamente la función `toUTCStrin()`. Se muestra un ejemplo de dicha conversión:

```
var fecha = new Date();
var cadena = fecha.toUTCStrin(); // Ahora cadena contendrá la fecha ac-
tual en formato UTC "Sun, 13 Jan 2019 07:45:49 GMT".
```

2.3. Introducción a las funciones

Las funciones son uno de los elementos de la base de la programación estructurada. Reutilizar el código es algo básico en cualquier lenguaje de programación porque la regla que se debe seguir es “escribe una vez y utilízala tantas veces como puedas”. La variación del resultado de las funciones dependerá de sus argumentos.

Un ejemplo de estructura básica de una función es el siguiente:

```
function suma(a, b) {
    return a + b;
}
```

Como se puede ver, se tiene la palabra reservada *function* para que JavaScript interprete que se encuentra frente a una función seguida de su nombre (en el ejemplo anterior, *suma*). Esta función aceptará dos parámetros *a* y *b* y devolverá (return) la suma de ambos.



- Una función puede tener cero, uno o varios parámetros.
- El código de la función estará dentro de las llaves { y }.
- La función devolverá el resultado con la sentencia return.

Véase un ejemplo de utilización de la función anterior:

```
var c = suma(3,3); // c valdrá 6
c = suma(c,3); // ahora c valdrá 9
var text = "El valor de c será ahora: "+c;
```

Actividad propuesta 2.1



Vista la función suma, realiza la función resta, multiplicación y división.

2.4. Introducción a los objetos en JavaScript

Desde hace mucho tiempo, la programación estructurada era el único paradigma efectivo y eficiente en programación, pero las cosas cambiaron y nació la programación orientada a objetos, que rompía con lo establecido. No era una evolución, era una nueva filosofía que no tenía nada que ver con lo anterior.

Hoy en día, es prácticamente imposible programar sin utilizar programación orientada a objetos. Esta forma de programar es más cercana a cómo se expresarían las cosas en la vida real que en otros tipos de programación clásicos.

Analistas y programadores piensan y describen las realidades y el entorno de una manera distinta para plasmar dichos conceptos en programas en términos de objetos, atributos y métodos.

Véase cómo se definiría un objeto en JavaScript:

```
var perro = {  
  raza: "Podenco",  
  peso: 12,  
  altura: 58,  
  color: "negro"  
};
```

Como se puede observar, un objeto, al igual que en la vida real, puede tener muchas características o atributos, y cada atributo se verá reflejado en una variable dentro del objeto. Un objeto se diferenciará de otros por el valor de sus atributos.

También se pueden encontrar definiciones de objetos en una sola línea (aunque la forma anterior parece más legible):

```
var perro = {raza: "Podenco", peso: 12, altura: 58, color: "negro"};
```

Para acceder a los atributos de un objeto, se puede hacer de la siguiente manera:

```
perro.raza;
```

O bien:

```
perro["raza"];
```



Actividad propuesta 2.2

Crea una página web con un script que contenga un objeto de la clase persona, la cual tendrá los siguientes atributos:

- Nombre: Dimas.
- Edad: 28.
- Altura: 185.

Una vez creado el objeto, se deberá mostrar por consola: “El usuario Dimas tiene 28 años y mide 185 centímetros”.

Todo ello utilizando los atributos del objeto.

2.5. Variables y ámbitos de utilización

A continuación, se estudiará la diferencia que existe entre variables globales y locales en JavaScript. Véase un ejemplo:

```
var a; // Esta es una variable global
a=10;

function suma() {
  var b = 5;
  return a + b; // la función devolverá 15
}
```

La diferencia entre las variables *a* y *b* es que *a* puede utilizarse en otras funciones que se puedan crear, mientras que *b* solamente existirá dentro de la función *suma*.

¿Qué es lo que ocurre cuando no se declara una variable como la variable *d* en el siguiente código?

```
var a; // Esta es una variable global
a=10;
function suma() {
  var b = 5;
  c = 20;
  return a + b; // la función devolverá 15
}
```

Que será global a todo el JavaScript de esa página.

RECUERDA

- ✓ Las variables, y obviamente sus valores, desaparecen cuando se cierra la página web.

2.5.1. Las diferencias entre var y let

JavaScript permite declarar las variables con `var` y `let`. La diferencia entre ambas palabras reservadas es que, cuando se declara una variable con `var`, la variable puede utilizarse fuera del bloque donde fue declarada. Véase un ejemplo de esto:

```
var a = 10; // x vale 10
{
  let a = 5; // a vale 5
}
// aquí a vale 10
{
  let b=3;
}
// aquí no se puede utilizar b
{
  var c=7;
}
// aquí sí se puede utilizar c y valdrá 7.
```

🎯 FUNDAMENTAL

Las palabras reservadas `let` y `const`

El estándar ES2015 añadió las palabras clave *let* y *const*. La primera se utiliza para declarar variables en un bloque determinado y la segunda, para definir constantes. Anteriormente, en JavaScript, las variables estaban definidas a nivel global o a nivel de función.

2.6. Eventos

Los eventos son cualquier suceso que le pueda ocurrir a un elemento HTML. Como se puede suponer, JavaScript puede darse cuenta de ese evento y reaccionar a este ejecutando el código que se haya programado.

Algunos eventos que pueden ocurrir en una página web son, entre otros:

- Pulsar un botón.
- Modificar un campo de texto.
- Pulsar una tecla.
- La página ha terminado de cargarse.
- Hacer clic sobre un elemento HTML.

El formato de programar un evento en JavaScript sería el siguiente:

```
<Elemento_HTML evento='código_JavaScript'>
```

También es posible cambiar las comillas simples por comillas dobles. Se pueden utilizar de forma indistinta.

Véase un ejemplo de evento en JavaScript programado para un botón:

```
<button onclick='this.innerHTML=Date()>Pulsa para saber la hora</button>
```

2.6.1. Tipos de eventos en JavaScript

En un navegador, hay muchos tipos de eventos que pueden ser controlados por JavaScript. En los siguientes cuadros, se muestra una clasificación de los distintos tipos de eventos clasificados por campos de texto, ratón y carga:

CUADRO 2.5
Eventos en campos de texto

Evento	Ejecutado
onblur	Cuando se abandona un campo de entrada.
onchange	Cuando se cambia el contenido de un campo de entrada o cuando un usuario selecciona un valor de una lista desplegable.
onfocus	Cuando obtiene el foco un campo de entrada.
onselect	Cuando se selecciona el texto de entrada.
onsubmit	Cuando se hace clic en el botón de enviar.
onreset	Cuando se hace clic en el botón de reinicio.
onkeydown onkeypress	Cuando se presiona o mantiene presionada una tecla.
onkeyup	Cuando se deja de presionar una tecla.

CUADRO 2.6
Eventos relativos al ratón

Evento	Ejecutado
onclick	Cuando se hace clic en un botón.
ondblclick	Cuando se hace doble clic en un texto.
onmouseover onmouseout	Cuando el ratón pasa sobre un elemento.
onmousedown onmouseup	Al presionar o soltar un botón del ratón.
onmousemove	Al mover el puntero del ratón sobre de una imagen.
onmouseout	Al mover el puntero del ratón fuera de una imagen.
onmouseover	Al mover el ratón sobre una imagen.
onmouseout	Al mover el ratón fuera de una imagen.

CUADRO 2.7
Eventos de carga

Evento	Ejecutado
onload	Cuando se carga una imagen o una página.
onerror	Cuando se produce un error al cargar una imagen.
onunload	Cuando el navegador cierra el documento (página web).
onresize	Cuando se cambia el tamaño de la ventana del navegador.

2.6.2. Los listener

Otra forma de añadir los eventos a los elementos u objetos HTML del DOM (listeners):

```
<!DOCTYPE html>
<html>
<body>

<p id="myFPtext">texto a modificar</p>
<button id="myFPboton">Dale fuerte!</button>

<script>

function dale(){
var x = document.getElementById("myFPtext");
x.innerHTML = "hola";
}

document.getElementById("myFPboton").addEventListener("click", dale,
false);

</script>

</body>
</html>
```

Como se puede observar, en el código anterior se utiliza el método `addEventListener` para vincular un evento a una función JavaScript. Esta forma de programar los listeners es más limpia y clara.

Los parámetros de la función `addEventListener()` son los siguientes:

1. El evento (sin el "on").
2. La función por ejecutar.
3. `useCapture`. Se utilizará si el usuario quiere iniciar la captura para que todos los eventos de ese tipo se lancen en el mismo listener. Generalmente, se utiliza `false` en este parámetro.

2.7. Objeto string o cadena de caracteres

En anteriores apartados, se han visto ejemplos de funcionamiento de las cadenas de caracteres o strings:

```
var web = "myfpschool.com";  
var web = 'myfpschool.com';
```

Como ya se ha explicado, se podían utilizar comillas dobles y simples para definir valores a los string.

En el caso de que se quieran utilizar las comillas dentro de un string, se necesitará utilizar secuencias de escape. Un ejemplo de introducción de una barra invertida dentro de una cadena sería el siguiente:

```
var web = "la mejor \"web\" de tecnología";
```

El contenido del string web anterior sería: la mejor “web” de tecnología.

TOMA NOTA



Es posible incluir tabulación (\t) y saltos de línea (\n) dentro de las cadenas de caracteres. Ejemplo:

```
alert("hola \tme llamo \nCarlos");
```

2.8. Números

A continuación, se explicarán particularidades sobre los números en JavaScript.

2.8.1. Convertir un número en un string

En JavaScript, al igual que en otros lenguajes, existe el método toString(), el cual convierte un número en una cadena de caracteres.

Véase un ejemplo de su uso:

```
var num = 999;  
num.toString(); // devolverá 999 pero como cadena de caracteres  
(888).toString(); // devuelve 888 que lo toma del literal 888  
(888 + 111).toString(); // realiza la operación y devuelve 999 pero como  
cadena de caracteres
```

2.8.2. Ajuste de decimales. Número de decimales de un número

En JavaScript, el método `toFixed()` servirá para ajustar el número de decimales de un número.

```
var num = 9.99;
num.toFixed(0); // devolverá 10. 0 lugares decimales. Realiza redondeo.
num.toFixed(1); // devolverá 10.0 . 1 lugar decimal. Realiza redondeo.
num.toFixed(2); // devolverá 9.99. 0 lugares decimales.
num.toFixed(3); // devolverá 9.990. 0 lugares decimales.
```

2.8.3. Precisión de un número

En JavaScript, el método `toPrecision()` servirá para ajustar la precisión de un número:

```
var num = 9.99;
num.toPrecision(); // devolverá 9.99.
num.toPrecision(1); // devolverá 1e+1 .
num.toPrecision(2); // devolverá 10.
num.toPrecision(3); // devolverá 9.99.
```

2.8.4. Convertir cualquier variable en un número

La función `Number()` permite convertir cualquier variable en un número. Véase unos ejemplos de uso:

```
Number(true); // devuelve 1
var num = false;
Number(num); // devuelve 0
```

2.9. Fechas

Las fechas en JavaScript se pueden visualizar como un número o un string. Véase un ejemplo de una fecha visualizada como cadena de caracteres:

```
<p id="fecha"></p>
<script>
document.getElementById("fecha").innerHTML = Date();
</script>
```

El resultado de visualizar la fecha actual con el script anterior sería algo similar a:

Mon Jun 27 2019 10:45:12 GMT+0200 (CEST)

2.9.1. Creación de un objeto de tipo fecha

En JavaScript, existen cuatro formas diferentes (cuatro constructores diferentes) de inicializar un objeto de tipo fecha:

```
new Date()  
new Date(milisegundos)  
new Date(fechaString)  
new Date(año, mes, día, horas, minutos, segundos, milisegundos)
```

Véanse algunas formas de utilizar estos constructores:

```
var v = new Date(99,8,16,10,25,12,0);  
var v = new Date(92500000);  
var v = new Date();
```



SABÍAS QUE...

Para transformar una fecha en string, debe utilizarse el método `toString()`.

2.9.2. Formatos de tipo fecha

En JavaScript, existen distintos formatos de tipo fecha:

- Formato corto.
- Formato largo.
- Formato completo.
- Formato ISO.

Véase un ejemplo de cada uno de estos tipos:

```
var v = new Date("08/16/2016"); // Formato corto  
var v = new Date("Ago 16 2016"); // Formato largo  
var v = new Date("Fri Ago 16 2019 12:15:24 GMT+0100 (W. Europe Standard Time)"); // Formato completo  
var v = new Date("2019-08-16"); // Formato ISO
```

2.9.3. Métodos del tipo fecha

De nada sirve un objeto de tipo fecha sin sus correspondientes métodos que lo hagan potente y versátil. En JavaScript, los objetos tipo fecha tienen los siguientes métodos que pueden manejarse de una manera eficiente en cualquier script:

- `getDate()`. Obtiene el día del mes (1-31).

- `getDay()`. Obtiene el día de la semana en formato numérico (0-6).
- `getFullYear()`. Obtiene el año (2016, por ejemplo).
- `getHours()`.
- `getMilliseconds()`.
- `getMinutes()`.
- `getMonth()`.
- `getSeconds()`.
- `getTime()`. Obtiene la hora en milisegundos desde el 1 de enero de 1970.

2.10. Arrays

Los arrays o vectores son una serie de elementos u objetos contiguos, a los cuales pueden accederse utilizando un subíndice. Prácticamente todos los lenguajes de programación utilizan arrays por su utilidad.

A continuación, se muestra un ejemplo de utilización de arrays dentro de una página web:

```
<p id="aqui"></p>
<script>
var dias = [«lunes», «martes», «miércoles», «jueves», «viernes», «sábado»,
«domingo»];
document.getElementById(«aqui»).innerHTML = dias;
</script>
```

También se podría haber creado el array de la siguiente forma:

```
var dias = new Array («lunes», «martes», «miércoles», «jueves», «viernes»,
«sábado», «domingo»);
```

🎯 FUNDAMENTAL

¿Se pueden tener objetos de diferentes tipos en un array?

Para comprobar que funciona, basta con intentar hacer lo siguiente en el ejemplo anterior:

```
dias[1]=Date();
```

Actividad propuesta 2.3



En caso de que funcione el ejemplo de este apartado, cambia el formato de la fecha para que aparezca como el siguiente: dd/mm/yyyy.

2.10.1. Operaciones con arrays

Existen múltiples operaciones en JavaScript para manipular arrays. A continuación, se mostrarán las distintas posibilidades del lenguaje:

- Conocer su longitud:

```
longitud = dias.length;
```

- Ordenarlo en alfabéticamente:

```
dias = dias.sort();
```

- Añadir un nuevo elemento:

```
dias = dias.push("juernes");
```

- Extraer el último elemento:

```
ultimo = dias.pop();
```

El último elemento se eliminará del array. *Extraer* significa “retirar”.

- Eliminar el primer elemento:

```
dias.shift();
```

- Eliminar un elemento concreto:

```
delete dias[2];
```

2.10.2. Arrays multidimensionales

Los arrays multidimensionales consisten en tener en una posición de un array otro array de elementos. Se pueden tener arrays bidimensionales, tridimensionales, etc. Lo más común en programación es tener tablas, también llamados *arrays bidimensionales*.

A continuación, se muestra un ejemplo de creación y acceso a un array bidimensional. Como se puede ver, se han separado los días laborables del fin de semana en los dos elementos del array principal:

```
var dias = [ ["lunes", "martes", "miércoles", "jueves", "viernes"], ["sá-  
bado", "domingo"] ];  
console.log(dias[0][2]); //mostrará por consola miércoles  
console.log(dias[1][1]); //mostrará por consola domingo  
console.log(dias[1]); //mostrará por consola sábado, domingo
```

RECUERDA

- ✓ El acceso y recorrido de un array con varias dimensiones es igual a un array con solo una. Únicamente hay que tener en cuenta que hay que detallar la posición en todas y cada una de las dimensiones para acceder al elemento o elementos pretendidos.

2.11. Sentencias condicionales

JavaScript tiene diferentes sentencias condicionales. A continuación, se citarán cada una de ellas:

- *If*. Se ejecutará el bloque de código siguiente si la condición es verdadera.
- *Else*. Se ejecutará el bloque de código siguiente si la condición es falsa.
- *Else if*. Es combinación de las anteriores y se utilizará cuando la primera condición sea falsa.
- *Switch*. Se utilizará para especificar una serie de códigos que se ejecutarán de forma alternativa.

Todo esto se puede apreciar mejor con ejemplos:

- La sentencia IF:

```
if (new Date().getHours() < 18) {  
    saludo = "Buenos días";  
}
```

El código anterior hace que la variable saludo valga "Buenos días" si es antes de las 18:00.

- La sentencia ELSE:

```
if (new Date().getHours() < 18) {  
    saludo = "Buenos días";  
}else{  
    saludo = "Buenas tardes";  
}
```

El código anterior hace que la variable saludo valga "Buenos días" si es antes de las 18:00 y "Buenas tardes" en caso contrario.

- La sentencia ELSE IF:

```
if (new Date().getHours() < 10) {  
    saludo = "Al que madruga Dios le ayuda";  
}else if (new Date().getHours() < 18) {  
    saludo = "Buenos días";  
}else {  
    saludo = "Buenas tardes";  
}
```

El código anterior hace que la variable saludo valga “Al que madruga Dios le ayuda” si la hora es antes de las 10 de la mañana, “Buenos días” si es antes de las 18:00 y “Buenas tardes” en caso contrario.

- La sentencia SWITCH:

```
var = new Date().getDay();
switch (var) {
    case 0:
        dia = "Domingo";
        break;
    case 1:
        dia = "Lunes";
        break;
    case 2:
        dia = "Martes";
        break;
    case 3:
        dia = "Miércoles";
        break;
    case 4:
        dia = "Jueves";
        break;
    case 5:
        dia = "Viernes";
        break;
    case 6:
        dia = "Sábado";
    default:
        dia = " - ";
}
```

Como se puede ver, dependiendo del valor de la variable var, la variable dia tendrá un valor u otro en función del día de la semana en el que se esté.

TOMA NOTA



- Se utiliza la sentencia break para salir de cada bloque del switch.
- La sentencia default se ejecutará si ninguna de las opciones anteriores ha sido ejecutada con las opciones case.

2.12. Bucles

En cualquier lenguaje de programación, se necesita ejecutar un trozo de código repetidas veces. Para ello, se usan los bucles y los hay de distintos tipos, los comunes a todos los lenguajes son el bucle for y while. A continuación, se hará un repaso de cada una de ellos:

- Bucle for*. Se utiliza esta sentencia cuando se quiere ejecutar un bloque de código una serie de veces determinada (generalmente, se conoce de antemano el número de veces).
- Bucle while*. Se emplea esta sentencia cuando se desea ejecutar un bloque de código una serie de veces sin conocer de antemano cuántas. El bloque de código se ejecutará mientras se cumpla una condición determinada.

A continuación, se muestran algunos ejemplos de uso:

- La sentencia o bucle FOR:

```
for (i = 0; i <= 10; i++) {  
    text += "Número: " + i + " ";  
}
```

En el bucle for, se inicializa la variable *i* a 0 (*i*=0). Dicho bucle se ejecutará mientras que la variable *i* sea menor o igual que 10 (*i*<=10) y la variable *i* a su vez se irá incrementando en una unidad (*i*++) en cada pasada del bucle.

Existe una variante del bucle for que es el bucle for in. A continuación, se muestra un ejemplo sencillo de comprender:

```
var persona = {nombre:"Dimas", apellidos:"Moreno", edad:25};  
var texto = "";  
var x;  
for (x in persona) {  
    texto += persona[x];  
}
```



PARA SABER MÁS

Es posible crear un bucle decreciente estableciendo los parámetros de la cabecera con ese propósito. A continuación, se muestra una versión decreciente del bucle mostrado anteriormente:

```
for (i = 10; i >= 0; i--) {  
    text += "Número: " + i + " ";  
}
```

Como se puede ver en el ejemplo anterior, el valor de *i* configurado a 10, en un principio, irá decreciendo y el bucle se ejecutará siempre y cuando el valor de la variable *i* sea mayor o igual que 0.

- La sentencia o bucle WHILE:

```
i = 0;
while (i <= 10) {
    text += "Número " + i;
    i++;
}
```

Como puede observarse, es el anterior ejemplo del bucle for, pero ahora reconvertido a while.

2.12.1. Recorrido de un array

Existen dos formas de recorrer un array:

1. *Mediante el recorrido clásico.* Se utilizará un bucle para recorrer el array. Véase un ejemplo de uso:

```
for (i=0;i<dias.length;i++){
    console.log(dias[i]);
}
```

2. *Mediante el método `forEach()`.* Véase un ejemplo de uso:

```
var dias = ['lunes', 'martes', 'miércoles', 'jueves', 'viernes', 'sábado',
'domingo'];
dias.forEach(muestra);
```

Donde *muestra* es una función que será invocada por cada uno de los elementos del array *dias*.

A continuación, se muestra un ejemplo en el que se recorre el array de las dos formas citadas anteriormente:

```
<!DOCTYPE html>
<html>
<body>
<h3>Recorrido de un array</h3>
<p id=»aquí«></p>
<script>
var txt = «»;
var txt2 = «»;
var dias = ['lunes', 'martes', 'miércoles', 'jueves', 'viernes', 'sábado',
'domingo'];
dias.forEach(muestra);
document.getElementById(«aquí»).innerHTML = txt;

function muestra(valor, index, array) {
    txt = txt + valor + « »;
}
```

```

document.getElementById(«aqui»).innerHTML += '<br>-----<br>';
for (i=0;i<dias.length;i++){
    txt2 = txt2 + dias[i] + «    »;
}
document.getElementById(«aqui»).innerHTML += txt2;

</script>
</body>
</html>

```

El aspecto en el navegador del ejemplo anterior será el que se muestra observa en la figura 2.1.

Recorrido de un array

```

lunes martes miércoles jueves viernes sábado domingo
-----
lunes martes miércoles jueves viernes sábado domingo

```

Figura 2.1
Aspecto en el navegador
del recorrido de un array.

Actividad propuesta 2.4



Verifica que funciona el código empleado en este apartado en un navegador y muestra el resultado anteriormente expuesto. Al mismo tiempo, modifica el código y añade variantes de tu cosecha para intentar conocer mejor cómo funcionan los arrays y los bucles.

2.13. Práctica guiada

A continuación, utilizando bucles, se realiza un script que muestre en una página web lo que se observa en la figura 2.2.

```

      *
    ***
  *****
*****
  *****
    ***
      *

```

Figura 2.2
Rombo.

Se trata de crear un rombo como el de la figura 2.2 respetando los espacios y la forma en lo posible. Recuerda que, para el espacio, hay que utilizar el símbolo HTML ` ` y, para el salto de línea, la etiqueta `<br>`.

Se va a ir explicando la solución paso a paso. Ten en cuenta que se incluye el JavaScript dentro del HTML para una mejor comprensión del ejercicio, aunque, en un proyecto real, habría que tenerlo separado.

Para solucionar el ejercicio, se ha dividido el rombo en dos partes: una parte superior en la que va creciendo el número de asteriscos y otra parte inferior en la que el número de asteriscos es decreciente.

En las variables *asterisco* y *espacioHTML*, se van a almacenar respectivamente los asteriscos y los espacios HTML que han de mostrarse respectivamente. Si no se incluyen espacios el rombo no tendrá la forma que se desea conseguir.

En una variable de nombre *límite*, se registra el número de filas (4) que va a tener la parte creciente (la decreciente tendrá una menos).

El código del ejercicio será el siguiente:

```
<!DOCTYPE html>

<html>

<head>
  <title>Ejercicio Rombo</title>
</head>

<body>

<div id="caja"></div>

<script>

  var caja = document.getElementById("caja");

  var limite = 4;

  //Creciente
  for (var i = 0; i < limite; i++) {

    var asterisco = "";
    var espacioHTML = "";

    for (var k = 0; k < limite - i - 1; k++) {
      espacioHTML += "&nbsp;&nbsp;&nbsp;";
    }

    for (var j = 1; j <= 2 * i + 1; j++) {
      asterisco += "*";
    }

    caja.innerHTML = caja.innerHTML + espacioHTML + asterisco + "<br>";
  }
}
```

```

//Decreciente
for (var i = limite-2; i >= 0; i--) {
    var asterisco = "";
    var espacioHTML = "";

    for (var k = 0; k < limite - i - 1; k++) {
        espacioHTML += "&nbsp;&nbsp;&nbsp;";
    }

    for (var j = 1; j <= 2 * i + 1; j++) {
        asterisco += "*";
    }

    caja.innerHTML = caja.innerHTML + espacioHTML + asterisco+"<br>";
}

</script>

</body>

</html>

```

Ahora se pasará a analizar la parte creciente y decreciente del rombo:

- *Parte creciente.* En esta parte, se ejecuta un bucle creciente y se observa que el código más complejo son los bucles que almacenan los espacios HTML y los asteriscos en las variables `espacioHTML` y `asterisco`.

En el caso de los espacios, estos van a ir decreciendo, dado que se establece en el bucle que la variable índice k sea menor que $\text{limite} - i - 1$ (siendo i el índice del bucle principal).

```

for (var k = 0; k < limite - i - 1; k++) {
    espacioHTML += "&nbsp;&nbsp;&nbsp;";
}

```

En el caso de los asteriscos, es todo lo contrario. Al decrecer los espacios, crecen los asteriscos y, por tanto, el índice j del bucle será menor o igual a $2 * i + 1$.

```

for (var j = 1; j <= 2 * i + 1; j++) {
    asterisco += "*";
}

```

- *Parte decreciente.* Es igual a la parte creciente, pero se observa que, en este caso, el bucle principal tiene que ser decreciente. El bucle comienza en $\text{limite}-2$, dado que se va a ejecutar solamente tres veces con valores del índice i igual a 2, 1 y 0.

```

for (var i = limite-2; i >= 0; i--) {

```

Es importante que el lector estudie con detenimiento la cabecera de los bucles, puesto que es la base y la complejidad del ejercicio.



Actividad propuesta 2.5

Utiliza el código anterior y comprueba cómo se muestra un rombo en el navegador. Si puedes, añade alguna característica complementaria, como que el rombo pueda desplazarse un número de espacios del margen izquierdo. Ese número de espacios podría ser configurable por el usuario.

Resumen

- Los programadores utilizan la sentencia `console.log()` para escribir información por consola.
- El separador de decimales en JavaScript es el punto y no la coma.
- JavaScript es sensible a las mayúsculas y minúsculas (case-sensitive).
- Es importante no poner a una variable el nombre *var*, puesto que es una palabra reservada.
- Los operadores en JavaScript son similares a Java.
- El operador `typeof` devuelve el tipo de una variable.
- JavaScript tiene solamente cinco tipos de datos: `string`, `number`, `boolean`, `array` y `object`.
- Se utiliza `null` en los objetos para indicar que no existen o que no tienen todavía un valor asignado.
- Existen varias funciones de conversión de datos como `String()`, `Number()`, `parseInt()`, `parseFloat()`, `toDateString()` o `toUTCString()`.
- Las funciones en JavaScript comienzan con la palabra reservada *function*.
- Un ejemplo de objeto es el siguiente:

```
var perro = {raza:"Podenco", peso:12, altura:58,color:"negro"};
```

- Es importante el ámbito de utilización de una variable. Se recomienda no declarar funciones globales cuando solamente se van a utilizar en una función.
- Los eventos son cualquier suceso que le pueda ocurrir a un elemento HTML. El formato de programar un evento en JavaScript sería el siguiente:

```
<Elemento_HTML evento='código_JavaScript'>
```

- Con el método `addEventListener`, se puede vincular un evento a una función JavaScript.
- Con el método `toString()`, se puede convertir, por ejemplo, un número en una cadena de caracteres.
- En JavaScript, se pueden crear objetos de tipo fecha con el objeto `Date`, el cual tiene muchos constructores.

- El objeto Date tiene muchos métodos para obtener el día, la hora, el año, etc., de una fecha concreta.
- JavaScript tiene diferentes tipos de sentencias condicionales como if, else, else if o switch.
- En JavaScript, existen los bucles for y while.



Ejercicios propuestos

1. Realiza un programa JavaScript que calcule el área y la longitud de una circunferencia de radio 5.
2. Investiga el operador ternario y pon un ejemplo completo de uso.
3. Ejecuta las siguientes dos líneas de código y comprueba el contenido de la variable var. Interpreta y explica los resultados:

```
var dato = "Ronaldo " + 5 + 5;  
var dato = 5 + 5 + " Ronaldo ";
```

4. El operador typeof permite saber el tipo de una variable. Crea un programa en el que pongas ejemplos de su utilización.
5. Crea un script con algún ejemplo de cómo se pueden sobrescribir variables y funciones globales.
6. JavaScript puede manejar eventos en una página web para conocer, por ejemplo, cuándo el ratón pasa por encima de un elemento. Pon un ejemplo de uso para cada uno de los siguientes eventos:

- onmouseover.
- onmouseout.
- onblur.

7. Averigua qué resultado darán las siguientes líneas de código:

```
Number("9 9");  
Number("dos");
```

8. Realiza un script que extraiga el mes y el año de una fecha concreta.
9. Romualdo, que todavía no ha terminado el ciclo de DAW, se pregunta si puede crear el siguiente array:

```
var milista = ['panadero', 456, [0, 1, 2]];
```

También quiere saber si podría seleccionar la siguiente posición del array:

```
milista[2][2]
```

Comprueba si puede utilizarlo e intenta hacer una pequeña web que muestre todo el contenido del array milista (si es posible).

10. Como variante de la práctica guiada, se pide realizar un script utilizando bucles que muestre en una página web el rombo de la figura 2.3. El número de filas que tiene el rombo será configurable.

Figura 2.3
Rombo hueco
configurable.



Nota: aunque se puede realizar este ejercicio con todo lo aprendido en este capítulo, la función `repeat()` que se explica en el apartado 3.6.4 puede reducir y simplificar mucho el código resultante al eliminar y resumir algunos bucles.

ACTIVIDADES DE AUTOEVALUACIÓN

1. ¿Qué función se utiliza para escribir un mensaje por consola?:

- ☐ a) `console.log()`
- ☐ b) `console.alert()`
- ☐ c) `alert()`

2. ¿Cuál de las siguientes instrucciones no es correcta?:

- ☐ a) `var s = "hola";`
- ☐ b) `var s = 'hola';`
- ☐ c) `var pi = 3,14;`

3. Teniendo las siguientes líneas de código, la variable `x` valdrá:

```
var x = 10;
x %= 5;
```

- ☐ a) 5.
- ☐ b) 2.
- ☐ c) 0.

4. En JavaScript, la siguiente sentencia hará que `dato` valga:

```
var dato = "Ronaldo " + 10;
```

- ☐ a) "Ronaldo 10".
- ☐ b) "Ronaldo".
- ☐ c) Dará un error.

5. ¿Qué sintaxis se utilizará para definir una función suma en Javascript?:
- ☐ a) `suma(a, b) {`
 - ☐ b) `function suma(a, b) {`
 - ☐ c) `function suma[a, b] {`
6. ¿Cuál de los siguientes sería el formato de programar un evento en JavaScript?:
- ☐ a) `<Elemento_HTML evento='código_JavaScript'>`
 - ☐ b) `<Elemento_HTML evento={código_JavaScript}>`
 - ☐ c) `<Elemento_HTML evento=[código_JavaScript]>`
7. Cuando se abandona un campo de entrada, ¿cuál de los siguientes eventos se ejecuta?:
- ☐ a) `onleave`.
 - ☐ b) `onblur`.
 - ☐ c) `onchange`.
8. Para vincular un evento clic a un objeto HTML llamado *myFPboton* para que se ejecute la función *dale*, ¿cuál de los siguientes listeners habrá que añadir?:
- ☐ a) `document.getElementById("myFPboton").addEventListener("click", dale, false);`
 - ☐ b) `document.getElementById("myFPboton").addEventListener("onclick", dale, false);`
 - ☐ c) `document.getElementById("myFPboton").addEventListener("onclick", dale, false);`
9. En las siguientes líneas de código, la variable *num2* valdrá:
- ```
var num = 9.99;
var num2 = num.toPrecision(1);
```
- ☐ a) 10.
  - ☐ b) 1e+1.
  - ☐ c) 9.99.
10. ¿Cuál de las siguientes afirmaciones no es correcta?:
- ☐ a) Se pueden tener objetos de diferentes tipos en un array.
  - ☐ b) `Dias.pop()` retira el primer elemento del array *Dias*.
  - ☐ c) `Dias.pop()` elimina el primer elemento del array *Dias*.

### SOLUCIONES:

1. ☒ a ☐ b ☐ c  
 2. ☐ a ☐ b ☒ c  
 3. ☐ a ☐ b ☒ c  
 4. ☒ a ☐ b ☐ c

5. ☐ a ☒ b ☐ c  
 6. ☒ a ☐ b ☐ c  
 7. ☐ a ☒ b ☐ c  
 8. ☒ a ☐ b ☐ c

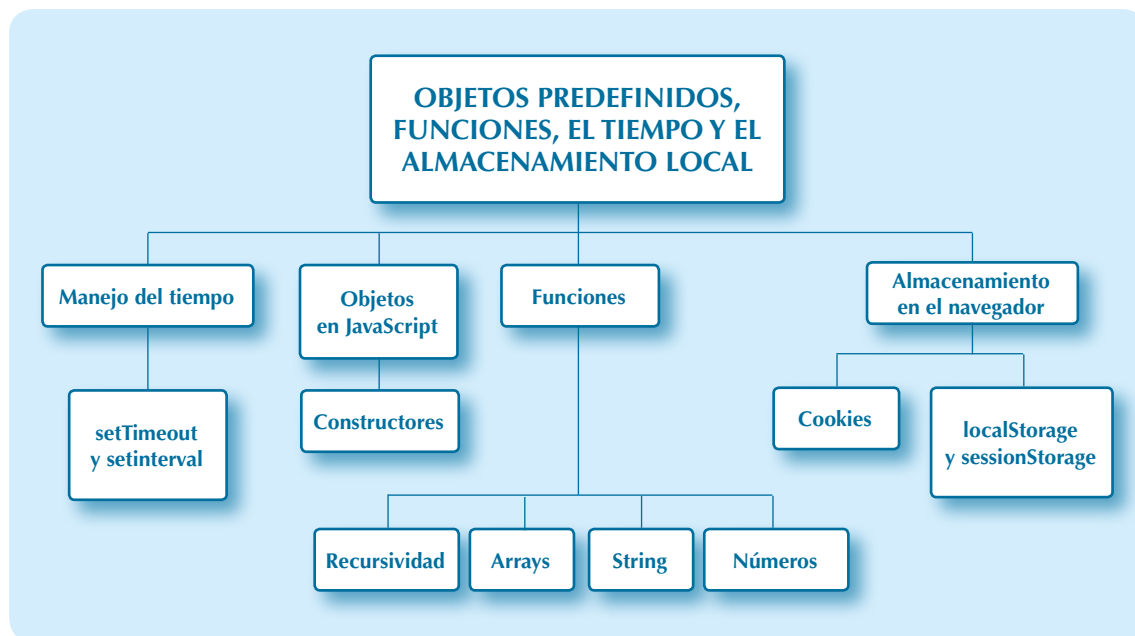
9. ☐ a ☒ b ☐ c  
 10. ☐ a ☒ b ☐ c

# Utilización de los objetos predefinidos del lenguaje

## Objetivos

- ✓ Aprender cómo se puede manejar el tiempo en JavaScript.
- ✓ Saber cómo se programa la ejecución de funciones de forma puntual y periódica en el tiempo.
- ✓ Registrar en el front-end información de navegación, usuario, etc., mediante cookies o mediante el almacenamiento local que ofrece el nuevo estándar HTML5.
- ✓ Profundizar en el concepto de objeto.
- ✓ Conocer y manejar funciones relativas al lenguaje sobre arrays, strings, números.

## Mapa conceptual



## Glosario

**Backtracking.** Estrategia utilizada en algoritmos que resuelven problemas que tienen ciertas restricciones. Este término fue creado por primera vez por el matemático D. H. Lehmer en la década de los cincuenta.

**BOM (Browser Object Model).** Convención específica implementada por los navegadores para que JavaScript pudiese hacer uso de sus métodos y propiedades de forma uniforme.

**Expresión regular.** Secuencia de caracteres que forman un patrón determinado, generalmente un patrón de búsqueda.

**NaN.** Propiedad que indica Not a Number (valor no numérico).

**Objeto window.** Aquel que soportan todos los navegadores y que representa la ventana del navegador. Se estudiará en profundidad en capítulos posteriores.

**Sudoku.** Juego matemático que ha sido muy popular en Japón. El objetivo es rellenar una cuadrícula de  $9 \times 9$  celdas dividida a su vez en 9 cuadrículas de  $3 \times 3$  celdas con números diferentes del 1 al 9.

**URI (Uniform Resource Identifier).** Cadena de caracteres que identifica un recurso en una red de forma unívoca. Una URI puede ser una URL, una URN o ambas.

**URN.** Localizador de recursos en la web que funciona de forma parecida a una URL, pero su principal diferencia es que no indica exactamente dónde se encuentra dicho objeto.

### 3.1. Introducción

En el capítulo 2, se había hecho un repaso de la sintaxis del lenguaje JavaScript. En este capítulo, se profundiza en conceptos como los *objetos* y las *funciones*. En JavaScript, existen muchas funciones sobre string, números, etc., que ahorran mucho tiempo al programador y que es preciso que maneje con destreza.

Otro concepto que se trabaja en este capítulo es el almacenamiento local. Se verán las cookies, que era la forma que tenía el navegador de almacenar en el equipo información del usuario, y el almacenamiento local que, a partir de HTML5, ha desbancado a las primeras.

Además de lo citado anteriormente, es importante que el programador sepa cómo manejar el tiempo en JavaScript y, para ello, se explican en profundidad el objeto Date y sus funciones asociadas, así como las funciones setTimeout y setInterval para programar la ejecución de funciones de forma puntual y periódica.

### 3.2. Manejo del tiempo en JavaScript

JavaScript es la manera más útil y eficiente de trabajar con el tiempo en un navegador, dado que no recarga el servidor y tiene el objeto interno Date —que puede ser de suma utilidad al programador para definir contadores, relojes, etc.—. Además, permite actualizar el reloj mediante algunas funciones que se detallarán en este capítulo.

#### FUNDAMENTAL

##### *El BOM y el tiempo*

El BOM (Browser Object Model), más concretamente el objeto window, proporciona los métodos setTimeout() y setInterval() que permiten manejar el tiempo dentro del navegador.

El objeto Date se creó en JavaScript para almacenar fechas y horas. Una vez creado dicho objeto, es posible modificarlo y realizar los cálculos que el programador crea convenientes para modificar las fechas y las horas.

La creación de este objeto se podría hacer de la siguiente forma:

```
var miFecha = new Date();
```

Ahora, la variable miFecha contendrá la fecha y hora actual. Si se desea mostrar la fecha y la hora actuales en un campo de una página web, es tan fácil como utilizar el siguiente código:

```
<div id="laHora"></div>
<script>
var miFecha = new Date();
var texto = document.getElementById('laHora');
texto.innerHTML=miFecha;
</script>
```

El campo laHora, en el momento de la ejecución, podrá tener un valor parecido al siguiente:

Mon Jan 07 2019 17:14:45 GMT+0100 (hora estándar de Europa central)

El problema es que el aspecto de la hora que se muestra no es el que normalmente se visualiza en una página web y, por lo tanto, hay que utilizar algunas funciones (métodos) para mostrar la hora, los minutos y los segundos como son:

- *getHours()*. Método que extrae las horas del objeto Date actual.
- *getMinutes()*. Método que extrae los minutos del objeto Date actual.
- *getSeconds()*. Método que extrae los segundos del objeto Date actual.

El siguiente ejemplo sí mostrará la hora en un formato más convencional:

```
<div id="laHora"></div>
<script>
 var miFecha = new Date();
 var texto = document.getElementById('laHora');
 texto.innerHTML=miFecha.getHours() + ":"+miFecha.getMinutes() + ":"
 + miFecha.getSeconds();
</script>
```

El resultado puede ser parecido al siguiente:

18:46:58

El problema que se puede encontrar es que las funciones anteriores (*getHours*, *getMinutes* y *getSeconds*) no devuelven dos dígitos siempre. Si, por ejemplo, la función *getSeconds* tiene que devolver tres segundos, devolverá el número 3 y no 03, con lo cual habrá que modificar o mejorar el código anterior:

```
<div id="laHora"></div>
<script>
 var miFecha = new Date();
 var horas = miFecha.getHours();
 var minutos = miFecha.getMinutes();
 var segundos = miFecha.getSeconds();
 if (horas<10){horas='0'+horas;}
 if (minutos<10){minutos='0'+minutos;}
 if (segundos<10){segundos='0'+segundos;}

 var texto = document.getElementById('laHora');
 texto.innerHTML=horas+':'+minutos+':'+segundos;
</script>
```

En este momento, el reloj implementado mostrará siempre la hora de forma más homogénea.

Imagínese que se quiere mostrar la hora en formato 12 horas en vez de en el de 24, como se estaba haciendo. Habrá que modificar el código de la siguiente manera:

```

<div id="laHora"></div>
<script>
 var miFecha = new Date();
 var horas = miFecha.getHours();
 var minutos = miFecha.getMinutes();
 var segundos = miFecha.getSeconds();
 if (horas > 12){
 ampm='pm';
 horas-=12;
 }else{
 ampm='am';
 }
 if (horas<10){horas='0'+horas;}
 if (minutos<10){minutos='0'+minutos;}
 if (segundos<10){segundos='0'+segundos;}

 var texto = document.getElementById('laHora');
 texto.innerHTML=horas+':'+minutos+':'+segundos+' '+ampm;
</script>

```

Como se puede observar, lo que se hace es comprobar si son más de las 12 o no y se ajustan las horas colocando el sufijo correspondiente, ya sea *am* o *pm*.

La hora ahora podrá mostrarse en un formato parecido al siguiente:

06:46:58 pm

### 3.2.1. Las funciones `setTimeout` y `setInterval`

Para realizar una mejora sustancial al ejercicio desarrollado anteriormente, JavaScript ofrece las siguientes funciones:

- *setTimeout* (*Función\_a\_llamar, milisegundos*). Ejecutará la función *Función\_a\_llamar* transcurrido el tiempo indicado en el segundo parámetro. El tiempo se expresa en milisegundos. Es importante introducir en el primer parámetro solamente el nombre de la función sin paréntesis.
- *setInterval* (*Función\_a\_llamar, milisegundos*). Función parecida a la anterior, pero, en este caso, se ejecutará la función *Función\_a\_llamar* de manera periódica según los milisegundos introducidos en el segundo parámetro.
- *clearInterval()*. Para la ejecución iniciada con *setInterval()*. Véase un ejemplo de uso:

```

var elCrono = setInterval(crono, 1000);
clearInterval(elCrono);

```

- *clearTimeout()*. Para la ejecución iniciada con *setTimeout()*. Véase un ejemplo de uso:

```

var elTemporizador = setTimeout(laFuncion, 5000);
clearTimeout(elTemporizador);

```

En el caso del reloj que se está implementando, es obvio que la segunda función será la más indicada para actualizar el cronómetro cada segundo. El código completo de la página web quedará de la siguiente manera:

```
<!DOCTYPE html>
<html>
<head>
<title>Reloj</title>
</head>
<body>
<div id="laHora"></div>
<script>
function crono(){
var elCrono;
var miFecha = new Date();
var horas = miFecha.getHours();
var minutos = miFecha.getMinutes();
var segundos = miFecha.getSeconds();
if (horas > 12){
 ampm='pm';
 horas-=12;
}else{
 ampm='am';
}
if (horas<10){horas='0'+horas;}
if (minutos<10){minutos='0'+minutos;}
if (segundos<10){segundos='0'+segundos;}

var texto = document.getElementById('laHora');
texto.innerHTML=horas+':' +minutos+':' +segundos+' '+ampm;
}
window.onload=function(){
 elCrono = setInterval(crono,1000);
}
</script>
</body>
</html>
```

Como se puede observar, se utiliza el evento onload del objeto window, lo que implica que, en la carga de la página web, se ejecute la sentencia setInterval programando la ejecución de la función crono cada segundo.

### Actividad propuesta 3.1



Basándote en el código expuesto en este apartado, crea tu propio cronómetro.

### 3.3. Cookies

Generalmente cuando se navega, supuestamente, es de forma anónima, pero, en muchos de los casos, esto no es necesariamente útil para el servidor. Por ejemplo, imagine que está comprando en una página web y va desplazándose de un artículo a otro y quiere mantener el carrito de la compra. Imagine que cierra el navegador por hoy y que continúa mañana con las compras.



#### SABÍAS QUE...

Las cookies aparecieron hace años por la necesidad que tenían las páginas web (servidores) de conocer a sus clientes (navegadores).

Un buen sistema debería almacenar todos los artículos que se han añadido al cesto de la compra para luego poder venderlos.

Por lo tanto, las cookies sirven para *recordar información del usuario*. Son pequeños ficheros que se almacenarán en una ruta determinada en el equipo y que contienen pares clave = valor como, por ejemplo:

```
usuario = Dimas
```

Cuando el navegador pide una página web, las cookies asociadas a dicha página web se envían también en la petición y, de esa manera, los servidores tendrán *vigilado* al usuario.

#### 3.3.1. ¿Para qué se utilizan las cookies?

Estos son algunos ejemplos de uso de las cookies:

- *Monitorizar la actividad de los usuarios.* Esta es una de las opciones más controvertida, pues consiste en monitorizar patrones de actividad, gustos, navegación, etc. Muchas empresas utilizan esta técnica. Utilizar las cookies para estos propósitos hace que los usuarios se sientan espiados.
- *Para mantener opciones de visualización o de aspecto para el usuario.* Cuando un usuario quiere ver unas cosas en alguna página web y otras no, pueden utilizarse cookies para este propósito.
- *Almacenar variables en el lado del cliente.* Es conocido que, después de terminar la sesión, los datos desaparecen para el servidor, salvo que se utilicen cookies.
- *Identificación o autenticación.* Las cookies tienen un periodo de tiempo de validez, durante el cual se puede verificar cuándo un usuario se autentica en el sistema por primera vez. Se puede jugar con la caducidad de las cookies para mantener a un usuario autenticado y, de esa manera, mantener una utilización más eficiente de la página.

## RECUERDA

- ✓ Existen límites para la longitud de las cookies, el número de cookies máximas por servidor web y el número de cookies máximas soportadas por el navegador.

### 3.3.2. ¿Cómo crear una cookie?

Bastaría con incluir la siguiente línea de código en un script:

```
document.cookie="usuario=Dimas";
```

El valor de la cookie siempre tiene que estar correctamente codificado, puesto que se envía en la cabecera HTTP. Por lo tanto, para evitar problemas, se debería hacer lo siguiente:

```
var usuarioCookie="Dimas"
document.cookie="usuario="+ encodeURIComponent(usuarioCookie);
```

`encodeURIComponent()` y `decodeURIComponent()` codifican y decodifican en HTML los caracteres especiales como `, / ? : @ & = + $ #`.

Las cookies tienen una fecha de caducidad y, por tanto, las anteriores cookies se eliminarán al cerrarse el navegador. Si se desea que perduren se ha de añadir una fecha de caducidad:

```
document.cookie="usuario=Dimas;expires=Sat, 6 Aug 2020 12:15:00 GMT";
```

Como se puede imaginar, es posible hacer que caduque la cookie el año que viene o dentro de 5 años, así se quedará de forma “perpetua”.

En el siguiente código se muestra cómo se crearía un botón que cree una cookie asociada:

```
<button type="button" onclick='document.cookie="usuario=Dimas;expires=Sat, 6 Aug 2020 12:15:00 GMT"'>Crear una Cookie</button>
```

### 3.3.3. ¿Cómo leer las cookies?

En el ejemplo del apartado 3.3.2, se ha creado un botón que, al pulsarlo, creará una cookie, pero ¿hay una manera de visualizar las cookies? En el siguiente código, se crea un botón que mostrará las cookies registradas:

```
<button type="button" onclick='alert(document.cookie)''>Ver las Cookies</button>
```

## TOMA NOTA



Para modificar una cookie, basta con volver a crearla con los nuevos datos. Para borrar una cookie, solo hay que volver a crear la cookie con una fecha pasada. También se puede utilizar el parámetro `max-age` de la siguiente forma:

```
document.cookie = "usuario=; max-age=0";
```

**Actividad propuesta 3.2**

Teniendo en cuenta el código expuesto en los apartados 3.3.2 y 3.3.3, elabora una página web en la que se cree y se borre una cookie comprobando su funcionamiento en un navegador.

## 3.4. Almacenamiento local

Como se ha visto en el apartado 3.3, el almacenamiento en el lado del cliente solamente es posible a través de cookies. Tras aparecer HTML5, las cookies pasaron a un segundo plano para dar paso al almacenamiento local. El almacenamiento local es un sistema más sencillo y eficiente que las antiguas cookies.

HTML5 pensó que el navegador puede almacenar mucha información (como mucho 5 MB) y, de esa manera, no sobrecargar el servidor enviando y recibiendo datos. Se supone que, dentro de un dominio y un protocolo, la información es compartida por todas y cada una de las páginas.

### 3.4.1. El objeto `localStorage`

Este objeto permite guardar y recuperar información en el navegador sin importar que sea otra sesión. Su utilización es similar a las cookies y puede utilizarse con los métodos:

- `setItem(clave,valor)`*. Se emplea para guardar información y tiene dos parámetros, que son la clave y el valor.
- `getItem(clave)`*. Para recuperar información, habrá que especificar la clave para poder recuperar el valor de esta.

Véase un ejemplo de utilización de almacenamiento local con el objeto `localStorage`:

```
<!DOCTYPE html>
<html>
<body>
```

```
<script>
 localStorage.setItem("usuario", "Dimas");
 alert(localStorage.getItem("usuario"));
</script>
</body>
</html>
```

Para eliminar un dato del almacenamiento local, se utilizará el siguiente método:

```
localStorage.removeItem(clave);
```



#### SABÍAS QUE...

¿Qué ocurre si solamente se necesita que los datos se guarden durante la sesión y no de forma permanente? HTML5 ha creado el objeto `sessionStorage`, que se utiliza de la misma manera que `localStorage`, pero con la particularidad de que los datos solamente se almacenarán durante la sesión.

### 3.5. Objetos en JavaScript

En JavaScript, prácticamente todo es un objeto, las fechas, las expresiones regulares, los arrays, las funciones, etc. Salvo valores y tipos primitivos como `false`, `true`, `null`, etc., los demás componentes del lenguaje son objetos en JavaScript.

Aunque se desaconseja (porque ralentizan el código y no añaden ninguna ventaja), se pueden declarar tipos primitivos como objetos. A continuación, se muestran las variables `string`, `number` y `boolean` de tipo objeto:

```
var a = new String();
var b = new Number();
var c = new Boolean();
```

Por lo tanto, en el siguiente código:

```
var cadena = "Hola";
var cadena2 = new String("Hola");
```

Tanto `cadena` como `cadena2` contendrán lo mismo y, si se hace una comparación (`cadena == cadena2`), el resultado será `true`, pero, en una comparación estricta (`cadena === cadena2`), el resultado será `false`. Esto es así porque `cadena` es de tipo `string`, mientras que `cadena2` es de tipo `object`. Véase el siguiente código:

```
alert(typeof(cadena)); //mostrará string
alert(typeof(cadena2)); //mostrará object
```

En JavaScript, pueden crearse objetos de dos formas, tal y como se señala en el cuadro 3.1. Aunque los dos códigos tienen la misma funcionalidad, el primero es más rápido, por lo tanto, es aconsejable utilizar la primera forma.

**CUADRO 3.1**  
Objetos en JavaScript

Primera forma	Segunda forma
<pre>var coche = {   modelo:"Mercedes C320",   color:"azul",   kms:15000,   combustible:"diésel" };</pre>	<pre>var coche = new Object(); coche.modelo = "Mercedes C320"; coche.color = "azul"; coche.kms = 15000; coche.combustible = "diésel";</pre>

### 3.5.1. Recorrer la información de un objeto

En ocasiones, es necesario recorrer los campos de un objeto para poder procesarlos. Véase el siguiente código:

```
var usuario={nombre:"Felipe",apellido:"Ranas",edad:30,esAdmin:true};
for(campo in usuario){
 alert(campo);
 alert(usuario[campo]);
}
```

El navegador mostrará mensajes emergentes con el nombre y valor de cada uno de los atributos del objeto (nombre, Felipe, apellido, Ranas, etc.).

#### FUNDAMENTAL

##### *Acceso a los campos de un objeto*

Imagínese que se tiene el siguiente objeto:

```
var usuario={
 nombre:{first:'Pedro',last:'Salas'}
,edad:30,esAdmin:true
};
```

Las dos líneas siguientes mostrarían la misma información (Pedro):

```
alert(usuario.nombre.first);
alert(usuario['nombre']['first']);
```

La primera forma de acceso a los miembros de un objeto es parecida a la utilizada en Java.

### 3.5.2. Constructores de JavaScript

Hasta ahora no se habían visto métodos de objetos en JavaScript; el siguiente código muestra un constructor para la clase coche:

```
function coche(modelo, color, kms, combustible) {
 this.modelo = modelo;
 this.color = color;
 this.kms = kms;
 this.combustible = combustible;
}
var elmio = new coche("Mercedes E330", "negro", 120000, "diésel");
var eltuyo = new coche("BMW 318", "blanco", 210000, "gasolina");
```

Imagínese ahora el siguiente código:

```
var miBMV = eltuyo;
```

En la última sentencia del código anterior, JavaScript no creará una copia del objeto, las variables `miBMV` y `eltuyo` apuntarán al mismo objeto. Cualquier modificación de un atributo en una variable repercutirá en la otra.



#### TOMA NOTA

JavaScript puede añadir nuevos atributos, a pesar de que el objeto haya sido creado previamente. La siguiente línea de código funcionará sin problema alguno:

```
elmio.matrícula = "4321 JPH"
```

Un objeto se caracteriza por tener un estado (atributos) y un comportamiento (métodos). Estos son algunos métodos (un setter y un getter) para la clase coche creada previamente:

```
function coche(modelo, color, kms, combustible) {
 this.modelo = modelo;
 this.color = color;
 this.kms = kms;
 this.combustible = combustible;
 this.setmodelo = function (nuevomodelo) {
 this.modelo = nuevomodelo;
 }
 this.getmodelo = function () {
 return this.modelo;
 }
}
```

### 3.6. Funciones en JavaScript

Como se ha dicho anteriormente, las funciones en JavaScript son objetos. En el capítulo 2, se explicaba la función `suma`, que tenía el siguiente código:

```
function suma(a, b) {
 return a + b;
}
```

A diferencia de otros lenguajes de programación, esta función se puede crear mediante un constructor llamado `Function()` de la siguiente manera:

```
var suma = new Function("a", "b", "return a + b");
```

El resultado de la siguiente línea de código será idéntico tanto si se declara la función de forma clásica como si se crea mediante un constructor:

```
var c = suma(2,2);
```

También se puede declarar la función después de utilizarla. No es lo más común en los lenguajes de programación, pero JavaScript permite crear el siguiente código:

```
var c = suma(2,2);
var suma = new Function("a", "b", "return a + b");
```

Como se puede observar, la declaración de la función se realiza después de llevar a cabo su llamada. Esto es así porque el navegador recibe todo el código JavaScript, lo interpreta y luego lo ejecuta. En lenguajes de programación compilados (no interpretados), esto no sería posible, el compilador mostrará un mensaje de error.

Por otro lado, la función flecha o anónima es muy utilizada por los programadores en la actualidad. Sustituye a la expresión de función tradicional, como se puede observar a continuación:

```
// función tradicional JavaScript
function resta (a,b){return a-b;};
// función flecha 1
let resta = (a,b)=>{return a-b;};
// función flecha 2
let resta = (a,b)=>a-b;
```

Las tres funciones anteriores son equivalentes. Como se puede observar, en las funciones dos y tres se ha sustituido la palabra reservada `function` por su equivalente inline, que es la función arrow `"=>"`.

Esta función flecha se utiliza mucho en frameworks y otro tipo de funciones para arrays, como `map()`, `filter()`, etc.

A continuación, se muestran otras tres líneas de código equivalentes. En este caso, para la función cuadrado, la cual, dado un número, devuelve el cuadrado del mismo.

```
let cuadrado = (a)=> a*a;
let cuadrado = a=> a*a;
let cuadrado = (a)=> {return a*a};
```

En los ejemplos anteriores, las funciones siempre tenían parámetros. Un ejemplo de función flecha sin parámetros sería el siguiente:

```
let hola = ()=>"hola";
```

### 3.6.1. La recursividad en JavaScript

La recursividad es una función o algoritmo recursivo que genera la solución realizando llamadas a sí mismo. La ventaja de la recursividad es la reducción de la complejidad del problema hasta que se llegue a un punto en la cual ya no haga falta utilizar la recursión, puesto que la resolución es algo directo o simple.

En JavaScript, es posible utilizar la recursividad para la resolución de problemas. Como ejemplo se muestra el siguiente código, en el cual se puede observar cómo se resuelve el factorial del número 10 mediante la aplicación de la recursividad:

```
<!DOCTYPE html>
<html>
 <head><title>Ejemplo de recursividad</title></head>
 <body>
 <script>
 function factorial(num){
 if(num == 0){
 return 1;
 }else{
 return (num * factorial(num -1));
 }
 }
 var numero = factorial(10);
 document.write(numero);
 </script>
 </body>
</html>
```

#### Actividad propuesta 3.3



Teniendo en cuenta el código expuesto en este apartado, crea una función recursiva que muestre la sucesión de Fibonacci. Comprueba en un navegador que la función visualiza correctamente la secuencia deseada.

### 3.6.2. Los parámetros de las funciones

En JavaScript, es posible llamar a una función con menos parámetros de los previstos. No dará error en ejecución, pero hay que tener previsto en el código esta eventualidad. Véase un ejemplo de esto en la siguiente función:

```
function suma(a, b) {
 if (b === undefined){ //en el caso que no se introduzca segundo parámetro
 return a + a;
 }
}
```

```

 }
 return a + b;
}
// llamada normal
var c = suma(2,2);
// llamada con un parámetro solo
var c = suma(2);

```



## PARA SABER MÁS

Siempre que hay que utilizar fórmulas u operaciones matemáticas, es necesario utilizar una librería u objeto matemático. En JavaScript, está el objeto Math, el cual tiene numerosos métodos y propiedades que pueden servir para ejecutar muchas de las necesidades que se tengan en una aplicación.

Véanse algunos ejemplos:

```

// propiedades
var x1 = Math.LN10; // devuelve el logaritmo natural de 10 (2,302)
var x2 = Math.E; // devuelve el valor del número e (2,718)
var x3 = Math.PI; // devuelve el valor del número pi (3,1415)
// métodos
var y1 = abs(-5); // devuelve el valor absoluto del parámetro introducido.
var y2 = Math.sqrt(49); // devuelve la raíz cuadrada de 49
var y3 = Math.random(); // devuelve un número aleatorio entre cero y uno.

```

### 3.6.3. Funciones y métodos en objetos de tipo array

Los arrays son uno de los objetos que tienen más métodos (funciones) para su tratamiento. A continuación, se muestran algunos de los métodos más útiles y comunes de este objeto:

- concat()*. Concatena y devuelve una copia de los arrays concatenados.
- fill()*. Rellena un array con un valor suministrado.
- filter()*. Devuelve un array con los elementos que pasen el test suministrado por parámetro. Ejemplo:

```

//objetivo: devolver un array con los números naturales encontrados [13,
1, 8, 2].
var datos= [13, -5, -9, 1, 8, 2];

function natural(dato) {
 return dato>= 0;
}

```

```
function buscaNumerosNaturales() {
 document.getElementById("cualquiercampo").innerHTML = datos.filter(-
 natural);
}
```

- d) *find()*. Devuelve el valor del primer elemento que pase el test suministrado por parámetro. Ejemplo:

```
//Objetivo: devolver el primer número natural encontrado (13).
var datos= [13, -5, -9, 1, 8, 2];

function natural(dato) {
 return dato>= 0;
}

function buscaNumerosNaturales() {
 document.getElementById("cualquiercampo").innerHTML = datos.find(na-
 tural);
}
```

- e) *forEach()*. Realiza una llamada a una función por cada uno de los elementos de un array. Ejemplo:

```
<!DOCTYPE html>
<html>
<body>

<p id="diasDeLaSemana">Aquí aparecerán los días de la semana:
</p>

<script>
textoDias = document.getElementById("diasDeLaSemana");
var dias = ["lunes","martes","miércoles","jueves","viernes","sábado","-
domingo"];

function myFunction(item, index) {
 textoDias.innerHTML = textoDias.innerHTML + "index[" + index + "]: " +
 item + "
";
}
</script>

<button onclick="dias.forEach(myFunction)">Dale!</button>

</body>
</html>
```

- f) *includes()*. Comprueba si en el array está un elemento especificado.  
g) *indexOf()*. Busca en el array un elemento y devuelve su posición.  
h) *isArray()*. Comprueba si un objeto determinado es un array.

- i) *lastIndexOf()*. Devuelve la posición de un elemento de un array. Importante, este método, al contrario de lo que pudiese pensarse, comienza la búsqueda por el final. Ejemplo:

```
var dias = ["Lunes", "Jueves", "Miércoles", "Jueves", "Viernes", "Sábado",
"Domingo"];
var pos = dias.lastIndexOf("Jueves"); //ahora pos valdrá 3 porque busca
desde el final.
```

- j) *pop()*. Elimina un elemento de la última posición de un array. Este método devuelve dicho elemento.
- k) *push()*. Añade un elemento nuevo al array en su última posición. Devuelve la longitud del nuevo array. Con los métodos *push()* y *pop()*, se puede implementar una pila.
- l) *shift()*. Elimina el primer elemento del array. Este método devuelve dicho elemento. Útil cuando se desea realizar algún proceso con los elementos de un array, pero no se desea conservarlos. También es útil para hacer una cola de elementos en combinación con *push()*.
- m) *slice()*. Selecciona parte de un array.

```
var dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado",
"Domingo"];
var finde = dias.slice(5, 7); // finde contendrá ahora "Sábado,Domingo"
```

- n) *sort()*. Ordena los elementos de un array.
- o) *splice()*. Método muy útil para añadir o eliminar elementos de un array.
- p) *toString()*. Convierte un array en un string.
- q) *valueOf()*. Devuelve los valores de un array. Puede ser útil para hacer copias de un objeto array. Ejemplo:

```
var dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado",
"Domingo"];
var dias2 = dias.valueOf(); //copiará los elementos de la variable dias a
dias2
```

## A) La función para arrays *map()*

Muchas veces necesitamos realizar una operación por cada elemento de un array y la solución pasa por realizar un bucle entre todos los elementos del array e ir recuperando y ejecutando cierto código por cada elemento. ¿Sería posible hacer esto de una forma sencilla? La respuesta es sí, mediante la función *map()*.

A continuación, se muestra un código sencillo explicado paso a paso:

```
<!DOCTYPE html>
<html>
<body>

<p id="resultado">Tenemos el array definido de la siguiente manera:
```

let numeros = [1, 2, 3, 4, 5, 6, 7]; cuando pulses el botón aquí aparecerán los mismos números incrementados en uno</p>

```
<button onclick="dale()">ejecuta</button>
```

```
<script>
```

```
let numeros = [1, 2, 3, 4, 5, 6, 7];
```

```
function dale() {
```

```
 let obj = document.getElementById("resultado")
```

```
 obj.innerHTML = numeros.map(function(valor){return valor+1;});
```

```
 /* map irá ejecutando la función incluida como parámetro la cual no tiene
 nombre porque no es necesario. Esta función tomará como parámetro valor (el
 valor de esa posición del array) y devolverá dicho valor aumentado en uno.
 La función map devuelve otro array con el resultado de aplicar la función
 anterior a los elementos de todo el array */
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

#### RECUERDA

- ✓ La función map devuelve otro array con el resultado de aplicar la función pasada como parámetro a los elementos del array. Map es la función SIN (sin bucles y sin tener que crear un array, puesto que lo hace ella sola).

El prototipo de esta función map() es el siguiente:

```
array.map(function(elemento, indice, el_array), valor_this)
```

La función map tiene dos parámetros, como se puede observar. El primero es la función callback que, a su vez, tiene tres parámetros, y el segundo parámetro es valor\_this, el cual es opcional y se podrá utilizar como “this” para referenciar al objeto de la llamada.

En la función callback el único parámetro obligatorio “es elemento” (el primero). “Índice”, que indica el índice del elemento, y “el\_array”, que apunta al array protagonista de la llamada, son opcionales.

Si queremos realizar un código que tenga en cuenta el elemento y su índice, un ejemplo podría ser el siguiente:

```
let numeros = [1, 2, 3, 4, 5, 6, 7];
```

```
function dale() {
```

```
 let obj = document.getElementById("resultado")
```

```
obj.innerHTML = numeros.map(function(valor,indice){return valor+indice;});
/* el resultado de esta llamada será [1,3,5,7,9,11,13] */
}
```

El anterior código es una modificación del programa ejemplo primero y su función es sumar a cada elemento del array el índice que ocupa en dicho array.

## FUNDAMENTAL

No hace falta que utilices la función `map()` obligatoriamente para devolver un array. Puedes utilizarla para realizar una operación por cada elemento del array:

```
let numeros = [1, 2, 3, 4, 5, 6, 7];
numeros.map(function(valor,indice){console.log("valor:"+valor+"
 indice:"+indice)});
```

Como se puede ver en el código anterior, se mostrará por consola el valor de los elementos del array y su índice:

En muchas ocasiones, lo más normal es encontrarnos con la función `map()` en combinación con la función flecha `=>` o función anónima. Veamos dos líneas de código funcionalmente iguales:

```
obj.innerHTML = numeros.map(function(valor,indice){return valor+indice;});
obj.innerHTML = numeros.map((valor,indice)=>{return valor+indice;});
```

Como se puede observar, en el código anterior se ha sustituido la palabra reservada *function* por su equivalente inline, que es la función arrow “`=>`”.

La función flecha se utiliza para escribir funciones callback *ad hoc* y es muy común su utilización en muchos frameworks como ReactJS o Angular. Muchos de estos frameworks utilizan muy a menudo funciones callback. La función arrow es una forma intuitiva, fácil y rápida de programar dichos callback.

## B) La función para arrays *reduce()*

La función *reduce* permite iterar un array acumulando las operaciones que se vayan haciendo sobre los elementos en otro llamado acumulador. Se ponen dos ejemplos de uso:

```
let numeros = [1, 2, 3, 4];
console.log (numeros.reduce((acc,v)=>{return acc+v;}));
let numeros2 = ["uno","dos","tres","cuatro"];
console.log (numeros2.reduce((a,b)=>{return a.concat(b);}));
```

En el primero de ellos se sumarán todos los números del array (1 + 2 + 3 + 4), lo cual mostrará por consola en la segunda línea el valor 10.

El segundo ejemplo mostrará por consola la concatenación de todos y cada uno de los elementos del array. El resultado será: “unodostrescuatro”.

En el siguiente esquema de la figura 3.1 se puede ver tanto el prototipo de la función `reduce()` como su aplicación al primer ejemplo del código anterior.

### Ejemplo

```
let numeros = [1, 2, 3, 4];
console.log (numeros, reduce((acc, v)=>{return acc+v;}));
```

Prototipo:

array.reduce(f\_callback(acumulador, valorActual, índice, Array){código})

=>

no usados

**Figura 3.1**

Prototipo de la función `reduce()`.

Como se puede observar, `reduce` también utiliza funciones callback y, por lo tanto, la función arrow (`=>`), vista anteriormente, es una candidata perfecta para utilizar.

#### RECUERDA

- ✓ Utiliza la función `map()` cuando tengas que realizar una operación por cada elemento de un array.  
Utiliza la función `reduce()` cuando necesites realizar una operación por cada elemento de un array y haya que ir acumulando el resultado generado.  
Utiliza la función arrow (`=>`) para funciones callback.

### 3.6.4. Funciones y métodos en objetos y variables de tipo string

JavaScript, al igual que otros lenguajes de programación, tiene muchos métodos para manejar strings. Véase un ejemplo para calcular la longitud de un string:

```
var web = "myfpschool.com";
var longitud = web.length; //longitud valdrá 14
```

Imagínese que se quiere conocer la posición (el lugar donde comienza) de la palabra *myfpschool* dentro del string *web*:

```
var web = "La web myfpschool es una de las mejores en tecnología";
var posicion = web.lastIndexOf("myfpschool"); //posición valdrá 7
```

Imagínese que la palabra o substring (no tiene por qué ser una palabra) se repite dentro del texto varias veces. Con el método `LastIndexOf()`, puede conocerse la posición de la última ocurrencia. Otras funciones muy útiles de los string son las siguientes:

- *search(string1)*. Busca en un string un determinado valor (o expresión regular) y devuelve la posición donde la encontró.
- *slice(inicio, fin)*. Extrae parte de un string y devuelve el nuevo string.
- *substring(inicio, fin)*. Extrae parte de un string comenzando por la posición inicio y terminando en la posición fin.
- *substr(inicio, longitud)*. Extrae caracteres de un string comenzando en la posición especificada en el primer parámetro con una longitud expresada en el segundo parámetro.
- *replace(string1, string2)*. Reemplaza en un string una cadena de caracteres (o una expresión regular) por la cadena expresada en el segundo parámetro. Esta función devuelve un string con los reemplazos realizados.
- *toUpperCase()*. Convierte un string a mayúsculas.
- *toLowerCase()*. Convierte un string a minúsculas.
- *concat(string1, string2)*. Concatena dos o más cadenas de caracteres devolviendo el string resultante.
- *charAt(indice)*. Devuelve el carácter que se especifica en la posición expresada en el primer parámetro.
- *split(separador, límite)*. Crea un array de un string utilizando como separador el primer parámetro. El segundo parámetro es opcional y especifica el número máximo de elementos que puede tener dicho array. Pasado dicho número, JavaScript no creará nuevos elementos en el array.
- *repeat(numVeces)*. Repite el string del cual se invoca el método el número de veces especificado en el primer parámetro.

Véase el siguiente código con los comentarios correspondientes:

```
var stringSearch = "Hola Mundo";
console.log(stringSearch.search("Mun"));
// mostrará por consola 5

console.log(stringSearch.slice(2, stringSearch.length));
// mostrará por consola la Mundo

console.log(stringSearch.substring(1, 4));
// mostrará por consola ola
console.log(stringSearch.substr(1, 4));
// mostrará por consola ola

console.log(stringSearch.replace("a", "X"));
// mostrará por consola HolX Mundo

console.log(stringSearch.toUpperCase());
// mostrará por consola HOLA MUNDO

console.log(stringSearch.toLowerCase());
// mostrará por consola hola mundo
```

```

var cad = "mmm";
console.log(stringSearch.concat(cad));
// mostrará por consola Hola Mundommm

console.log(stringSearch.charAt(5));
// mostrará por consola M

console.log(stringSearch.split(" "));
// Creará el Array ["Hola", "Mundo"]. Se ha utilizado como separador
el espacio

console.log(stringSearch.repeat(2));
// mostrará por consola Hola MundoHola Mundo

```

### 3.6.5. Funciones globales del lenguaje JavaScript relativas a números

Las funciones siguientes son sumamente útiles cuando se validan formularios o campos especiales. Es conocido que HTML5 ya provee al programador de ciertas validaciones nativas, pero, en ocasiones, se necesita una validación más sofisticada. Es en estas ocasiones cuando se utilizarán estas funciones.

1. *isFinite()*. Evalúa si un dato determinado es un valor finito. En caso positivo, devuelve true y, de lo contrario, devuelve false. Si se le suministra algo que no es un número o un número no finito, devuelve false.

```

var dato1 = isFinite(5/0); //devuelve false
var dato2 = isFinite(-3.14); //devuelve true
var dato3 = isFinite(0); //devuelve true
var dato4 = isFinite("5/9"); //devuelve false
var dato5 = isFinite("2018/12/05"); //devuelve false
var dato6 = isFinite("myFPschool.com"); //devuelve false

```

2. *isNaN()*. Evalúa si un dato determinado no es numérico.

```

isNaN(456) //devuelve false porque es un número
isNaN(0) //devuelve false
isNaN(-3.14) //devuelve false
isNaN(9+4) //devuelve false porque al evaluar la expresión el resultado
es numérico
isNaN('666') //devuelve false porque al evaluar el contenido el resultado
es numérico
isNaN('') //devuelve false
isNaN(true) //devuelve false
isNaN(8/0) //devuelve false
isNaN(0/0) //devuelve true
isNaN('Sintesis') //devuelve true al ser un string
isNaN('2005/12/12') //devuelve true al ser una fecha
isNaN(undefined) //devuelve true

```

```
isNaN('NaN') //devuelve true
isNaN(NaN) //devuelve true
```

3. *Number()*. Convierte un dato en un número.

```
Number(true) // devuelve 1
Number(false) // devuelve 0
Number("666") // devuelve 666
```

4. *parseFloat()*. Analiza un string y devuelve un número en coma flotante.

```
parseFloat("10") //devuelve 10
parseFloat("3.14") // devuelve 3.14
parseFloat(" 100 ") // devuelve 100
parseFloat("80 monos") // devuelve 80
```

5. *parseInt()*. Analiza un string y devuelve un número entero.

```
parseInt("10") //devuelve 10
parseInt("3.14") // devuelve 3
parseInt(" 100 ") // devuelve 100
parseInt("80 monos") // devuelve 80
```

6. *String()*. Convierte un dato en un string. Generalmente, se utiliza cuando se quiere convertir un dato numérico en un string.

```
String("666") //devuelve 666
String(3.14) //devuelve 3.14
```

### 3.6.6. Propiedades globales de JavaScript

JavaScript tiene propiedades globales a todos los objetos creados, como son las siguientes:

- *infinity*. Es un valor numérico que representa el infinito. Se visualiza en ocasiones cuando se supera el límite superior o inferior (*infinity* y *-infinity*) de números en coma flotante. A continuación, se muestra un ejemplo de *infinity*.

La siguiente página web:

```
<!DOCTYPE html>
<html>
<body>

<p id="masinfinito"></p>
<p id="menosinfinito"></p>

<script>
//1.797693134862315E+308 es el límite de un número en coma flotante.

document.getElementById("masinfinito").innerHTML = 1.8E+308;
```

```
document.getElementById("menosinfinito").innerHTML = -1.8E+308;

</script>

</body>
</html>
```

Mostrará en el navegador lo siguiente:

```
Infinity
-Infinity
```

Porque ambos valores asignados a las etiquetas `<p>` superan positiva y negativamente el límite de un número en coma flotante.

- *NaN*. La propiedad NaN representa un valor no numérico. En JavaScript, generalmente, se utiliza la función `isNaN()` vista anteriormente.
- *undefined*. La propiedad undefined indica que a una variable no le ha sido asignado ningún valor (o que no ha sido declarada). Ejemplo:

```
var x;
if (typeof x === "undefined") {
 y = "no definida";
}
```

El código anterior asignará a la variable `y` el valor “no definida”.

### 3.6.7. Algunas funciones globales de JavaScript

JavaScript tiene funciones globales como las que se exponen a continuación:

- *encodeURIComponent()*. Codifica a URI. Codifica caracteres que no están permitidos en una URL como `>`, `“`, `[`, etcétera.
- *encodeURIComponent()*. Codifica un componente URI. Codifica, además de los caracteres anteriores (`encodeURIComponent()`), otros como `?`, `=`, `:`, etcétera.
- *decodeURI()*. Decodifica una URI.
- *decodeURIComponent()*. Decodifica un componente URI.

Ejemplo de uso de las funciones anteriores:

```
var uri = "http://myfpschool.com";
var uriencoded = encodeURIComponent(uri);
var uridecoded = decodeURIComponent(uriencoded);

//uriencoded contiene: http%3A%2F%2Fmyfpschool.com
//uridecoded contiene: http://myfpschool.com
```

- *eval()*. Evalúa un string y lo ejecuta como si fuera código (un script).

Ejemplo de uso de la función `eval()`:

```
var a = 2;
var b = 3;
var c = eval("a+b");
// c valdrá 5
```



### Actividad propuesta 3.4

Teniendo en cuenta el código expuesto en este apartado, crea una página web en la que se compare el funcionamiento de las funciones `encodeURIComponent()` y `encodeURIComponent()`.

## 3.7. Prácticas guiadas

Con objeto de afianzar lo estudiado, proponemos dos interesantes prácticas, cuyo desarrollo queda claramente detallado.

### 3.7.1. Uso de estructuras JavaScript: creación de un generador automático de historias

A continuación, se muestra un divertido ejercicio que es un generador automático de historias. El objetivo es practicar todas las estructuras de JavaScript vistas hasta ahora y crear una página web como la de la figura 3.1.

Habrà que crear varios párrafos diferentes y estos se combinarán aleatoriamente (para ello, habrá que utilizar arrays). Las historias tendrán que enlazar correctamente.

También tendrá que aparecer el nombre del protagonista al menos una vez y, dependiendo de si es una historia de leperos o murcianos, el argumento tendrá que cambiar.

Otra cuestión importante es que, para dejar el ejercicio más bonito, habría que hacer que el aspecto de la solución y del ejemplo sean lo más parecidos.

Nombre del protagonista:

Gonzalo

LEPERO • MURCIANO

Generar una historia aleatoria

Estaban a 44 grados centígrados afuera, así que Mariano el Chapas se fue a dar un paseo. Cuando llegaron a la cocina, se quedaron mirando horrorizados durante unos instantes, luego se comieron un potaje y se echaron una siesta. Gonzalo lo vio todo, pero no se sorprendió: su padre es capaz de comerse una vaca en monopatin.

**Figura 3.2**  
Generador de historias.

Archivo index.html:

```
<!DOCTYPE html>
<html lang="es" dir="ltr">
 <head>
 <meta charset="utf-8">
 <title>Historias de Leperos y Murcianos</title>
 </head>
 <body>

 <label for="nombre">Nombre del Protagonista:</
 label>

 <input type="text" id="nombre" required />

 <label for="lepero">LEPERO:</label>
 <input type="radio" name="localidad" value="lepero" id="lepero"/>
 <label for="murciano">MURCIANO:</label>
 <input type="radio" name="localidad" value="murciano" id="mur-
 ciano"/>

 <input type="button" id="btnSubmit" value="Generar Historia" />

 <p id="texto" style="width: 40%; background-color: #FFC126; co-
 lor:black;"></p>

 <script src="./script/script.js"></script>
 </body>
</html>
```

Archivo script.js:

```
var texto = document.getElementById('texto');
var button = document.getElementById('btnSubmit');
if(document.getElementById('lepero').checked){
 button.addEventListener("click", creaHistoriaLepero, false);
}else{
 button.addEventListener("click", creaHistoriaMurciano, false);
}

function creaHistoriaMurciano() {

 texto.innerHTML = "";
 var nombre = document.getElementById('nombre').value.toString();

 var historiasMurciano = [
```

```

 "En Murcia las naranjas siempre han sido muy baratas, pero ahora le
 parecen caras.",
 "La ciudad se llena siempre en verano, pero en invierno es un de-
 sierto. ",
 "" + nombre + " visitaba a su abuela, ella siempre le hacía buena
 comida. ",
 "La Juventud lo dejó deprimido y ahora " + nombre + " solo tiene a
 su peluche. ",
 "No tenía ganas de hacer deporte así que " + nombre + " engordo
 hasta el límite. ",
 "Los donuts eran su debilidad, pero últimamente estaba comportán-
 dose de forma extraña. ",
 "" + nombre + " antes era calvo, pero se hizo un injerto capilar. "

];

 while (historiasMurciano.length > 0) {

 var nAleatorio = Math.round(Math.random() * historiasMurciano.len-
 gth -1);

 if(historiasMurciano[nAleatorio] == undefined){
 nAleatorio = Math.round(Math.random() * historiasMurciano.length
 -1);
 }else{
 texto.innerHTML += historiasMurciano[nAleatorio];
 historiasMurciano.splice(nAleatorio, 1);
 }
 }
}

function creaHistoriaLepero() {

 texto.innerHTML = "";
 var nombre = document.getElementById('nombre').value.toString();

 var historiasLepero = [

 "En Lepe las flores se marchitan en Primavera, así que no había
 excusa. ",
 "" + nombre + " era una persona modesta que conducía un Ford Ka. ",
 "Cada semana " + nombre + " visitaba a sus padres. ",
 "Cualquiera lo hubiera adivinado. ",
 "Llovía fuerte aquella noche y " + nombre + " no podía dejar de
 pensar en los campos de fresas. ",
 "A " + nombre + " no le gustaba la informática. "

];

};

```

```

while (historiasLepero.length > 0) {

 var nAleatorio = Math.round(Math.random() * historiasLepero.length
 -1);

 if(historiasLepero[nAleatorio] == undefined){
 nAleatorio = Math.round(Math.random() * historiasLepero.length
 -1);
 }else{
 texto.innerHTML += historiasLepero[nAleatorio];
 historiasLepero.splice(nAleatorio, 1);
 }
}
}

```

A continuación se comentarán las partes del código más relevantes:

```

if(document.getElementById('lepero').checked){
 button.addEventListener("click", creaHistoriaLepero, false);
}else{
 button.addEventListener("click", creaHistoriaMurciano, false);
}

```

En las sentencias anteriores, se asocia el evento click del botón que tiene la página a una función diferente dependiendo si se ha seleccionado el radiobutton “lepero” o “murciano”.

```

while (historiasMurciano.length > 0) {

 var nAleatorio = Math.round(Math.random() * historiasMurciano.length
 -1);
 if(historiasMurciano[nAleatorio] == undefined){
 nAleatorio = Math.round(Math.random() * historiasMurciano.length
 -1);
 }else{
 texto.innerHTML += historiasMurciano[nAleatorio];
 historiasMurciano.splice(nAleatorio, 1);
 }
}

```

En el código anterior, se va a hacer un bucle en el array definido anteriormente (en este caso, `historiasMurciano`), mientras que su longitud sea mayor que 0 (que tenga algún elemento). Con la sentencia:

```
var nAleatorio = Math.round(Math.random() * historiasMurciano.length -1);
```

Se genera un número aleatorio entre 0 y el número de elementos del array menos 1. Ese número indicará una posición del array, la cual se va a añadir al texto de la página web. Posteriormente, se elimina esa posición del array con la sentencia:

```
historiasMurciano.splice(nAleatorio, 1);
```

En la sentencia anterior, se elimina un elemento del array (segundo parámetro) partiendo de la posición `nAleatorio`.

En este ejercicio, HTML y JavaScript estarán en ficheros separados. El código JavaScript estará en una carpeta llamada *script*. Para una carga rápida, se colocará la llamada al script al final del fichero HTML.



### Actividad propuesta 3.5

Basándote en el ejemplo expuesto en este apartado, crea tu propio generador automático de historias.

### 3.7.2. Utilización avanzada de arrays: generación de un sudoku aleatorio

Para generar un sudoku aleatorio, hay dos posibilidades:

1. *Generar un sudoku desde cero.* El problema que tiene esta técnica es que, en ocasiones, al crear el sudoku, se encuentran situaciones en las que no hay una solución y el algoritmo tiene que volver hacia atrás (backtracking) y volver a intentar generar una solución válida.
2. *Generar un sudoku a partir de otro ya realizado.* Es la más sencilla. Bastaría con intercambiar filas y columnas con cierto criterio, de tal manera que los números se vayan descolocando y el sudoku vaya siendo más y más aleatorio.

	1	2	3
1	2 1 5	9 3 8	4 6 7
2	7 8 6	1 2 4	3 5 9
3	9 3 4	6 5 7	2 8 1
4	8 6 9	5 4 2	1 7 3
5	1 4 3	7 8 6	5 9 2
6	5 2 7	3 9 1	8 4 6
7	6 7 2	4 1 5	9 3 8
8	4 9 8	2 6 3	7 1 5
9	3 5 1	8 7 9	6 2 4

**Figura 3.3**  
Filas y columnas de un sudoku agrupadas (tres grupos por fila y columna).

Para intercambiar las columnas, hay que tener en cuenta lo siguiente: hay que intercambiar dos columnas que pertenezcan al mismo grupo (1, 2 o 3) y no se pueden intercambiar columnas de distinto grupo.

Para intercambiar las filas, hay que tener en cuenta lo siguiente: hay que intercambiar dos filas que pertenezcan al mismo grupo (1, 2 o 3) y no se pueden intercambiar filas de distinto grupo.

El resultado del ejercicio tendrá que ser algo parecido al de la figura 3.3. Cada vez que se refresque la página el sudoku aleatorio será diferente.

Sudoku original:	Sudoku aleatorio:
2 1 5 9 3 8 4 6 7	7 8 6 1 2 4 3 5 9
7 8 6 1 2 4 3 5 9	9 3 4 6 5 7 2 8 1
9 3 4 6 5 7 2 8 1	2 1 5 9 3 8 4 6 7
8 6 9 5 4 2 1 7 3	8 6 9 5 4 2 1 7 3
1 4 3 7 8 6 5 9 2	1 4 3 7 8 6 5 9 2
5 2 7 3 9 1 8 4 6	5 2 7 3 9 1 8 4 6
6 7 2 4 1 5 9 3 8	6 7 2 4 1 5 9 3 8
4 9 8 2 6 3 7 1 5	4 9 8 2 6 3 7 1 5
3 5 1 8 7 9 6 2 4	3 5 1 8 7 9 6 2 4

**Figura 3.4**

Resultado del ejercicio resuelto de un sudoku original y de un sudoku aleatorio generado.

Véase el fichero HTML del programa (fichero sudoku.html):

```
<!doctype html>
<html>
 <head>
 <meta charset="uft-8"/>
 <title>Sudoku aleatorio</title>
 </head>
 <body>
 Sudoku original:
 <p id="sudokuOrig"></p>

 Sudoku aleatorio:
 <p id="sudokuAleat"></p>

 <script src="./script/sudoku.js"></script>
 </body>
</html>
```

A continuación, se muestra el archivo donde se almacena todo el código JavaScript (fichero sudoku.js). El fichero se ha comentado para una mejor comprensión:

```
// En original se almacena el sudoku original. Es un sudoku ya resuelto.
// Consiste en un array de 81 posiciones, las primeras 9 corresponderán a
// la primera fila,
// las segundas 9 a la segunda y así sucesivamente.
var original=[
 2,1,5,9,3,8,4,6,7,
 7,8,6,1,2,4,3,5,9,
 9,3,4,6,5,7,2,8,1,
```

```

8,6,9,5,4,2,1,7,3,
1,4,3,7,8,6,5,9,2,
5,2,7,3,9,1,8,4,6,
6,7,2,4,1,5,9,3,8,
4,9,8,2,6,3,7,1,5,
3,5,1,8,7,9,6,2,4];

// En modificado en principio se va a colocar el sudoku original pero este
array es el que va a ir modificándose para generar
// el sudoku aleatorio.
var modificado=[
 2,1,5,9,3,8,4,6,7,
 7,8,6,1,2,4,3,5,9,
 9,3,4,6,5,7,2,8,1,
 8,6,9,5,4,2,1,7,3,
 1,4,3,7,8,6,5,9,2,
 5,2,7,3,9,1,8,4,6,
 6,7,2,4,1,5,9,3,8,
 4,9,8,2,6,3,7,1,5,
 3,5,1,8,7,9,6,2,4];

// La función generaSudoku realiza todo el trabajo de generación
del sudoku
function generaSudoku()
{
 var salida,i,j;
 var pos=0;
 //En este primer bucle se muestra en la página web el sudoku
 original
 //Se va recorriendo el array de 9 en 9 posiciones para ir pre-
 sentando el sudoku en la web
 //dentro del párrafo con id="sudokuOrig"
 for (i=0;i<9;i++){
 salida="";
 for (j=0;j<9;j++){
 pos=9*i+j;
 salida += original[pos]+" ";
 }
 document.getElementById("sudokuOrig").innerHTML = document.
 getElementById("sudokuOrig").innerHTML + salida + "
";
 }

 // En las siguientes 4 líneas es donde se van cambiando las
 filas y columnas para hacer el sudoku
 // lo más aleatorio posible.
 cambiaColumnas();
 cambiaFilas();
 cambiaColumnas();
 cambiaFilas();

```

```

//Cuantas más llamadas se hagan a las funciones cambiaColumnas() y cambiaFilas(), más aleatorio será el sudoku.
//Se pueden invocar estas funciones cuantas veces se quiera.

//En este primer bucle se muestra en la página web el sudoku generado
//Se va recorriendo el array de 9 en 9 posiciones para ir presentando el sudoku en la web
//dentro del párrafo con id="sudokuAleat"
pos=0;
for (i=0;i<9;i++){
 salida="";
 for (j=0;j<9;j++){
 pos=9*i+j;
 salida += modificado[pos]+" ";
 }
 document.getElementById("sudokuAleat").innerHTML = document.getElementById("sudokuAleat").innerHTML + salida + "
";
}
}

// La función aleatorio genera un número aleatorio entre min y max
function aleatorio(min,max){
 var horquilla=max-min;
 var aleatorio = Math.round(Math.random()*horquilla)+min;
 return aleatorio;
}

//La función cambiaColumnas() intercambia columnas de cada grupo de manera aleatoria.
function cambiaColumnas(){
 var pos=0,i,aux;
 // En este primer bucle se cambian las columnas del grupo 1
 var columnaA=aleatorio(0,2)+pos;
 var columnaB=aleatorio(0,2)+pos;
 // columnaA y columnaB contienen el valor de las columnas a intercambiar.
 // Ambas variables contendrán un valor aleatorio entre 0 y 2.
 for (i=0;i<9;i++){
 aux = modificado[columnaA];
 modificado[columnaA]=modificado[columnaB];
 modificado[columnaB]=aux;
 //En las tres líneas anteriores se intercambian los valores
 columnaA+=9;
 columnaB+=9;
 //hay que saltar 9 porque el siguiente elemento de la columna está 9 posiciones más adelante en el array
 }
}

```

```

pos=3;
// En este primer bucle se cambian las columnas del grupo 1
columnaA=aleatorio(0,2)+pos;
columnaB=aleatorio(0,2)+pos;
// columnaA y columnaB contienen el valor de las columnas a
intercambiar.
// Ambas variables contendrán un valor aleatorio entre 3 y 5.
for (i=0;i<9;i++){
 aux = modificado[columnaA];
 modificado[columnaA]=modificado[columnaB];
 modificado[columnaB]=aux;
 columnaA+=9;
 columnaB+=9;
}

pos=6;
// En este primer bucle se cambian las columnas del grupo 1
columnaA=aleatorio(0,2)+pos;
columnaB=aleatorio(0,2)+pos;
// columnaA y columnaB contienen el valor de las columnas a
intercambiar.
// Ambas variables contendrán un valor aleatorio entre 6 y 8.
for (i=0;i<9;i++){
 aux = modificado[columnaA];
 modificado[columnaA]=modificado[columnaB];
 modificado[columnaB]=aux;
 columnaA+=9;
 columnaB+=9;
}

}

//La función cambiaFilas() intercambia filas de cada grupo de manera
aleatoria.
function cambiaFilas(){
 var pos=0,i,aux;
 var filaA=9*aleatorio(0,2)+pos;
 var filaB=9*aleatorio(0,2)+pos;
 //En filaA y filaB se almacenan los primeros elementos de cada fila
 a intercambiar.
 //En este caso las filas a intercambiar serán de la 0 a la 2.
 for (i=0;i<9;i++){
 aux = modificado[filaA];
 modificado[filaA]=modificado[filaB];
 modificado[filaB]=aux;
 filaA+=1;
 filaB+=1;
 }
}

```

```

 // como los elementos de una fila están consecutivos solamente
 hay que incrementar en una unidad la posición.
 }

 pos=27;
 // Las siguientes tres líneas comienzan a partir de la posición
 27.
 filaA=9*aleatorio(0,2)+pos;
 filaB=9*aleatorio(0,2)+pos;

 for (i=0;i<9;i++){
 aux = modificado[filaA];
 modificado[filaA]=modificado[filaB];
 modificado[filaB]=aux;
 filaA+=1;
 filaB+=1;
 }

 pos=54;
 // Las siguientes tres líneas comienzan a partir de la posición
 54.
 filaA=9*aleatorio(0,2)+pos;
 filaB=9*aleatorio(0,2)+pos;

 for (i=0;i<9;i++){
 aux = modificado[filaA];
 modificado[filaA]=modificado[filaB];
 modificado[filaB]=aux;
 filaA+=1;
 filaB+=1;
 }
}

//Por último se llama a la función generaSudoku() para que se
ejecute el script y se generen los sudokus.
generaSudoku();

```

### Actividad propuesta 3.6



Copia y ejecuta el código expuesto en este apartado y comprueba que funciona correctamente. Como habrás deducido del código, cuando se van a elegir dos filas o dos columnas de forma aleatoria, en ocasiones, las columnas o filas que han de intercambiarse son la misma. Mejora el código para que, cuando se elija una fila o columna de cada grupo, nunca se repitan.

## Resumen

- El objeto `Date()` tiene los métodos `getHours()`, `getMinutes()` y `getSeconds()` para poder mostrar la hora del modo en que el programador desee.
- `setTimeout()` y `setInterval()` permiten al programador llamar una función en un momento determinado o poder ejecutarla repetidamente cada *x* milisegundos.
- Las cookies son una manera clásica de almacenar información local del usuario.
- Para modificar una cookie, basta con recrearla de nuevo y, para borrarla, con establecer una fecha de expiración pasada sería suficiente.
- El objeto `localStorage` con sus métodos `setItem()`, `removeItem()` y `getItem()` es una versión más nueva de almacenamiento local que las cookies.
- Si solamente se desean mantener los valores durante una sesión, se puede utilizar el objeto `sessionStorage`, que funciona de la misma manera que el objeto `localStorage`.
- Para recorrer los campos de un objeto, se puede utilizar el bucle `for(campo in nombreObjeto){ ... }`.
- En JavaScript, se pueden crear métodos en los objetos como los setters y getters, por ejemplo. Los métodos en JavaScript no son tan utilizados como sus homólogos en Java.
- En JavaScript, es posible utilizar recursividad. No obstante, dado que se ejecuta en un navegador, la velocidad de procesamiento puede no ser alta.
- Se pueden realizar llamadas a las funciones con menos parámetros de los definidos. En ese caso, los parámetros no definidos tendrán valor `undefined`.
- El objeto `Math` tiene funciones y variables muy útiles cuando se necesita utilizar operaciones matemáticas complejas.
- Existen numerosas funciones para objetos de tipo `array` como `concat()`, `fill()`, `filter()`, `forEach()`, etcétera.
- Existen numerosas funciones para objetos de tipo `string` como `search()`, `slice()`, `replace()`, `split()`, etcétera.
- Los números en JavaScript tienen funciones que permiten su validación y conversión como `isFinite()`, `isNaN()`, `Number()`, `parseFloat()` y `parseInt()`.

## Ejercicios propuestos



1. Lleva a cabo un ejercicio que muestre un enlace a [Myfpschool.com](http://Myfpschool.com) si sacando un número de forma aleatoria es menor que 0,5. En caso contrario, que salude con buenos días (antes de las 15:00) o buenas tardes dependiendo de la hora en la que se cargue la página. Utiliza el método `Math.random()` y el atributo `document.getElementById("id").innerHTML` para realizar el ejercicio.
2. Borrando atributos de un objeto:

```
function coche(modelo, color, kms, combustible) {
 this.modelo = modelo;
```

```

 this.color = color;
 this.kms = kms;
 this.combustible = combustible;
 }
 var elmio = new coche("Mercedes E330", "negro", 120000, "diésel");
 var eltuyo = new coche("BMW 318", "blanco", 210000, "gasolina");

```

En el ejemplo anterior, ¿se puede borrar un atributo con la siguiente sentencia?:

```
delete elmio.matrícula;
```

Crea un programa que pruebe si esto es posible.

3. Realiza un script que, cuando pulse un botón, te diga la última posición de una determinada palabra dentro de una frase. El script solamente se ejecutará cuando se pulse el botón.

Imagínate que la palabra o substring (no tiene por qué ser una palabra) se repite dentro del texto varias veces. Con el método `LastIndexOf()`, puede conocerse la posición de la última ocurrencia.

4. Antonio está mostrando los mensajes relativos a la Navidad depurando código y utilizando funciones con string:

```

<html>
<body>

<script>
var list = document.querySelector('ul');
list.innerHTML = "";
var saludos = ['¡Feliz cumpleaños!', 'Feliz navidades a todos', 'Te
deseo una feliz navidad', 'En Navidades nos vamos de fiesta',
'Pasa un buen fin de semana'];

for(var i = 0; i < saludos.length; i++) {
 var input = saludos[i];
 if(saludos[i].indexOf('Navidad') !== -1) {
 var result = input;
 var listItem = document.createElement('li');
 listItem.textContent = result;
 list.appendChild(listItem);
 }
}
</script>
</body>
</html>

```

Ten en cuenta la web anterior, pero sin que realice lo que Antonio el programador desea. Échale una mano y modifica el código para que muestre correctamente todos los mensajes relativos a las Navidades.

5. Repite el ejercicio 4, pero consiguiendo que no salga ningún mensaje que contenga la palabra *fiesta*.
6. Mariano tiene un problema con la siguiente página web, cuando quiere mostrar los nombres de los equipos correctamente, le sale una serie de números que no desea. Por ejemplo:

MAN : Manchester United

Como no ha terminado todavía el ciclo de DAW, te ha pedido que le echés una mano para arreglar la página web.

```
<html>
<body>

<script>
var list = document.querySelector('ul');
list.innerHTML = "";
var equipos = ['MAN675847583748sjt567654;Manchester United',
'RMD576746573fhdg4737dh4;Real Madrid',
'LIV5hg65hd737456236dch46dg4;Liverpool FC',
'SEV4f65hf75f736463;Sevilla FC',
'BAR5767ghtyfyr4536dh45dg45dg3;Barcelona FC'];

for (var i = 0; i < equipos.length; i++) {
var input = equipos[i];
// escribe el código que haga que funcione aquí debajo
var result = input;
var listItem = document.createElement('li');
listItem.textContent = result;
list.appendChild(listItem);
}
</script>
</body>
</html>
```

7. En el código siguiente, se presenta un objeto del tipo persona que tiene una serie de atributos (name, age, gender e interests) y dos métodos: saluda y bio.
  - a) Prueba el código y comprueba que funciona.
  - b) Crea dos botones biografía y saluda y haz que cada uno llame al método correspondiente y muestre el resultado en la página web.

```
<html>
<body>
<script>
```

```

var persona = {
 name: ['Rafa', 'Nadal'],
 age: 30,
 gender: 'masculino',
 interests: ['tenis', 'futbol'],
 bio: function() {
 alert(this.name[0] + ' ' + this.name[1] + ' tiene ' + this.age
 + ' años y le gusta el ' + this.interests[0] + ' y el ' + this.
 interests[1] + '.');
 },
 saluda: function() {
 alert('Hola, me llamo ' + this.name[0] + '.');
 }
};
</script>
</body>
</html>

```

8. Benito no sabe si el código siguiente funciona ni para qué sirve:

```

var perrillos = ["Rocket","Flash","Bella","Slugger"];
Console.log(perrillos.toString());
var ciudades = 'Manchester,London,Liverpool,Birmingham,Leeds,Carlisle';
perrillos = ciudades.split(',');
Console.log(perrillos.toString());

```

- Explícale si funciona correctamente (corrige lo que esté mal) y realiza un procedimiento que elimine del array perrillos todos los elementos que contengan una C sin importar si está en mayúscula o minúscula.
- Modifica el programa para que perrillos contenga los nombres de los perros y las ciudades (ambos).
- Comprueba si funciona lo siguiente:

```
perrillos.unshift('Estepona');
```

9. Teniendo un array muy grande con elementos de la siguiente estructura:

```
"28924;Estepona"
```

Crea un texto y un botón que ponga *buscar* que muestre el CP y el nombre de la ciudad que coincida o contenga el texto del campo creado. No importan las mayúsculas y minúsculas en la búsqueda.

- Elabora una página web que cree dos cookies con el nombre y el contenido que consideres oportuno.

## ACTIVIDADES DE AUTOEVALUACIÓN

1. ¿Qué hace setTimeout?:
  - ☐ a) Ejecuta una función transcurrido un tiempo determinado.
  - ☐ b) Hace que se ejecute el script transcurrido un tiempo determinado.
  - ☐ c) Ejecuta una función de forma repetida cada x milisegundos.
2. ¿Qué se usa para borrar una cookie?:
  - ☐ a) Se utilizará la función removeCookie().
  - ☐ b) Se utilizará la función deleteCookie().
  - ☐ c) Se puede utilizar el parámetro max-age=0.
3. ¿Hasta qué capacidad puede almacenar información el navegador?:
  - ☐ a) 50 MB.
  - ☐ b) 5 MB.
  - ☐ c) Ilimitada.
4. El objeto localStorage para almacenar información utiliza el método:
  - ☐ a) setItem(clave,valor).
  - ☐ b) storeData(clave,valor).
  - ☐ c) saveData(clave,valor).
5. Una de las siguientes líneas de código no funciona, ¿cuál es?:
  - ☐ a) `var suma = new Function("a", "b", "return a + b");`
  - ☐ b) `var suma = new Function("a", "b", "{return a + b}");`
  - ☐ c) `var suma = new Function("a", "b", {"return a + b"});`
6. ¿Qué función se utiliza para rellenar un array con un valor determinado?:
  - ☐ a) fill().
  - ☐ b) push().
  - ☐ c) slice().
7. ¿Qué función se utiliza para eliminar elementos concretos de un array?:
  - ☐ a) deleteItem().
  - ☐ b) splice().
  - ☐ c) removeItem().
8. ¿Qué función se usa para crear un array de un string que tenga campos delimitados por algún carácter?:
  - ☐ a) split().
  - ☐ b) slice().
  - ☐ c) splice().
9. De las siguientes expresiones, ¿cuál devolverá false?:
  - ☐ a) `isFinite(-3.14)`
  - ☐ b) `isFinite(0)`
  - ☐ c) `isFinite("5/9")`

10. De las siguientes expresiones, ¿cuál devolverá true?:

- ☐ a) `isNaN(true)`
- ☐ b) `isNaN(8/0)`
- ☐ a) `isNaN(0/0)`

#### SOLUCIONES:

1. ☒ a ☐ b ☐ c

2. ☐ a ☐ b ☒ c

3. ☐ a ☒ b ☐ c

4. ☒ a ☐ b ☐ c

5. ☐ a ☐ b ☒ c

6. ☒ a ☐ b ☐ c

7. ☐ a ☒ b ☐ c

8. ☒ a ☐ b ☐ c

9. ☐ a ☐ b ☒ c

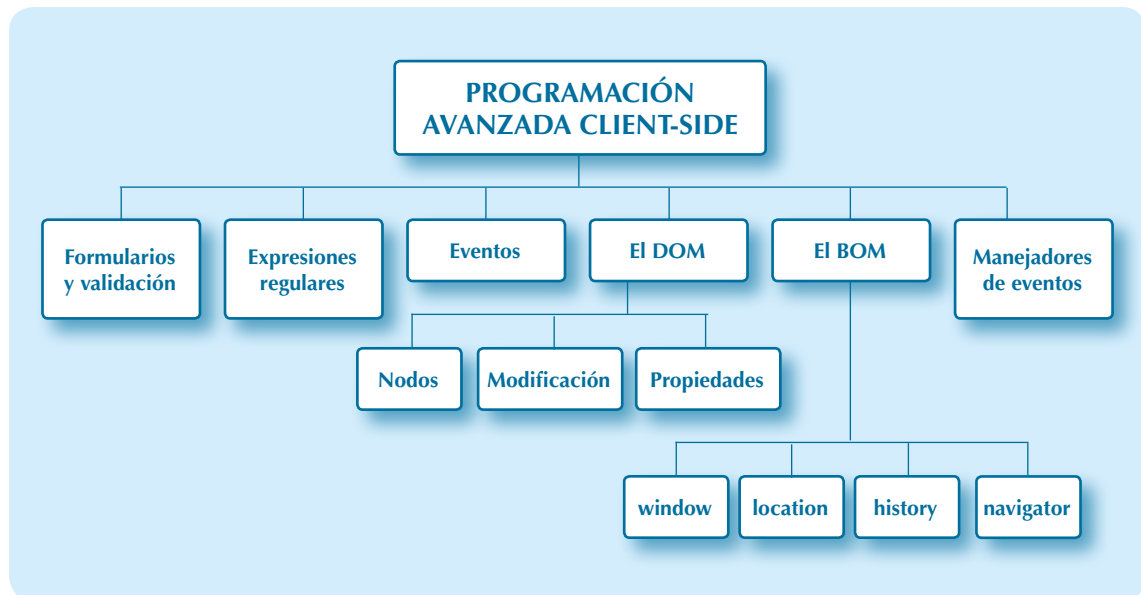
10. ☐ a ☐ b ☒ c

# El DOM y el BOM

## Objetivos

- ✓ Manejar la validación de campos en una interfaz HTML mediante JavaScript.
- ✓ Conocer y saber utilizar las expresiones regulares.
- ✓ Trabajar en profundidad el DOM (Document Object Model) conociendo sus propiedades y saber modificar el mismo.
- ✓ Identificar el concepto de BOM con los objetos que lo componen y saber utilizarlo en el desarrollo de aplicaciones web.

## Mapa conceptual



## Glosario

**BOM.** Objeto en el que se representa el navegador, no solo el documento. Mediante el BOM se puede acceder a datos como el historial de navegación, dimensiones de la ventana, etcétera.

**DOM.** Plataforma e interfaz de un documento que permite a los scripts y programas acceder y modificar dinámicamente su contenido, estructura y estilo.

**EventListener.** Código que se ejecuta cuando en la interfaz se genera un evento concreto (clic del ratón, pulsar una tecla en un campo de un formulario, etcétera).

**Expresión regular.** Secuencia de caracteres para realizar búsquedas, filtros o sustituciones en cadenas de caracteres.

**Tag.** Término en inglés que significa "etiqueta" y hace referencia a una palabra clave que sirve para describir un documento.

## 4.1. Introducción

En este capítulo, se trabajará en profundidad el concepto de *DOM* y de *BOM*. La ventaja que tiene JavaScript es poder acceder a ambos objetos, lo cual consigue que las aplicaciones sean interactivas.

Accediendo al DOM y al BOM, el programador va a poder conocer la potencia de la programación del lado del cliente. Este hecho permitirá que mucha de la lógica de las aplicaciones web corra en este lado dedicándose el servidor a temas más específicos de back-end.

## 4.2. Formularios

El trabajo con campos, ya sea en formularios o de forma aislada, es la base de las aplicaciones web. Con HTML5, es posible comprobar el contenido de muchos de los campos de una web, pero, en ocasiones, se necesitan ciertas validaciones y acciones extra que pueden ser realizadas en JavaScript.

### 4.2.1. La validación de campos

A continuación, se mostrará un ejemplo en el que se intenta validar un campo de texto, de tal manera que solamente se permita introducir un número entre 5 y 10:

```
<!DOCTYPE html>
<html>
<body>
<p>Introduzca un número entre 5 y 10:</p>
<input id="aqui" type="text">
<button type="button" onclick="dale()">Comprueba</button>
<p id="msg"></p>
<script>
function dale() {
 var msg, x;
 msg = document.getElementById("msg");
 msg.innerHTML = "";
 x = document.getElementById("aqui").value;
 try {
 if(x == "") throw "Está vacío";
 if(isNaN(x)) throw "No es un número";
 x = Number(x);
 if(x < 5) throw "No llega";
 if(x > 10) throw "Se pasa";
 }
 catch(err) {
 msg.innerHTML = "Resultado: " + err;
 }
}

</script>
</body>
</html>
```

En primer lugar, se puede observar que, en el código, existe un campo input de texto que es el que va a validarse con identificador (ID) "aqui".

Aparte del campo por validar, se añade un botón que, al presionarlo, ejecute la función `dale()`, que es la encargada de validar el campo anterior.

Para la validación, se utilizan las sentencias `try` y `catch`, de tal manera que, si el campo de texto contiene algún valor incorrecto, el código lanzará una excepción recogida con la sentencia `catch`. El bloque del `catch` se encargará de mostrar el texto lanzado en la excepción.



#### SABÍAS QUE...

La validación anterior es posible realizarla solamente con HTML.

### 4.3. Expresiones regulares

Las expresiones regulares son objetos que se pueden utilizar en los métodos `exec` y `test` del objeto `RegExp` y también en los métodos `search`, `search`, `replace` y `match` del objeto `string`.

Una expresión regular es una secuencia de caracteres que forman un patrón determinado. Los patrones se utilizan para realizar operaciones de búsqueda y reemplazamiento en strings. Las expresiones regulares son un concepto heredado del Unix donde se utilizan mucho en comandos del sistema como `grep`, `find`, `awk`.

Las expresiones regulares pueden ser sencillas o complejas dependiendo del objetivo que se necesite conseguir.

#### RECUERDA

- ✓ Las expresiones regulares son ampliamente utilizadas en la validación de campos por su potencia y efectividad.

Véase la sintaxis de una expresión regular:

```
/patrón/modificadores
```

Un ejemplo sencillo de una expresión regular sería el siguiente:

```
var patron = /myfpschool/i;
```

Dónde `/myfpschool/i` es la expresión regular. El objetivo es buscar la cadena “myfpschool” de forma case-insensitive (se le ha añadido el modificador `i`).

Obsérvese que no se utilizan comillas al definir el patrón. No es un string.

Aplicado a un código concreto, sería algo parecido a esto:

```
var cadena = "Aprende en MYfpschool javascript";
var patron = /myfpschool/i;
var pos = cadena.search(patron);
```

Al ejecutar el código anterior, el valor de la variable pos sería 11 porque la cadena myfpschool aparece en dicha posición.

Véase un ejemplo de utilización con otras expresiones regulares:

```
<!DOCTYPE html>
<html lang=»en» dir=»ltr»>
 <head>
 <meta charset="utf-8">
 <title>Expresiones regulares</title>
 </head>
 <body>

 <script>

var cadena = "Sintesis1@sintesis.com";
var patron = /\w/g; // \w selecciona los caracteres de una cadena de
caracteres
var resultado = cadena.match(patron);
//devuelve un array de palabras y números sin otros símbolos
//['S','i','n','t','e','s','i','s','l','s','i','n','t','e','s','i','s',
s','c','o','m']
console.log(resultado);

var patron = /\d/g; // \d selecciona los dígitos de una cadena de ca-
racteres
var resultado = cadena.match(patron);
//devuelve un array con los dígitos
//['1']
console.log(resultado);

var patron = /\W/g; // \W selecciona los NO caracteres de una cadena
de caracteres
var resultado = cadena.match(patron);
//devuelve un array con los no caracteres
//['@','.']
console.log(resultado);

var cadena = "Aprende JavaScript en Sintesis";
var patron = /\s/g; // \s elimina los espacios de una cadena de carac-
teres
var resultado = cadena.match(patron);
//devuelve un array de los elementos del string sin espacios en blanco
//['A','p','r','e','n','d','e','J','a','v','a','S','c','r','i','p','t',
t','e','n','s','i','n','t','e','s','i','s']
console.log(resultado);

var patron = /r+/g; // r+ busca si en la cadena tiene una o más erres.
var resultado = cadena.match(patron);
```

```

//devuelve un array de los elementos encontrados
//['r', 'r']
console.log(resultado);

var cadena = "Ponte a 100 o a 1000 con Sintesis";
var patron = /\d{2}/g; // busca si hay una secuencia de dos dígitos
var resultado = cadena.match(patron);
//devuelve un array de las secuencias encontradas
//['10', '10', '00']
console.log(resultado);

var patron = /\d{2,3}/g; // busca si hay una secuencia de dos o tres
dígitos
var resultado = cadena.match(patron);
//devuelve un array de las secuencias encontradas
//['100', '100']
console.log(resultado);
</script>

</body>
</html>

```



#### PARA SABER MÁS

El objeto RegExp, como se ha explicado, tiene, entre otros, los métodos `exec` y `test` para ejecutar una expresión regular y testearla respectivamente. Las siguientes líneas de código tendrían la misma funcionalidad:

```

var patron = /myfpschool/i;
var re = new RegExp('/myfpschool/i');

```

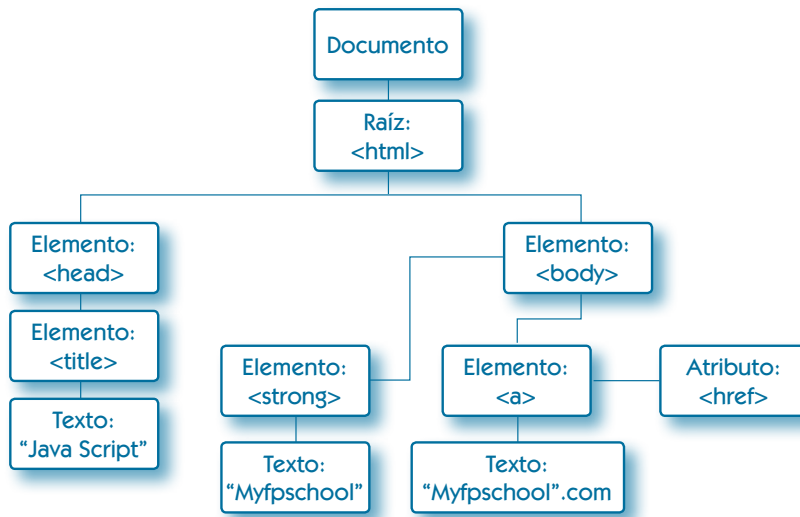
Para más información sobre las expresiones regulares, se puede acceder a un recurso de MDN a través del siguiente código QR:



## 4.4. DOM

Según el W3C, el DOM es una plataforma e interfaz de un documento que permite a los scripts y los programas acceder y modificar dinámicamente su contenido, estructura y estilo.

En la figura 4.1, puede observarse un ejemplo visual del DOM.

**Figura 4.1**

Detalle del DOM de una página concreta  
(en la raíz, el objeto document).

Como ya se ha comentado, JavaScript puede acceder al DOM (Document Object Model) de cualquier página web. Al contrario que las páginas HTML estáticas, con JavaScript, se puede hacer que una web sea dinámica cambiando el contenido HTML de cualquier elemento, cambiando el estilo CSS de esta, reaccionando a los eventos del DOM o incluso añadiendo o eliminando elementos HTML de tu web.

## FUNDAMENTAL

### *El objeto document*

Cuando se carga un documento en el navegador, se crea el objeto document, el cual, además de ser la raíz de todo el documento HTML, tiene una serie muy amplia de propiedades y métodos. Algunas de ellas se citarán a continuación:

- *Propiedades:* activeElement, cookie, domain, images, links, referrer.
- *Métodos:* open(), querySelector(), getElementById(), createEvent().

Desde el capítulo 1, se ha visto cómo se podía acceder al DOM y modificar su contenido. Recuerdese uno de los primeros ejemplos:

```

<!DOCTYPE html>
<html>
<body>
 <h1>Modificando el código HTML</h1>
 <p id="prueba">Modificando el contenido.</p>

```

```

 <button type="button" onclick="document.getElementById('prueba').
 innerHTML = 'CAMBIANDO el contenido!'">¡Dale!</button>
 </body>
</html>

```

Para ello, se utilizaba el método `getElementById()`, que permitía obtener el identificador (ID) de un elemento HTML, en este caso, el elemento prueba. Una vez seleccionado el elemento concreto, mediante la propiedad `innerHTML`, será posible cambiar el contenido de un elemento HTML.

Además de buscar elementos por el ID, se pueden buscar elementos por el tag [utilizando el método `getElementsByTagName()`] o también por la clase [utilizando el método `getElementsByClassName()`].

Observe el siguiente código:

```

<p id="texto">Curso de JavaScript</p>
<p id="texto2">Curso de React</p>
<script>
 var a = document.getElementById("texto");
 var b = a.getElementsByTagName("strong");
 alert (b[0].innerHTML);
</script>

```

En el ejemplo anterior, se muestra un mensaje de alerta con el texto “JavaScript”. En la primera línea del script, se selecciona el párrafo con ID igual a “texto” y, en la segunda, se extraen de ese texto todas las etiquetas en negrita, con lo cual, en la posición 0, se almacenará el objeto JavaScript.

Véase un ejemplo más completo de utilización del DOM con JavaScript. A continuación, se muestra un ejemplo de test sobre nomofobia:

```

<!DOCTYPE html>
<html>
<body>

<form id="frm1">

 <input type="checkbox" name="nomo" >
 1. Cuando mandas un mensaje por whatsapp esperas
 siempre al doble check. Si no aparece vuelves a abrir el terminal
 para revisarlo al rato.

 <input type="checkbox" name="nomo" >
 2. Antes de acostarte siempre miras el móvil a ver
 si tienes mensajes o notificaciones.

 <input type="checkbox" name="nomo" >
 3. Te despiertas antes de tiempo para jugar, mandar
 mensajes, actualizar perfiles,... con el teléfono móvil.

```

```


 <input type="checkbox" name="nomo" >
 4. Si sales de casa sin el móvil volverías a co-
 gerlo aunque llegues tarde a tu cita.

 <input type="checkbox" name="nomo" >
 5. Tienes miedo a quedarte sin batería.

 <input type="checkbox" name="nomo" >
 6. Cuando tienes la batería baja desactivas cier-
 tas aplicaciones u opciones del teléfono como la WiFi, bluetooth
 para no quedarte sin batería.

 <input type="checkbox" name="nomo" >
 7. Tienes ansiedad cuando tienes llamadas perdi-
 das. Llamas a los números y te preocupas si no responden.

 <input type="checkbox" name="nomo" >
 8. Miras la cobertura cuando estás en algún sitio
 con los amigos, esperando, etc.

 <input type="checkbox" name="nomo" >
 9. Sueles hacer alguna otra cosa mientras que mi-
 ras al móvil como comer, lavarte los dientes, etc.

 <input type="checkbox" name="nomo" >
 10. Vas al baño siempre con el móvil.

</form>

<p>Recuerda marcar todas las cuestiones arriba expuestas que creas que
siempre realizas.

</p>

<button onclick="myFunction()">Comprueba si sufres nomofobia</but-
ton>

<h3><p id="resultado"></p></h3>
<h4><p id="consejo"></p></h4>

```

```

<script>
function myFunction() {
 var cont=0
 var x = document.forms["frm1"];
 var text = "";
 var i;
 for (i = 0; i < x.length ;i++) {
 if (x.elements[i].checked){ cont++; }
 }

 document.getElementById("resultado").innerHTML = "Resultado: En
 principio no tienes nada de que preocuparte.";
 document.getElementById("consejo").innerHTML = "Resultado: En
 principio no tienes nada de que preocuparte.";

 if (cont>5){
 document.getElementById("resultado").innerHTML = "Resultado:
 Empiezas a tener signos de dependencia del móvil. ";
 document.getElementById("consejo").innerHTML = " Consejo: Pue-
 des utilizar técnicas como apagar el móvil cuando no lo necesi-
 tas, cuando duermes, etc.";
 }

 if (cont>6){
 document.getElementById("resultado").innerHTML = "Resultado:
 Tienes una gran dependencia del móvil. ";
 document.getElementById("consejo").innerHTML = "Consejo:Debe-
 rías seguir un plan de desintoxicación del
 móvil como por ejemplo dejar el móvil en casa cuando vas a com-
 prar, apagarlo durante la noche, apagarlo durante horas de clase
 o trabajo, etc.";
 }

 if (cont>8){
 document.getElementById("resultado").innerHTML = "Resultado:
 Tus síntomas de dependencia son muy preocupantes. ";
 document.getElementById("consejo").innerHTML = "Consejo: Ade-
 más de todas las técnicas de los apartados anteriores deberías
 plantearte un plan de desintoxicación del móvil que consista en
 estar una o dos semanas sin utilizarlo. Si ves que no puedes
 hacerlo por ti mismo, pide ayuda a un profesional.";
 }
}
</script>

</body>
</html>

```

En el script del ejemplo anterior, se puede ver cómo se hace un recorrido por todos los campos del formulario [función `myFunction()`] y se van registrando en la variable `cont` de la función anterior todos aquellos que estén seleccionados cambiando el contenido de dos párrafos con ID “resultado” y “consejo”.

### Recurso web

www

Si necesitas saber más sobre el DOM en la página oficial del W3C, puedes acceder desde el siguiente código QR:



#### 4.4.1. El acceso al DOM. El método `document.querySelector()`

El método `querySelector()` del objeto `document` devuelve el primer elemento que encuentre y que concuerde con el selector que se le introduzca como parámetro. Es importante recalcar que solamente devolverá el primer elemento y no todos. Para seleccionar todos los elementos que concuerden con un selector, habrá que utilizar el método `querySelectorAll()`.

Véase un ejemplo de uso en una página web:

```
<!DOCTYPE html>
<html>
<body>

<h3 class="ejemplo">introduzca una expresi&ocute;n numérica</h3>

<label for="texto">Introducir datos:</label>
<input type="text" id="texto">

<button>Ejecutar expresi&ocute;n</button>

<p class="ejemplo">Un párrafo de la clase ejemplo.</p>

<p>otro párrafo.</p>

<button onclick="laFuncion()">Cambiar color</button>

<script>
function laFuncion() {
 document.querySelector(".ejemplo").style.backgroundColor = "red";
 document.querySelector("p").innerHTML = "rojo";
}
```

```

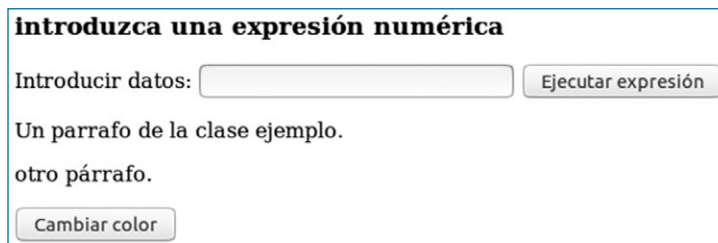
var input = document.querySelector('input');
var boton = document.querySelector('button');
var parrafo = document.querySelector('p');
boton.onclick = function() {
 var dato = input.value;
 parrafo.innerHTML = eval(dato);
}

</script>

</body>
</html>

```

El aspecto de la página web creada con el código anterior será el que se muestra en la figura 4.2.



**Figura 4.2**  
Resultado de la página  
creada con el código  
anterior.

Como se puede observar en el código, cuando se pulsa el botón “Cambiar color”, se ejecuta la función de nombre *laFuncion()*.

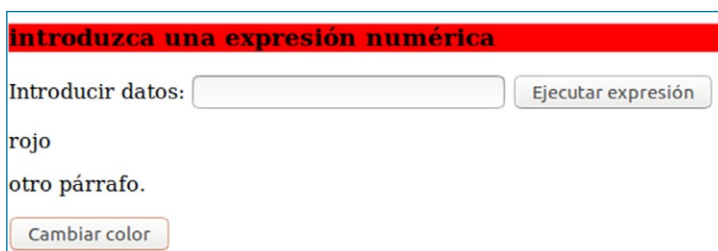
```

function laFuncion() {
 document.querySelector(".ejemplo").style.backgroundColor = "red";
 document.querySelector("p").innerHTML = "rojo";
}

```

En esta función, se cambiará el color de fondo (primera línea de la función) de aquel elemento de la clase ejemplo, que, en este caso, será el título h3. Además, se seleccionará el primer párrafo (segunda línea de la función) y cambiará el texto que tenga por “rojo”.

El resultado de pulsar este botón será el que se indica en la figura 4.3.



**Figura 4.3**  
Pulsando el botón  
cambiar color.

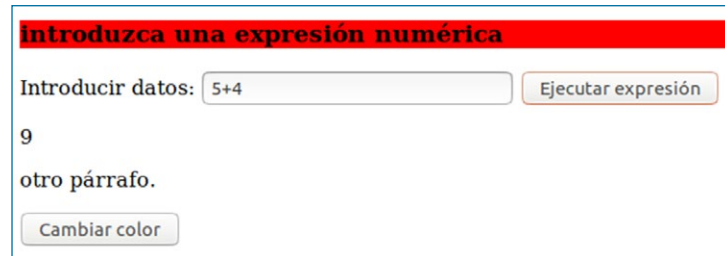
En la segunda parte del script, se selecciona el campo input, el primer botón y el primer párrafo (véanse las tres primeras líneas siguientes) y se le asigna al botón una función.

```
var input = document.querySelector('input');
var boton = document.querySelector('button');
var parrafo = document.querySelector('p');
boton.onclick = function() {
 var dato = input.value;
 parrafo.innerHTML = eval(dato);
}
```

En esta función, se utiliza la función eval para ejecutar la expresión que se introduzca en el campo input.

El resultado de introducir la expresión 5+4 y ejecutar el botón “Ejecutar expresión” será el que se muestra en la figura 4.4.

**Figura 4.4**  
Ejecutando la expresión  
5+4.



## 4.5. Eventos del DOM

Ya en el segundo capítulo se hizo una rápida introducción a los eventos que se podían capturar en una página web, así como a los manejadores de eventos. En esta sección, se profundizará más sobre estos conceptos.

JavaScript puede reaccionar a cualquier evento que ocurra en una página web. Hasta ahora, el evento utilizado era el clic sobre un botón de la página web, pero ¿qué tipos de eventos puede controlar JavaScript? A continuación, se muestran algunos de los eventos que pueden ser capturados:

- Pulsar una tecla.
- Enviar un formulario.
- Modificar un campo de texto.
- Cuando el ratón pasa sobre un elemento.
- Cuando se carga una imagen.
- Cuando el usuario hace clic sobre un elemento.
- Cuando se carga una página web.

Véase un ejemplo de control del ratón sobre un texto:

```
<!DOCTYPE html>
<html>
<body>
```

```

<div onmouseover="dentro(this)" onmouseout="fuera(this)">
Editorial Síntesis</div>

<script>
function dentro(obj) {
 obj.innerHTML = "Desarrollo web en entorno cliente"
}
function fuera(obj) {
 obj.innerHTML = "Editorial Síntesis"
}
</script>

</body>
</html>

```

En el ejemplo anterior, se controla si el ratón está sobre la etiqueta <div> cambiando el texto que la contiene y, en caso de que el ratón deje de estar sobre ella, se restablecerá el valor original de la etiqueta.

#### RECUERDA

- ✓ Todos los eventos que ocurren a un objeto están basados en el objeto event. Este objeto contiene las propiedades y métodos que son comunes a todos los eventos.

Actualmente, los programadores prefieren el método `addEventListener` en vez de añadir el evento en todos y cada uno de los elementos HTML afectados. Esta forma de programar es más limpia, puesto que independiza el código HTML del código JavaScript. El W3C pone énfasis en cambiar el modelo tradicional de registro de eventos por este nuevo método.

Véase otro ejemplo que comprueba cuando se produce la carga de la página si las cookies están habilitadas o no:

```

<!DOCTYPE html>
<html>
<body onload="miraCookies()">
<p id="muestracookies"></p>
<script>
function miraCookies() {
 var text = "";
 if (navigator.cookieEnabled == true) {
 text = "Las cookies están habilitadas.";
 } else {
 text = "Las cookies están deshabilitadas.";
 }
 document.getElementById("muestracookies").innerHTML = text;
}

```

```

}
</script>
</body>
</html>

```

Como se puede observar, en este caso, se ejecutará el script en la carga de la página (evento onload).

## 4.6. Manejadores de eventos

Cuando ocurre un evento de un objeto del documento o página web, es posible con JavaScript asociarlo a una función. Ya en el primer capítulo se pudieron ver scripts que asociaban al evento click del ratón sobre un botón con una función JavaScript.

Con el método `addEventListener`, es posible asociar evento y función JavaScript desde el propio script limpiando de código el HTML. Véase un ejemplo de utilización:

```

<!DOCTYPE html>
<html>
<body>
<button id="unboton">Dale</button><p id="texto"></p>

<script>
function MuestraFecha() {
 document.getElementById("texto").innerHTML = Date();
}
document.getElementById("unboton").addEventListener("click", MuestraFecha);
</script>

</body>
</html>

```

Como puede observarse, por un lado, está la función `MuestraFecha()`, que asigna al contenido HTML del elemento con `id=texto` la fecha actual.

Por otro lado, está el método `addEventListener` asociado al botón que se ha creado con `identificador=unboton`. Lo que hace ese método es asociar el evento click del botón con la función `MuestraFecha`.



### Actividad propuesta 4.1

Del ejemplo expuesto en este apartado, desliga el código JavaScript a otro fichero y haz que funcione la página web. Coloca el código JavaScript en una subcarpeta `scripts` para tener una estructura de proyecto más ordenada.

## 4.7. Nodos del DOM

Cuando el navegador recibe la página, la interpreta y va creando una estructura arborescente con los elementos recibidos llamada *DOM*.



### TOMA NOTA

Hasta que el navegador no haya construido totalmente el árbol (DOM) cargándose la página por completo, no será posible acceder hasta él.

En el DOM, existen muchos tipos de nodo como *Attr*, *CDataSection*, *Comment*, *Document*, *DocumentFragment*, *DocumentType*, *Element*, *Entity*, *EntityReference*, *Notation*, *ProcessingInstruction* o *Text*. Generalmente, los programadores web utilizan sobre todo cuatro nodos cuando desean manipular el DOM:

1. *Attr*. Este nodo representa los atributos de las etiquetas, que tienen el formato atributo=valor.
2. *Document*. Es el nodo más importante que constituye la raíz del que derivan los demás nodos del DOM.
3. *Element*. Cada etiqueta tendrá un nodo de tipo *element*. Estos nodos pueden tener atributos y, además, otros nodos podrán derivar de ellos.
4. *Text*. En este nodo, se almacena el texto de una etiqueta.

### 4.7.1. ¿Cómo se accede a los nodos (elementos HTML)?

En JavaScript, se puede acceder a los nodos del DOM utilizando las funciones que el DOM proporciona. Las operaciones más comunes al acceder al DOM son las siguientes:

- Acceder al valor de un atributo.
- Establecer el valor de un atributo.
- Crear o añadir nuevos nodos.
- Borrar nodos.
- Mover nodos de su lugar en el árbol.

Los nodos se pueden acceder de forma directa, a través de un nodo específico y realizando las operaciones que se deseen sobre él o a través de todos sus antecesores, lo cual es una operación menos usada y más tediosa. Lo normal es acceder a los nodos de forma directa mediante alguna de las siguientes funciones:

- *getElementById()*. Es el más utilizado. Se accede por el identificador que le ha dado el programador al elemento y, generalmente, este identificador es único en la página (debería serlo).

- `getElementsByTagName()`. Acceder a los nodos mediante su tipo de etiqueta. Se accederá a todos los nodos que tengan un tipo de etiqueta concreta y esta llamada devolverá un array de elementos.
- `getElementsByName()`. Acceder a los nodos por su nombre. En HTML5, ya no se utiliza dado que el nombre (name) ha sido reemplazado por su identificador (ID).
- `getElementsByClassName()`. Se accede al elemento por el nombre de la clase (CSS) o selector CSS a la que pertenece. La llamada a este método devuelve una colección (array) de elementos, el cual se podrá ir recorriendo para acceder a todos ellos.
- `querySelector()`. Es otro método del DOM que permite acceder al primer elemento que concuerde con el selector que se le pasa como parámetro. Es un método parecido al anterior.

A continuación, se muestra un ejemplo del método `getElementsByClassName()`:

```
<!DOCTYPE html>
<html>
<body>

<p class="parrafo">Primer párrafo.</p>

<p class="parrafo">Segundo párrafo.</p>

<button onclick="dale()">Cambia el texto</button>

<script>
function dale() {
 var p = document.getElementsByClassName("parrafo");
 p[0].innerHTML = "Párrafo primero.";
 p[1].innerHTML = "Párrafo segundo.";
}
</script>

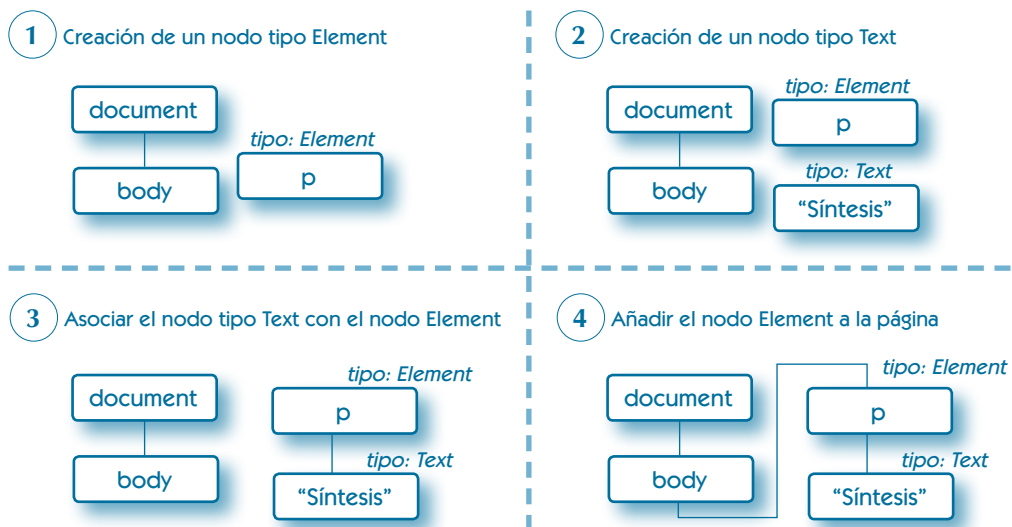
</body>
</html>
```

Como se puede ver en el ejemplo anterior, se accede a todos los elementos del tipo `<p>` y se modifica su contenido.

#### 4.7.2. ¿Cómo se crea un nuevo nodo?

Como se puede ver, los pasos para crear un nuevo nodo son los siguientes:

- *Paso 1.* Crear un nuevo nodo de tipo `Element`.
- *Paso 2.* Crear un nuevo nodo de tipo `Text` (para albergar el contenido del elemento anterior).
- *Paso 3.* Vincular el nodo `Text` al nodo `Element`.
- *Paso 4.* Vincular el nodo `Element` a la página de tal manera que el nodo esté en el lugar correspondiente donde se quiera insertar el elemento.

**Figura 4.5**

Proceso gráfico de creación de un nodo tipo párrafo.

El código de los pasos anteriores sería el siguiente:

```
// Crear nodo de tipo Element
var parrafo = document.createElement("p");
// Crear nodo de tipo Text
var contenido = document.createTextNode("Síntesis");
// Vincular el nodo Text como hijo del nodo Element
parrafo.appendChild(contenido);
// Vincular el nodo Element como hijo de la página
document.body.appendChild(parrafo);
```

Como se puede ver en el ejemplo anterior, se han utilizado las siguientes funciones del DOM:

- *createElement()*. Para crear un nodo de tipo Element.
- *createTextNode()*. Para crear un nodo de tipo Text.
- *appendChild()*. Para vincular un nodo hijo a un nodo padre.

### 4.7.3. ¿Cómo se elimina un nodo?

Imagínese que se tiene un elemento HTML en una web como el siguiente:

```
<p id="editorial">Editorial Síntesis</p>
```

Para eliminar dicho elemento, habrá que invocar desde el nodo padre a la función `removeChild()`, la cual requerirá como parámetro el nodo que se va a borrar.

Para acceder al nodo padre de un elemento HTML, se puede utilizar la propiedad `parentNode`. Véase un ejemplo de código JavaScript para eliminar el párrafo anterior de una página web:

```
var pEditorial = document.getElementById("editorial");
pEditorial.parentNode.removeChild(pEditorial);
```

Como se puede apreciar, eliminar un nodo o elemento de una página web es más sencillo que su creación.

#### RECUERDA

- ✓ Cuando se elimina un nodo, se eliminarán todos los nodos hijos que pueda poseer. De esa manera, eliminar un nodo puede equivaler a eliminar más de un elemento.

### 4.7.4. Práctica guiada: creación y eliminación de elementos

En el siguiente ejercicio, se va a mostrar una lista de elementos y dos botones: añadir y eliminar. Pulsando el botón añadir, se podrá dar de alta un nuevo elemento a la lista y, pulsando eliminar, se eliminará el último elemento par.

El aspecto de la página será el que se observa en la figura 4.6.

Pulsa el botón para  o  un elemento a la lista.

1. primer dato
2. segundo dato
3. tercer dato
4. cuarto dato

**Figura 4.6**  
Aspecto de la página.

El código de la página será el siguiente:

```
<!DOCTYPE html>
<html>
<body>

<p>Pulsa el botón para
 <input type="button" value="añadir" onclick="añade();" />
 o
 <input type="button" value="eliminar" onclick="elimina();" />
 un elemento a la lista.
</p>

 <ol id="lista">
```

```

 primer dato
 segundo dato
 tercer dato
 cuarto dato

<script>

function aniade(){

 do{
 var palabras = prompt('Introduce el texto que quieras.');
```

while (palabras == "");

```

 // Si ha pulsado cancelar no ejecutará el siguiente código
 if (palabras != null){
 var elemento = document.createElement('li');
 var texto = document.createTextNode(palabras);
 var lista = document.getElementById('lista');

 // con esto creamos el nodo li y el texto.
 elemento.appendChild(texto);
 lista.appendChild(elemento);
 }
 }

function elimina(){
 var lista = document.getElementById('lista');

 if(lista.children.length > 1){
 var elemento = lista.children[lista.children.length -1];
 //Si es par borro el último.
 if(lista.children.length % 2 == 0){
 elemento.parentNode.removeChild(elemento);
 }else{
 // si no es par tengo que borrar el anterior
 elemento = lista.children[lista.children.length -2];
 elemento.parentNode.removeChild(elemento);
 }
 }else{
 alert("No hay elementos a borrar");
 }
}

</script>
</body>
</html>

```

Como se puede observar en el código, la función que añade elementos a la lista se ha visto en el apartado de la creación de un nodo, pero la de borrado añade una dificultad y es que el nodo por borrar tiene que ser par. Para ello, se utiliza la expresión `(lista.children.length % 2 == 0)`. Si se cumple esa expresión, el último nodo de la lista será par y, en caso contrario, habrá que borrar el anterior a dicho nodo.

## 4.8. Propiedades del DOM

A continuación, se muestran algunas propiedades avanzadas de uso del DOM.

### 4.8.1. El acceso a los nodos

En apartados anteriores, se ha estudiado cómo se puede acceder a los elementos del DOM. El manejo de los métodos de acceso es importante, puesto que el programador puede, por ejemplo:

1. *Conocer el número de enlaces que tiene una página web.* Para ello, se puede utilizar el método `getElementsByTagName()` del DOM preguntando por los elementos `<a>`.

```
var numEnlaces = document.getElementsByTagName('a');
```

La línea anterior almacenará todos los enlaces de la página (en un array de objetos), pero, para saber cuántos enlaces hay en dicho array, habrá que utilizar la función `length` ya estudiada anteriormente.

```
var mensaje = 'Se han encontrado '+numEnlaces.length+' enlaces';
```

2. *Conocer la dirección a la que apunta el último enlace.* Bastaría con acceder a la última posición del array y preguntar por la propiedad `href` del objeto contenido en él.

```
var mensaje = 'El penúltimo enlace apunta a '+numEnlaces[numEnlaces.length-1].href;
```

3. *Conocer el número de enlaces que están en el tercer párrafo del documento.* Bastaría con extraer todos los párrafos del documento para, posteriormente, extraer los enlaces.

```
var parrafos = document.getElementsByTagName('p');
enlaces = parrafos[2].getElementsByTagName('a');
numEnlaces = enlaces.length;
```

Ahora, en `numEnlaces`, se almacenará el número de enlaces contenidos en el tercer párrafo.

### 4.8.2. Capturar ciertos eventos

Para capturar el evento `click` en una página, se le puede asignar al evento `document.onclick` una función creada por el programador. Si se desea saber dónde ha hecho clic el usuario en la página web, habrá que colocar la siguiente línea:

```
document.onclick = hahehoclickaqui;
```

El script hahehoclickaqui será una función que se disparará cuando el usuario haga clic. El código de esta función puede ser el siguiente:

```
function hahehoclickaqui(elEvento){
 var evento = elEvento || window.event;

}
```



#### PARA SABER MÁS

¿Qué significa el `elEvento || window.event`? En los navegadores tipo Internet Explorer, el objeto event se obtiene directamente mediante:

```
var evento = window.event;
```

Por otra parte, en el resto de navegadores, el objeto event se obtiene mágicamente a partir del argumento que el navegador crea automáticamente:

```
function manejadorEventos(elEvento) {
 var evento = elEvento;
}
```

Si se quiere programar una aplicación que funcione correctamente en todos los navegadores, es necesario obtener el objeto event de forma correcta según cada navegador. El siguiente código muestra la forma correcta de obtener el objeto event en cualquier navegador:

```
function manejadorEventos(elEvento) {
 var evento = elEvento || window.event;
}
```

Una vez obtenido el objeto event, ya se puede acceder a toda la información relacionada con el evento, que depende del tipo de evento producido.

Por lo tanto, en `evento.clientX` y `evento.clientY`, se almacenarán las coordenadas donde el usuario haya hecho clic.

## 4.9. Modificando el DOM

Ya se ha visto en apartados anteriores cómo modificar la estructura del DOM añadiendo o eliminando nodos, accediendo a ellos y capturando ciertos eventos. En este apartado, se va a trabajar con mayor profundidad la modificación del DOM cambiando propiedades de los elementos como textos, mostrando y ocultando elementos, deshabilitando objetos o comprobando la información introducida en campos de texto.

### 4.9.1. Cambiar el tamaño del texto

Véase la siguiente página web:

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Cambiano el tamaño del texto</title>
</head>
<body>

 <p id="parrafo1">Texto a agrandar, reducir o dejar de forma
 original</p>

 <button type="submit" onclick="modificarTexto('aumentar', 0.05);
 ">Agrandar</button>
 <button type="submit" onclick="modificarTexto('reducir', 0.05);
 ">Reducir</button>
 <button type="submit" onclick="modificarTexto('original', 0.05);
 ">Original</button>

<script>
 var tamaño = 1;
 var tamañoOriginal = 1;
 function modificarTexto(elEvento, pixel){
 var elemento = document.getElementById('parrafo1');
 switch(elEvento){
 case 'aumentar':
 if(tamaño > 2){
 alert('superado el tamaño máximo');
 break;
 }else{
 tamaño = tamaño + pixel;
 break;
 }
 case 'reducir':
 if(tamaño < .9){
 alert('superado el tamaño mínimo');
 break;
 }else{
 tamaño = tamaño - pixel;
 break;
 }
 case 'original':
 tamaño = tamañoOriginal;
 break;
 }
 }
```

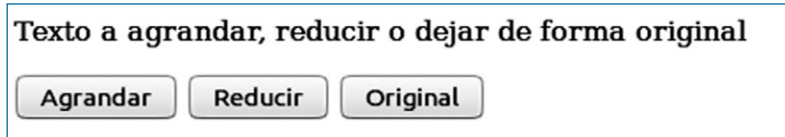
```

 elemento.style.fontSize = tamano+'em';
 }

</script>
</body>
</html>

```

El aspecto de esta página web será el que se observa en la figura 4.7.



**Figura 4.7**  
Aspecto de la página web anterior.

Como se puede ver, existen tres botones en ella que pueden agrandar, disminuir o restablecer el tamaño original del párrafo con ID “parrafo1”. En el evento onclick de los botones, se hace una llamada para modificar el párrafo de la página web.

### Actividad propuesta 4.2



Partiendo del código expuesto en este apartado, comprueba que funciona en tu navegador e intenta realizar una página web con alguna variante.

## 4.9.2. Mostrar y ocultar elementos de una página web

Cambiar el estilo es muy útil en muchas aplicaciones JavaScript. A continuación, se presenta un ejemplo más sofisticado en el que se muestra o se oculta un elemento de una página web. Para ello, se han utilizado dos clases en CSS y, mediante JavaScript, se irá cambiando la clase al elemento del DOM:

```

<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Ocultando y mostrando texto</title>
 <style>
 .oculto {
 visibility: hidden;
 }
 </style>

```

```

 .visible {
 visibility: visible;
 }
 </style>
</head>
<body>
 <p id="parrafo1"> Este es el primer párrafo</p>
 <p id="parrafo2"> Este es el segundo párrafo</p>

 <button type="submit" onclick="ocultar('parrafo1');">Ocultar/mostrar<
 /button>
 <button type="submit" onclick="ocultar('parrafo2');">Ocultar/mostrar<
 /button>

<script>
 function ocultar(parrafo){
 var elemento = document.getElementById(parrafo);
 if (elemento.className != 'oculto'){
 elemento.className = 'oculto';
 // también funcionaría la siguiente línea pero se ha prefe-
 rido modificar el estilo CSS
 //elemento.style.visibility = 'hidden';

 }else{
 elemento.className = 'visible';
 // también funcionaría la siguiente línea pero se ha prefe-
 rido modificar el estilo CSS
 //elemento.style.visibility = 'visible';
 }
 }

</script>
</body>
</html>

```

De la manera anterior, se puede ocultar y mostrar un elemento de una página web de forma alternativa. Cambiar la clase CSS a cualquier elemento del DOM puede dar al programador muchos recursos para hacer formularios o páginas web más interactivas e intuitivas.



### Actividad propuesta 4.3

Basándote en el código expuesto en este apartado, logra que tu programa pueda mostrar u ocultar cuatro párrafos.

### 4.9.3. Deshabilitar objetos

En ocasiones, los programadores necesitan deshabilitar ciertos elementos de la interfaz dependiendo de la interacción con el usuario. A continuación, se muestra el código de un botón que hace que se desactiven o activen ciertos elementos de la página web (en este caso, otros botones):

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Habilitar y deshabilitar elementos del DOM</title>
</head>
<body>
 <input type="submit" value="Boton1">
 <input type="submit" value="Boton2">
 <input type="submit" value="Boton3">
 <input type="submit" value="Boton4">
 <input type="submit" value="Boton5">

 <button type="submit" onclick="bloquear();" id="boton">Desactivar
 todos</button>

<script>
 function bloquear(){
 var elementos = document.getElementsByTagName('input');
 if (elementos[0].className != 'desactivo'){
 for (var i = 0; i < elementos.length; i++) {
 elementos[i].disabled = true;
 elementos[i].className = 'desactivo';
 }
 document.getElementById('boton').innerHTML = "Activar to-
 dos";
 }else{
 for (var i = 0; i < elementos.length; i++) {
 elementos[i].disabled = false;
 elementos[i].className = 'activo';
 }
 document.getElementById('boton').innerHTML = "Desactivar
 todos";
 }
 }
</script>
</body>
</html>
```

#### 4.9.4. Comprobar que se introduce información en un campo de texto

Muchas veces, la validación de un campo de texto implica que contenga información. A veces, los usuarios introducen espacios en blanco que hacen que un programa no funcione de forma correcta. Se muestra un código que obliga a que el usuario introduzca algo en un campo de texto con id="campo".

```
valor = document.getElementById("campo").value;
if(valor == null || valor.length == 0 || /^s+$/ .test(valor)) {
 return false;
}
```

Para que se dé por completado un campo de texto obligatorio, se comprueba que el valor introducido sea válido, que el número de caracteres introducido sea mayor que 0 y que no se hayan introducido solo espacios en blanco.

- La palabra reservada *null* es un valor especial que se utiliza para indicar “ningún valor”. Si el valor de una variable es *null*, la variable no contiene ningún valor de tipo objeto, array, numérico, cadena de texto o booleano.
- La segunda parte de la condición obliga a que el texto introducido tenga una longitud superior a 0 caracteres, esto es, que no sea un texto vacío.
- Por último, la tercera parte de la condición que es una expresión regular (*test(valor)*) obliga a que el valor introducido por el usuario no solo esté formado por espacios en blanco. Esta comprobación se basa en el uso de *expresiones regulares*. Por lo tanto, solo es necesario copiar literalmente esta condición, poniendo especial cuidado en no modificar ningún carácter de la expresión.

A continuación, se mostrará cómo se validaría un email. Obsérvese que la expresión regular en este caso es más compleja, puesto que hay que realizar más comprobaciones:

```
valor = document.getElementById("campo").value;
if(!(/\w+([-\.']\w+)*@\w+([-\.']\w+)*\. \w+([-\.']\w+)* / .test(valor))) {
 return false;
}
```

A veces, también el programador tiene que tratar con teléfonos. A continuación, se muestra la expresión regular para validar teléfonos:

```
valor = document.getElementById("campo").value;
if(!(/\^d{9}$ / .test(valor))) {
 return false;
}
```

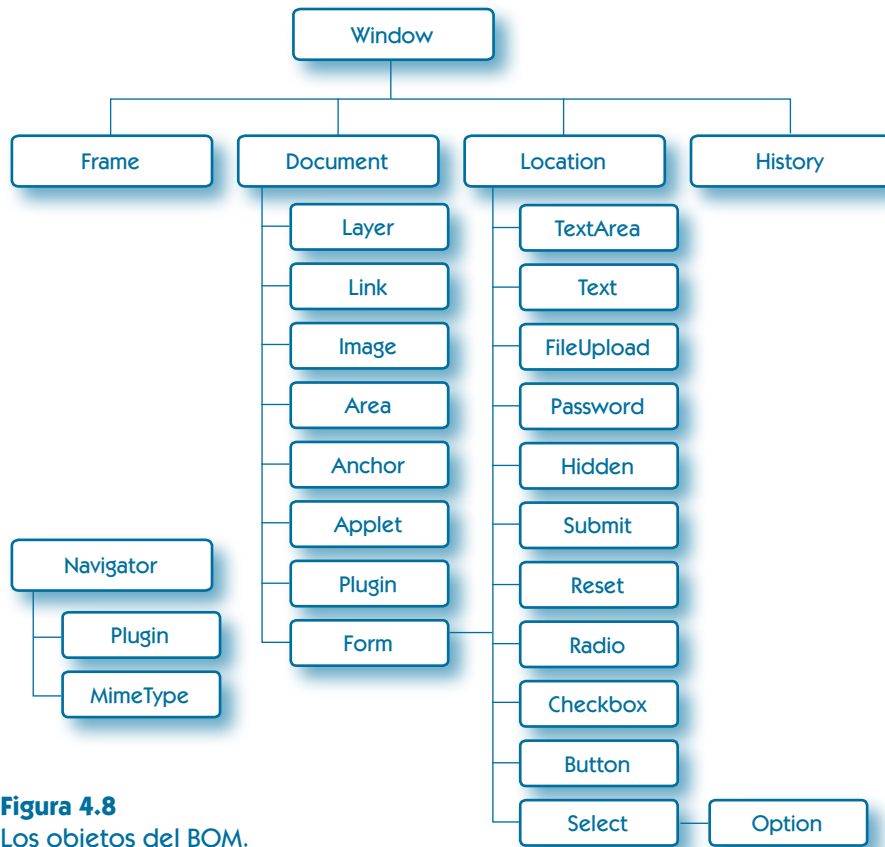


#### Actividad propuesta 4.4

Realiza un formulario en el que se valide una cantidad de euros, el nombre de un usuario –que tendrá que ser mayor a 4 caracteres–, su email y su teléfono.

## 4.10. BOM

El BOM (Browser Object Model) en su momento fue implementado por los navegadores para que JavaScript pudiese hacer uso de sus métodos y propiedades de forma uniforme.



**Figura 4.8**  
Los objetos del BOM.

Como se puede observar en la figura 4.8, el objeto raíz del BOM es el objeto window, que está soportado por cualquier navegador y que, a su vez, contiene otros objetos de menor rango jerárquico, como puede ser el objeto document (muy importante también) u otros.

Las diferencias entre BOM y DOM serían las siguientes:

- Con el DOM, JavaScript puede acceder a los elementos de un documento o página web mediante su estructura interna.
- Con el DOM, no se puede acceder a ciertos aspectos del navegador como puede ser la URL o las dimensiones de la ventana (navegador) ni tampoco se puede cerrar o redimensionar la ventana del navegador, gestionar cookies, etc. Para ello, se utiliza el BOM, que es otra estructura arborescente similar al DOM y algo más completa al añadirsele otra serie de objetos.
- A diferencia del DOM, el elemento raíz es el objeto window.

Por lo tanto, podrían ejecutarse las siguientes líneas de código:

```
h = window.document.getElementById("header");
```

O bien:

```
h = document.getElementById("header");
```

Y el resultado de ambas líneas sería el mismo.

## FUNDAMENTAL

### *El objeto document en el BOM y en el DOM*

Como se puede observar, en el BOM existe un objeto document al igual que en el DOM. En el DOM, el objeto document es la raíz del árbol de objetos de la página web, mientras que en el BOM se puede acceder a propiedades como:

- `document.URL`. Contiene la URL de la página web actual.
- `document.referrer`. Contiene la URL desde que se accedió a la página actual.
- `document.title`. Se accede al texto de la etiqueta `<title>` de la página web.
- `document.lastModified`. Contiene la fecha de última modificación de la página web.

Véase un ejemplo de script que muestra el ancho y alto del navegador. Pruebe a recargar la página y se dará cuenta que las dimensiones van cambiando.

```
<!DOCTYPE html>
<html>
<body>
<p id="dimension"></p>
<script>
var ancho = window.innerWidth;
var alto = window.innerHeight;
document.getElementById("dimension").innerHTML = "Ancho: " + ancho + "
--- Alto: " + alto ;
</script>
</body>
</html>
```

#### TOMA NOTA



El navegador Internet Explorer suele ser un poco diferente a otros navegadores como Firefox, Safari u Opera. Siempre que utilice objetos del BOM, compruebe que su página web funciona en Explorer y en los demás navegadores.

### 4.10.1. El objeto window

Como se ha visto, el objeto window representa la ventana del navegador en la que se muestra un documento del DOM. El objeto window controla, entre otras cuestiones, las dimensiones de la ventana, las barras laterales de desplazamiento, la barra de estado, etc.

Actualmente, como los navegadores tienen varias pestañas, cada una de ellas tendrá un objeto `window` asociado. No obstante, métodos como `window.resizeTo` o `window.resizeBy`, como es de entender, se aplicarán a la ventana de la aplicación y no a la pestaña específica.

WWW

## Recurso web

El objeto `window` tiene una infinidad de propiedades y métodos. Si se desea conocer el listado completo de propiedades y métodos, se puede acceder al siguiente recurso de MDN a través del código QR adjunto:



### 4.10.2. El objeto `location`

Un objeto muy útil del DOM es el objeto `window.location`. Véanse algunas de sus propiedades:

- `window.location.href`. Devuelve la URL de la página actual.
- `window.location.hostname`. Devuelve el dominio de la web.
- `window.location.pathname`. Devuelve el path y el fichero de la página actual.
- `window.location.protocol`. Devuelve el protocolo utilizado que debería ser o `http`: o `https`: (o `file`: si se está ejecutando una página desde el sistema de archivos local).
- Método `window.location.assign()` o `location.assign()`. Carga una nueva página web. Véase un ejemplo:

```
window.location.assign("http://www.sintesis.com");
```

El código anterior cargará en la misma página la web que se pasa como parámetro. En este caso, `http://www.sintesis.com`.

### 4.10.3. El objeto `history`

Es sabido que los navegadores tienen dos botones: uno que carga la web anterior (muy útil, por cierto) y otro que carga la siguiente web que se ha tecleado a la actual (la mayoría de las veces, el usuario está en la más reciente, salvo que vaya navegando hacia atrás).

Desde JavaScript, puede manejarse la historia del navegador con los métodos del objeto `history`:

- `history.back()`. Cargará la página web visitada anterior a la actual.
- `history.forward()`. Cargará la página web visitada siguiente a la actual.

### 4.10.4. El objeto `navigator`

El objeto `navigator` tiene las siguientes propiedades:

- `navigator.appName`.

- navigator.appCodeName.
- navigator.platform.

Las dos primeras contienen la aplicación que se está utilizando (por ejemplo, appName=Netscape y appCodeName=Mozilla) y en el tercero, la plataforma o sistema operativo utilizado (por ejemplo, Linux x86\_64).



#### SABÍAS QUE...

¿Es posible pedir información al usuario de forma interactiva? No es normal, pero sí sería posible. Observe el siguiente ejemplo:

```
<!DOCTYPE html>
<html>
<body>
<p id="cp"></p>
<script>
var codigopostal = prompt("Por favor introduzca su código
postal", "29680");
if (codigopostal != null) {
 document.getElementById("cp").innerHTML = "Su código
postal es: " + codigopostal ;
}
</script>
</body>
</html>
```

En el ejemplo anterior, se pide de forma interactiva un código postal y se muestra en un párrafo en la página web.



#### Actividad propuesta 4.5

Basándote en el código expuesto en este apartado, crea un programa que pida un código postal y una población actualizando los campos correspondientes de la página web.

### Resumen

- Para validar formularios en JavaScript, se pueden utilizar las sentencias try, throw y catch como en Java.
- Las expresiones regulares se utilizan ampliamente en la validación de campos por su potencia y efectividad.

- Las expresiones regulares son objetos que se pueden utilizar en los métodos `exec` y `test` del objeto `RegExp` y también en los métodos `search`, `replace` y `match` del objeto `string`.
- La sintaxis de una expresión regular es la siguiente:

`/patrón/modificadores`

- El DOM es una plataforma e interfaz de un documento que permite a los scripts y programas acceder y modificar dinámicamente su contenido, estructura y estilo.
- Cuando se carga un documento en el navegador, se crea el objeto `document`, el cual, además de ser la raíz de todo el documento HTML, tiene una serie muy amplia de propiedades y métodos.
- En el DOM, además de buscar elementos por el ID, se pueden buscar elementos por el tag [utilizando el método `getElementsByTagName()`] o también por la clase [utilizando el método `getElementsByClassName()`].
- El método `querySelector()` del objeto `document` devuelve el primer elemento que encuentre y que concuerde con el selector que se le introduzca como parámetro.
- Para seleccionar todos los elementos que concuerden con un selector, habrá que utilizar el método `querySelectorAll()`.
- Todos los eventos que ocurren a un objeto están basados en el objeto `event`. Este objeto contiene las propiedades y métodos que son comunes a todos los eventos.
- Los cuatro nodos más utilizados para manipular el DOM son `attr`, `document`, `element` y `text`.
- Las funciones para acceder a los nodos son:
  - `getElementById()`.
  - `getElementsByTagName()`.
  - `getElementsByName()`.
  - `getElementsByClassName()`.
  - `querySelector()`.
- Los pasos que hay que seguir al crear un nodo son:
  - Paso 1. Crear un nuevo nodo de tipo `Element`.
  - Paso 2. Crear un nuevo nodo de tipo `Text` (para albergar el contenido del elemento anterior).
  - Paso 3. Vincular el nodo `Text` al nodo `Element`.
  - Paso 4. Vincular el nodo `Element` a la página de tal manera que el nodo esté en el lugar correspondiente donde se quiera insertar el elemento.
- El BOM es una estructura arborescente similar al DOM y algo más completa al añadirse otra serie de objetos.
- A diferencia del DOM, en el BOM, el elemento raíz es el objeto `window`.
- Algunos objetos importantes en el BOM son `window`, `location`, `history` y `navigator`.

## Ejercicios propuestos



1. En el apartado 4.9.1, se muestra cómo modificar el tamaño de un párrafo. El problema es que solamente funciona para ese párrafo concreto. Añade un tercer parámetro a la función para que pueda modificarse el párrafo que se quiera de la web introduciendo su ID. Para comprobar si el script funciona correctamente, deberás crear más de un párrafo.
2. Crea una página web con dos campos de texto (nombre y apellidos) y un botón que valide que ambos contienen información.
3. Mejora el formulario con un campo de email que también sea validado por el botón anterior.
4. Desarrolla el formulario con un campo de teléfono que también sea validado por el botón anterior.
5. Realiza una página web con un script que verifique qué hace el código siguiente y explica su funcionalidad.

```
var str = "El sistema operativo más seguro es Android";
var res = str.replace(/android/i, "Linux");
```

6. Investiga qué hace el código siguiente y crea una página web para realizar la comprobación.

```
var str = "Aprende javaScript de 10 en Síntesis";
var m = str.search(/[S|t]/i);
console.log(m);
var n = str.search(/[0-9]/i);
console.log(n);
var a = str.search(/0+/i);
console.log(a);
```

7. Elabora un test interactivo para verificar si un alumno está sufriendo acosos escolar.
8. Lleva a cabo un script que, cuando pase el ratón sobre un campo de texto, lo cambie a mayúsculas.
9. Diseña una página web que capture el evento de redimensionar la ventana por parte del usuario.
10. Concibe una página web con un botón que capture los eventos click, mouseover y mouseout.

El método `removeEventListener` es capaz de eliminar un evento asociado a un elemento del documento. Añade un segundo botón que vaya eliminando uno a uno los eventos asociados.

## ACTIVIDADES DE AUTOEVALUACIÓN

1. ¿Qué función tiene el patrón `/\d/g`?

- ☐ a) Selecciona los dígitos de una cadena de caracteres.
- ☐ b) Selecciona los datos de una cadena de caracteres.
- ☐ c) Selecciona los no caracteres de una cadena de caracteres.

2. ¿Qué hace el patrón `/\S/g?`:

- ☐ a) Selecciona los no caracteres de una cadena de caracteres.
- ☐ b) Elimina los espacios de una cadena de caracteres.
- ☐ c) Busca si hay una secuencia de dos dígitos.

3. ¿Qué busca el patrón `/\d{2,3}/g?`:

- ☐ a) Busca si existe el 2 o el 3 en la cadena.
- ☐ b) Busca si existe el número 2 tres veces en la cadena.
- ☐ c) Busca si hay una secuencia de dos o tres dígitos.

4. Entre otros, el objeto `RegExp` tiene los métodos:

- ☐ a) `pattern()` y `evaluateRegularExpression()`.
- ☐ b) `exec()` y `test()`.
- ☐ c) `pattern()` y `testPattern()`.

5. Entre otros, el objeto `document` tiene los métodos:

- ☐ a) `openDocument()`, `querySelector()`, `getElementById()`, etc.
- ☐ b) `open()`, `querySelector()`, `getElementById()`, etc.
- ☐ c) `openDocument()`, `querySelector()`, `getElementById()`, etc.

6. ¿Qué propiedad se puede usar para saber si las cookies están habilitadas?:

- ☐ a) `window.cookieEnabled`.
- ☐ b) `document.cookieEnabled`.
- ☐ c) `navigator.cookieEnabled`.

7. ¿Cuál de las siguientes líneas de código es correcta?:

- ☐ a) `document.getElementById("unboton").addEventListener("MuestraFecha","click");`
- ☐ b) `document.getElementById("unboton").addEventListener("click","MuestraFecha");`
- ☐ c) `document.getElementById("unboton").addEventListener("click",MuestraFecha);`

8. ¿Cuáles son los nodos del DOM?:

- ☐ a) `Attr`, `Document`, `Element` y `Text`.
- ☐ b) `Attr`, `window`, `Element` y `Text`.
- ☐ c) `Attr`, `location`, `history` y `Text`.

9. Para acceder al selector de la clase CSS a la que pertenece un elemento, ¿cuál de las siguientes opciones se utiliza?:

- ☐ a) `getElementsByTagName()`.
- ☐ b) `getElementsByCSSName()`.
- ☐ c) `getElementsByClassName()`.

10. De las sentencias siguientes, ¿cuál se ejecuta para eliminar un nodo?:

- ☐ a) `elemento.parentNode.removeChild(elemento);`
- ☐ b) `elemento.removeNode(elemento);`
- ☐ c) `elemento.currentNode.removeDOMNode(elemento);`

#### SOLUCIONES:

1. ☒ a ☐ b ☐ c

2. ☐ a ☒ b ☐ c

3. ☐ a ☐ b ☒ c

4. ☐ a ☒ b ☐ c

5. ☐ a ☒ b ☐ c

6. ☐ a ☐ b ☒ c

7. ☐ a ☐ b ☒ c

8. ☒ a ☐ b ☐ c

9. ☐ a ☐ b ☒ c

10. ☒ a ☐ b ☐ c

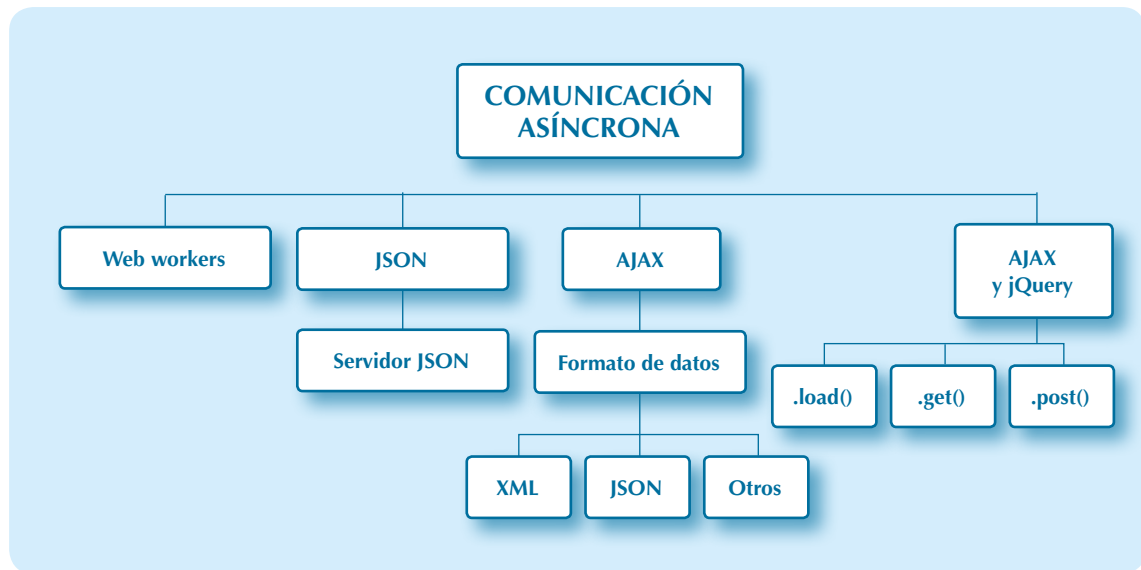


# Mecanismos de comunicación asíncrona

## Objetivos

- ✓ Entender el concepto de *web worker* y saber cómo se pueden realizar tareas en JavaScript de forma asíncrona.
- ✓ Comprender la sintaxis JSON y valorar el potencial de uso que puede tener para intercambiar datos entre la parte back-end y front-end.
- ✓ Conocer el modo de funcionamiento de AJAX y el tratamiento heterogéneo de datos que se puedan recibir (JSON, XML y otros tipos de datos).
- ✓ Utilizar jQuery y AJAX para realizar peticiones asíncronas a un servidor.

## Mapa conceptual



## Glosario

**Asíncrono.** Suceso o proceso que se produce de forma independiente al tiempo de procesamiento.

**CORS (Cross Origin Resource Sharing).** Mecanismo que, básicamente, consiste en acceder a información o a datos de otro dominio distinto al del origen de la petición.

**jQuery.** Biblioteca multipropósito que fue creada por John Resig y que ha sido mejorada en sucesivas versiones. Con esta librería, se puede manipular el DOM, crear animaciones, utilizar AJAX para realizar peticiones asíncronas al servidor, etcétera.

**JSON (JavaScript Object Notation).** Formato para el almacenamiento e intercambio de datos.

## 5.1. Introducción

La comunicación asíncrona es una de las cualidades que confiere a JavaScript su máxima potencia. Cuando AJAX surgió fue una revolución con respecto a las páginas web clásicas, puesto que permitía una comunicación y un refresco de información en una página web de forma que no había que recargarla de nuevo. Eso tiene todo su sentido, dado que ¿por qué tiene que refrescarse una página completa cuando solamente hay que cambiar el valor de un campo de esta?

Actualmente, no se concibe una aplicación web sin la utilización de este conjunto de técnicas. Hoy en día, el proceso en el lado del cliente ha ganado terreno frente a la lógica situada en el servidor, de tal manera que, en algunos frameworks, el back-end se utiliza solamente para persistencia de datos (guardar y recuperar datos de una base de datos).



CORS es el acrónimo de Cross Origin Resource Sharing y es un mecanismo por el cual una página web pide un permiso para poder acceder a recursos que se encuentran en otro servidor o dominio distinto al que pertenece el primero. Es muy común que una página web haga una petición XMLHttpRequest a otro servidor para cargar, por ejemplo, un JSON.

Los navegadores, por defecto, restringen este tipo de peticiones cruzadas por razones de seguridad. Tanto si el programador utiliza XMLHttpRequest como la API Fetch se va a encontrar con este problema, salvo que se utilicen encabezados CORS en las peticiones.

Si el lector quiere profundizar más en este concepto, puede acceder a las recomendaciones del W3C sobre CORS a través del siguiente código QR:



## 5.2. Los web workers

Un web worker es un proceso JavaScript que se ejecuta en segundo plano, de tal manera que la página web no se vea penalizada ni ralentizada por tener un proceso ejecutándose de forma permanente.

Se pueden utilizar web workers para analizar los patrones de navegación o para realizar llamadas asíncronas a un servicio de forma transparente al usuario.

Para ejecutar un web worker, el navegador tiene que soportar HTML5 y estar en una versión no excesivamente antigua.

A continuación, se muestra un ejemplo de web worker en el que se ha creado el web worker en un fichero aparte (es una buena práctica realizarlo así) llamado *trabajador.js*. El fichero HTML principal tendrá el siguiente aspecto:

```
<!DOCTYPE html>
<html>
<body>
<p>Secuencia de Fibonacci: <output id="result"></output></p>
<button onclick="startWorker()">Start Worker</button>
<button onclick="stopWorker()">Stop Worker</button>
<p>Note: Esto funciona en Firefox y Safari pero no en
Chrome.</p>
<script>
var w;
function startWorker() {
 if(typeof(Worker) !== "undefined") {
 if(typeof(w) == "undefined") {
 w = new Worker("trabajador.js");
 }
 w.onmessage = function(event) {
 document.getElementById("result").innerHTML = event.data;
```

```

 };
 } else {
 document.getElementById("result").innerHTML = "Fallo al ejecu-
 tar el web worker";
 }
}
function stopWorker() {
 w.terminate();
 w = undefined;
}
</script>
</body>
</html>

```

Como se puede observar, se ha incluido un botón para iniciar el web worker y otro botón para pararlo. Hay que tener en cuenta los siguientes aspectos del programa:

- El web worker se crea en la variable *w* siempre y cuando *w* tenga el tipo “undefined”.
- Cuando se crea el web worker, existe un modo de comunicación con la web host, que es el paso de mensajes. Se pueden capturar los mensajes mediante un listener “w.onmessage = function(event).....”.
- El web worker puede enviar mensajes a la página web host mediante la función `postMessage()`.

A continuación, se puede ver el contenido del web worker (fichero `trabajador.js`):

```

var anterior = 0;
var actual = 1;
var secuencia = "";
function temporizador() {
 if (!anterior){
 secuencia+=" "+actual;
 }else{
 secuencia+=" - "+actual;
 }
 postMessage(secuencia);
 aux = anterior + actual;
 anterior = actual;
 actual = aux;
 setTimeout("temporizador()",500);
}
temporizador();

```

### 5.3. JSON

JSON es el acrónimo de JavaScript Object Notation y es una sintaxis que se emplea para almacenar e intercambiar datos. Como se podrá verse a continuación, es una alternativa más fácil de usar a XML.

### 5.3.1. Sintaxis JSON

La sintaxis JSON en JavaScript tiene en cuenta cinco reglas:

1. Es un subconjunto de la sintaxis de JavaScript.
2. Los datos aparecen como pares nombre/valor.
3. Los datos se separan con comas.
4. Las llaves { } contienen objetos.
5. Los corchetes [ ] contienen arrays.

El siguiente ejemplo de JSON define el objeto empleados con un array que contiene tres registros del tipo empleado, el cual es un objeto:

```
{
 "empleados": [
 { "nombre": "Gerardo", "apellidos": "García" },
 { "nombre": "Maryna", "apellidos": "Hernández" },
 { "nombre": "Abel", "apellidos": "Pacheco" }
]
}
```

Entre las ventajas de JSON, cabe citar las siguientes:

- *Formato ligero de intercambio de datos.* Como se puede observar en el ejemplo anterior, se ha prescindido al máximo de elementos que sobrecarguen los datos de información innecesaria.
- *Independiente del lenguaje.* JSON utiliza la sintaxis JavaScript, pero el formato JSON es solo texto, al igual que XML. El texto puede leerse y utilizarse como formato de datos por cualquier lenguaje de programación. En el siguiente ejemplo, se podrá observar cuán fácil es incorporar datos en formato JSON a un código JavaScript.
- *Autodescriptivo y fácil de entender.*

A continuación, se muestra un ejemplo de utilización de datos en formato JSON:

```
<!DOCTYPE html>
<html>
<body>
<h2>Ejemplo de objetos JSON</h2>
<p id="demo"></p>
<script>
var text = '{"nombre":"IES MAR DE ALBORÁN","calle":"Fuente María Gil
S/N","teléfono":"951 33 56 45"}';
var obj = JSON.parse(text);
document.getElementById("demo").innerHTML =
obj.nombre + "
" +
obj.calle + "
" +
obj.teléfono;
</script>
```

```
</body>
</html>
```

Las diferencias entre JSON y XML son las siguientes:

- XML tiene que ser analizado con un analizador XML, mientras que JSON puede ser analizado por una función JavaScript estándar.
- JSON no utiliza etiqueta de fin.
- JSON es más corto.
- JSON es más rápido de leer y escribir.
- JSON puede utilizar matrices.
- JSON es más rápido y sencillo que XML.

Las similitudes entre ambos son:

- JSON y XML son autodescriptivos (legible por humanos).
- JSON y XML son jerárquicos (valores dentro de valores).
- JSON y XML pueden ser analizados y utilizados por muchos lenguajes de programación.
- Tanto JSON como XML se pueden obtener con una llamada XMLHttpRequest.

### 5.3.2. Práctica guiada: instalación de un servidor JSON

El objetivo de este ejercicio es mostrar cómo se puede instalar un servidor JSON en Node, desde el cual se puedan recuperar datos en este formato para luego ser utilizados en scripts (JavaScript).

Para este ejercicio, hay que tener Node instalado, en Ubuntu preferiblemente. En caso de no saber cómo instalar Node en su sistema operativo, se puede visitar el artículo siguiente: <https://nodejs.org/es/download/package-manager/>

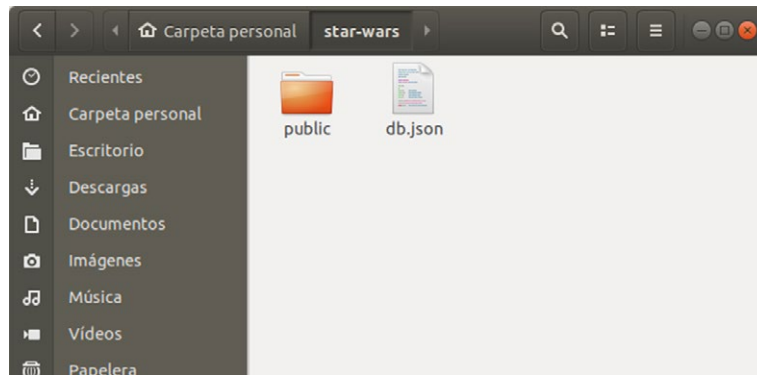
#### 1. Instalar el servidor JSON

El primer paso es instalar el módulo json-server en Node. Para instalar el servidor JSON, habrá que ejecutar el siguiente comando. De esa forma, se instalará de forma global el servidor.

```
sudo npm install json-server -g
```

#### 2. Configurar el servidor JSON

Una vez instalado el módulo, hay que configurar el servidor creando una carpeta con el nombre que se desee (*star-wars*, por ejemplo). En la carpeta, se puede crear un fichero JSON con los datos o se puede descargar el archivo (<http://myfpschool.com/wp-content/uploads/2018/11/starwars.zip>), que contiene una carpeta llamada *public* y un archivo de nombre *db.json* con los datos con los que se va a trabajar en formato JSON. Si se opta por la descarga, la carpeta tendrá el aspecto de la figura 5.1.



**Figura 5.1**  
Estructura de archivos y carpetas del servidor JSON.

### 3. Arrancar el servidor JSON

Es el momento de arrancar el servidor JSON utilizando el siguiente comando. Esto arrancará el servidor en modo lectura en el puerto 3005.

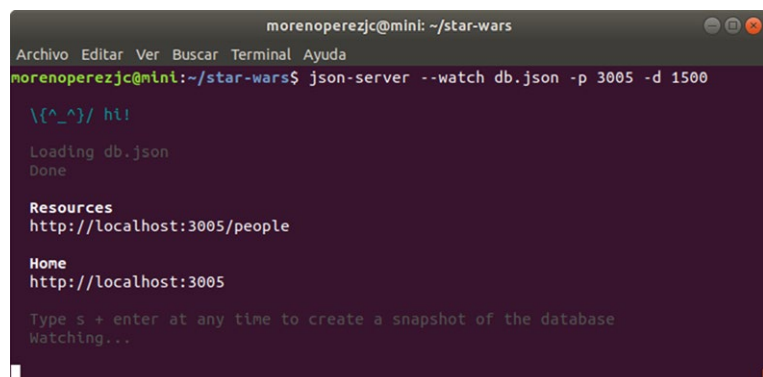
```
json-server -watch db.json -p 3005
```

#### RECUERDA

- ✓ Si vas a hacer prácticas en tu equipo, lo mejor es que establezcas un delay para que la respuesta a tu aplicación sea más o menos la de un servidor de internet. Para un delay de un segundo y medio (-d 1500), como se puede observar, el tiempo se especifica en milisegundos:

```
json-server -watch db.json -p 3005 -d 1500
```

En la figura 5.2, se muestra el aspecto del terminal al ejecutar el comando anterior con el delay.



**Figura 5.2**  
Arrancando el servidor JSON.

#### 4. Comprobar el funcionamiento del servidor JSON

Es el momento de comprobar si todo funciona. Para lo que hay que teclear dentro de un navegador la siguiente dirección:

```
http://localhost:3005/people
```

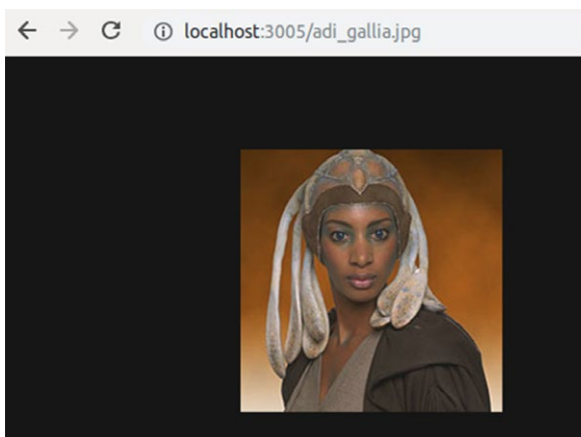
Si todo va bien, el resultado será el siguiente:

```
testeando el servidor json
```

Para comprobar si el servidor es capaz de mostrar imágenes, puede probarse a teclear la siguiente dirección:

```
http://localhost:3005/adi_gallia.jpg
```

Si el resultado es el que se muestra en la figura 5.3, el servidor estará funcionando correctamente.



**Figura 5.3**  
Comprobando el servidor  
JSON.

#### 5. Parar el servidor JSON

Para parar el servidor JSON, bastaría con teclear ctrl-C en la ventana del terminal.

### 5.4. Comunicación asíncrona. AJAX

AJAX es el acrónimo de Asynchronous JavaScript and XML. Con esta técnica, se pueden crear páginas dinámicas de forma rápida que consultan de una forma asíncrona fuentes de datos externas a la página.

Esta tecnología permite que una página se actualice sin tener que enviar información otra vez al servidor (y esperar a que el servidor la responda y envíe información de vuelta).

Como únicamente se intercambian datos, solo se envía lo que se necesita (no toda la página).

## TOMA NOTA



Es importante que, a partir de este capítulo, el lector se acostumbre a ejecutar las páginas web desde un servidor local o remoto. Utilizar un servidor local tipo XAMPP o LAMPP simplifica el desarrollo al no tener que subir los archivos al servidor remoto. Además, la ejecución será mucho más rápida.

Las páginas web antiguas había que recargarlas de forma completa (imagine un formulario con campos desplegables tipo país, provincia, población, etc., que se tienen que recargar cada vez que se hace una selección). El programador tenía dos opciones, o traerse mucha información durante la carga, lo que ralentizaba mucho la navegación, o realizar muchas recargas de la información. Ambas opciones eran poco operativas.

Cuando apareció la comunicación asíncrona, muchas aplicaciones comenzaron a utilizar AJAX como Google Maps, Gmail, YouTube, Facebook Tabs, etc.



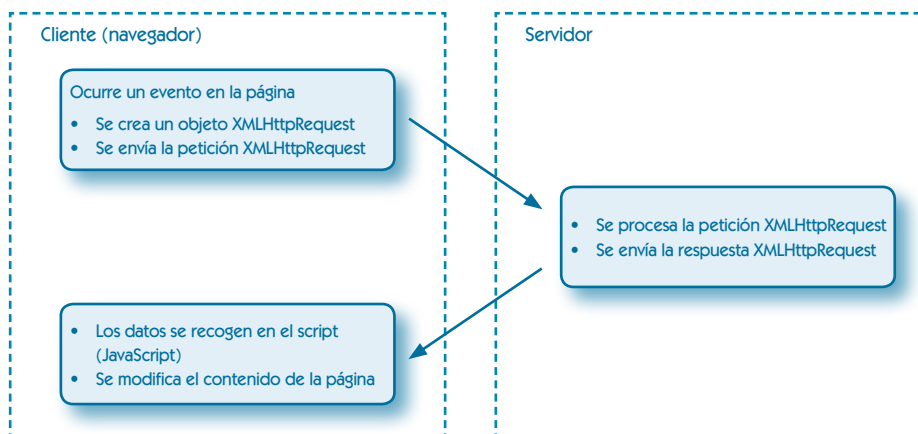
## SABÍAS QUE...

AJAX fue introducido en Google Suggest por primera vez. Cuando se escribía un texto en el campo de entrada de búsqueda de Google, un script te ofrecía una serie de sugerencias, con lo cual la gente se quedaba sorprendida. Eso no era posible realizarlo con HTML y las páginas normales, obviamente, no lo hacían.

AJAX se basa en el objeto XMLHttpRequest para intercambiar datos con el servidor de forma asíncrona. Existen otros métodos, como la librería o API Fetch, que han mejorado este tipo de acceso con primitivas más optimizadas. Algunos frameworks utilizan un objeto iframe en vez del objeto XMLHttpRequest para intercambiar datos.

Aunque AJAX aconseja XML para intercambio de datos, ya que muchas aplicaciones utilizan JSON por ser un formato más ligero y ofrecer ciertas ventajas frente al primero.

En la figura 5.4, se muestra el esquema de funcionamiento de una web que utilice AJAX.

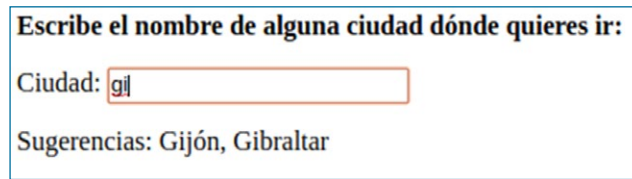
**Figura 5.4**

El mecanismo de comunicación asíncrona.

Como se puede observar, el procesamiento se produce una parte en el cliente y otra parte en el servidor.

### 5.4.1. Práctica guiada: sistema de localización de ciudades

Véase ahora un ejemplo de página web con un script en AJAX. En este ejemplo, se va a crear una pequeña página web en la que se pueda introducir el nombre de una ciudad y el sistema ofrezca sugerencias con ciudades que comiencen por las letras que se vayan introduciendo en un campo de texto. Básicamente, la página web tiene el aspecto que se muestra en la figura 5.5.



**Figura 5.5**  
Ejemplo de la página web de sugerencia de ciudades.

El sistema estará compuesto de dos partes: en una, se tiene el código HTML (index.html) con los script JavaScript integrados y la pertinente llamada XMLHttpRequest y, en otra, un fichero php llamado *getSugerencia.php*, el cual recibe la petición y envía de vuelta el nombre de las ciudades que ha encontrado en una lista creada previamente.

Fichero index.html:

```
<html>
<head>
<script>
function muestraSugerencia(str) {
 if (str.length == 0) {
 document.getElementById("txtSugerencia").innerHTML = "";
 return;
 } else {
 var xmlhttp = new XMLHttpRequest();
 xmlhttp.onreadystatechange = function() {
 if (this.readyState == 4 && this.status == 200) {
 document.getElementById("txtSugerencia").innerHTML = this.responseText;
 }
 };
 xmlhttp.open("GET", "http://localhost/Proyectos/JC/getSugerencia.php?param=" + str, true);
 // La dirección anterior corresponde a la ubicación de la página php en el servidor localhost
 // Esta dirección podrá diferir dependiendo de la necesidad del programador.
 }
}
```

```

 xmlhttp.send();
 }
}
</script>
</head>
<body>

<p>Escribe el nombre de alguna ciudad donde quieres ir:</p>
<form>
Ciudad: <input type="text" onkeyup="muestraSugerencia(this.value)">
</form>
<p>Sugerencias: </p>
</body>
</html>

```

El script anterior va a realizar las llamadas a:

- *var xmlhttp = new XMLHttpRequest()*. Para crear un objeto del tipo XMLHttpRequest.
- *xmlhttp.open()*. Para llamar al servicio o página web donde se van a recibir los datos.
- *xmlhttp.send()*. Para enviar la petición.

Como se va a hacer una petición asíncrona, el programador tiene que decir al sistema qué código tiene que ejecutarse cuando ocurra algo con la petición que ha realizado. Cada vez que ocurra algo en la petición, se ejecutará la función asociada al evento o método `onreadystatechange` del objeto XMLHttpRequest creado previamente:

- `xmlhttp.onreadystatechange = function() {....}`

Una vez que se esté dentro de esta función, se recoge el valor recibido y, en este caso, se le asignará a un elemento HTML creado para tal fin, pero ¿por qué se espera que se cumpla la condición “`this.readyState == 4 && this.status == 200`”?

La respuesta es porque cuando:

- `this.readyState == 4`: significa que la petición ha terminado (completada) y que la respuesta, por lo tanto, está lista.
- `this.status == 200`: significa que la petición se ha completado OK. Otros valores típicos de este atributo pueden ser el famoso 404 (Not found) o el 403 (Forbidden).

#### TOMA NOTA



Valores del atributo `readyState`:

- 0 (no inicializada).
- 1 (leyendo).
- 2 (leído).
- 3 (interactiva).
- 4 (completo).

Ahora quedaría la parte servidora, que procesaría la petición AJAX. En este caso, sería un script php colocado en el servidor de nombre getSugerencia.php:

```
<?php
// Array de nombres de ciudades
$arr[] = "Casares";
$arr[] = "Barcelona";
$arr[] = "Alcorcón";
$arr[] = "Málaga";
$arr[] = "Fuengirola";
$arr[] = "Marbella";
$arr[] = "Río de Janeiro";
$arr[] = "Gijón";
$arr[] = "Oviedo";
$arr[] = "Cáceres";
$arr[] = "Mérida";
$arr[] = "Estepona";
$arr[] = "París";
$arr[] = "Londres";
$arr[] = "Berlín";
$arr[] = "Moscú";
$arr[] = "Atenas";
$arr[] = "Kassel";
$arr[] = "Praga";
$arr[] = "Nueva York";
$arr[] = "Dublín";
$arr[] = "Manilva";
$arr[] = "Gibraltar";
$arr[] = "Ulan Bator";
$arr[] = "Bombay";
$arr[] = "Niza";
$arr[] = "Pekín";
$arr[] = "Tokio";
$arr[] = "Vasteras";
$arr[] = "Washington";
$arr[] = "México DC";
$arr[] = "Estocolmo";
$arr[] = "Stocholm";

// Primero recogemos el parámetro de la URL
$params = $_REQUEST["param"];

$sugerencia = "";

// Mira si coincide el parámetro con alguna de las ciudades
if ($params !== "") {
 $params = strtolower($params); //convierte el parámetro recibido a
 minúscula
```

```

$len=strlen($param); //determinamos la longitud del parámetro re-
cibido
foreach($arr as $nombre) {
// comparamos si el texto recibido coincide con el comienzo de al-
guna ciudad registrada en nuestro array
 if (strstr($param, substr($nombre, 0, $len))) {
 if ($sugerencia === "") {
 $sugerencia = $nombre; // primera sugerencia
 } else {
 $sugerencia .= ", $nombre"; //segunda y siguientes sugerencias
 }
 }
}
}
// En el caso de no encontrar ninguna sugerencia se le hace saber al usuario.
// en caso contrario devuelve las sugerencias encontradas
echo $sugerencia === "" ? "ninguna sugerencia" : $sugerencia;
?>

```

El script php es sencillo de comprender. En un bucle, se va recorriendo el array y seleccionando aquellas posiciones del array cuyas primeras letras coinciden con el texto que el usuario ha tecleado en la página web.

### 5.4.2. Ejemplo con AJAX y JSON

En este apartado, se explicará cómo se trata la información recibida en formato JSON por un script JavaScript. A continuación, se muestra un ejemplo sencillo en el que se tiene un archivo HTML llamado *json.html* y un archivo de texto *mywebs.txt* con información en formato JSON, más concretamente, un array de tres objetos en los que se almacena tanto el nombre como la dirección de tres páginas web.

A continuación, se muestra el contenido del fichero *json.html*:

```

<html>
<head>
</head>
<body>
<div id="lasWebs"></div>
<script>
 var xmlhttp = new XMLHttpRequest();
 var url = "mywebs.txt";
 xmlhttp.onreadystatechange = function() {
 if (this.readyState == 4 && this.status == 200) {
 var listaWebs = JSON.parse(this.responseText);
 getWebs(listaWebs);
 }
 };
 xmlhttp.open("GET", url, true);
 xmlhttp.send();

```

```
function getWebs(arr) {
 var out = "";
 var i;
 for(i = 0; i < arr.length; i++) {
 out += '' +
 arr[i].display + '
';
 }
 document.getElementById("lasWebs").innerHTML = out;
}
</script>
</body>
</html>
```

Como se puede observar, al recibir los datos, se utiliza el método parse del objeto JSON para interpretar la información obtenida:

```
var listaWebs = JSON.parse(this.responseText);
```

Esta llamada creará un array de objetos, el cual se envía como parámetro a la función getWebs(), que lo único que hace es recorrerlo y mostrar el contenido en el elemento <div> de la página web con el ID igual a "lasWebs".

A continuación, se muestra el contenido del fichero mywebs.txt:

```
[
 {
 "display": "Web de MyFPSchool",
 "url": "http://myfpschool.com"
 },
 {
 "display": "IES Mar de alborán",
 "url": "http://www.maralboran.es"
 },
 {
 "display": "SomeBooks",
 "url": "http://www.somebooks.es"
 }
]
```

En este fichero se puede observar lo explicado anteriormente. Es un array de tres posiciones en las que tiene tres objetos, cada uno con dos campos: display y url.

### Actividad propuesta 5.1



Verifica si el código expuesto en este apartado funciona correctamente en un navegador. Como característica especial, en vez de páginas web, utiliza razas de animales.

### 5.4.3. Ejemplo con AJAX y XML

Según el W3C, “el lenguaje XML (Extensible Markup Language) es un formato de texto simple y muy flexible derivado de SGML (ISO 8879). Se diseñó originalmente para la publicación electrónica a gran escala y en la actualidad desempeña un papel cada vez más importante en el intercambio de una amplia variedad de datos en la Web y en otros lugares”.

Primero se creará en el servidor un documento XML al que puedan realizarse consultas. El archivo al cual consultar sería el siguiente:

```
<concesionario>
 <coche combustible="D">
 <marca>Volkswagen</marca>
 <modelo>Passat</modelo>
 <color>Azul</color>
 <año>2005</año>
 <precio>7900.00</precio>
 </coche>
 <coche combustible="G">
 <marca>Volkswagen</marca>
 <modelo>Touran</modelo>
 <color>Azul</color>
 <año>2007</año>
 <precio>8200.00</precio>
 </coche>
 <coche combustible="D">
 <marca>Opel</marca>
 <modelo>Astra</modelo>
 <color>Negro</color>
 <año>2013</año>
 <precio>8490.00</precio>
 </coche>
</concesionario>
```

Este fichero se denominará *coches.xml*, se situará en el mismo directorio que el archivo HTML y será el fichero que servirá para realizar la consulta.

El siguiente paso es crear un archivo *coches.html* que contenga un script que lleve a cabo un filtrado sobre un determinado tipo de nodos del fichero *coches.xml*.

```
<!DOCTYPE html>
<html>
<body>

<p id="listado"></p>

<script>

var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
```

```

 if (this.readyState == 4 && this.status == 200) {
 recuperaDatos(this);
 }
 };
 xhttp.open("GET", "coches.xml", true);
 xhttp.send();

 function recuperaDatos(xml) {
 var datos = "";
 var xmlDatos = xml.responseXML;
 array = xmlDatos.getElementsByTagName("marca");
 for (i = 0; i < array.length; i++) {
 datos = datos + "marca: " + array[i].childNodes[0].nodeValue + "
";
 }
 document.getElementById("listado").innerHTML = datos;
 }

</script>
</body>
</html>

```

Como se puede observar, en las líneas de código:

```
var xmlDatos = xml.responseXML;
```

Se utiliza `responseXML` y no `responseText`, dado que el formato de los datos es XML.

Nótese que la variable `array` contiene los nodos que van a ser seleccionados, para lo cual se utiliza el método `getElementsByTagName` seleccionando solamente los nodos tipo “marca”. Al iterar sobre el `array`, se irá seleccionando el valor de los nodos “marca” del archivo XML.

### Actividad propuesta 5.2



Basándote en el código expuesto en este apartado, carga los datos de un fichero XML con la temática que desees.

## 5.5. Comunicación asíncrona con el servidor. AJAX y jQuery

Como ya se ha visto, AJAX es la forma que tiene JavaScript de comunicarse de forma asíncrona con un servidor. Utilizar jQuery es otra manera más de realizar peticiones por GET o POST a un servidor e incorporar el resultado a un campo de una página web.

### RECUERDA

- ✓ A veces, AJAX puede cambiar, dependiendo del navegador en el que se esté utilizando. En ocasiones, el programador tiene que comprobar el tipo de navegador antes de ejecutar un código u otro.

Puedes encontrar más información en la página web de jQuery, accediendo a través del siguiente código QR:



### 5.5.1. Carga simple de datos load()

La función `.load` de jQuery permite cargar datos directamente del servidor en un campo HTML que el programador decida. El formato de esta función es el siguiente:

```
jQuery.load(url [, datos] [, la_función])
```

Donde:

- *url* es un string que contiene la página web que se va a invocar.
- *datos* es un objeto o un string con la información que se quiere enviar al servidor.
- *la\_función* es una función que se desea invocar cuando la petición se complete.

Este es el método más sencillo de obtener datos de un servidor utilizando jQuery. Una vez que la petición se completa, la función `load()` cargará los datos recibidos en el campo que el programador decida. En el siguiente ejemplo, los datos se cargan en un campo con identificador `id=lista`:

```
$("#lista").load("datos.html");
```

Como se puede ver en la orden anterior, los datos se reciben de la página web `datos.html`.

A continuación, se muestra un ejemplo de llamada vía AJAX-jQuery para recuperar los datos de una página web:

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/jquery.
min.js"></script>
<script>
$(document).ready(function(){
 $("#btnDatos").click(function(){
 $("#lista").load("datos.html");
 });
});
</script>
</head>
<body>
<h3>Asignaturas</h3>
<ul id="lista">

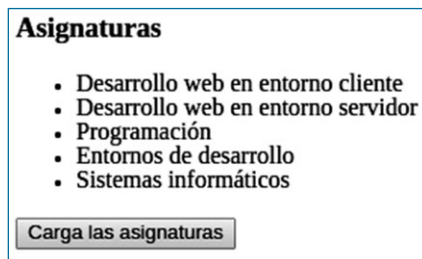
<button id="btnDatos">Carga las asignaturas</button>

</body>
</html>
```

El fichero datos.html contendrá el siguiente código HTML:

```
Desarrollo web en entorno cliente
Desarrollo web en entorno servidor
Programación
Entornos de desarrollo
Sistemas informáticos
```

El aspecto de esta página web tras pulsar el botón será el que se observa en la figura 5.6.



**Figura 5.6**  
Aspecto del ejemplo  
en el navegador.

### Actividad propuesta 5.3



Crea un programa basado en el código expuesto en este apartado y verifica que funciona correctamente en un navegador.

#### 5.5.2. Llamada asíncrona utilizando POST

A continuación se va a mostrar un ejemplo de llamada utilizando POST. Como todos los programadores saben, las llamadas por GET son similares. La función POST tiene el siguiente formato:

```
jQuery.post(url [, datos] [, la_función] [, dataType])
```

Donde:

- *url* es un string que contiene la página web que se va a invocar.
- *datos* es un objeto o un string con la información que se quiere enviar al servidor.
- *la\_función* es una función que se desea invocar cuando la petición se complete.
- *dataType* es el formato de datos que se espera recibir del servidor (XML, JSON, script, text o HTML).

En el ejemplo siguiente, la llamada es:

```
$.post("suma.php",
 $("#formulario").serialize(),
 function(data,status){
 $('#respSuma').html(data);
 });
```

Se puede comprobar en el código anterior que se hace una llamada a la página web suma.php, a la cual se le envían los datos del formulario. Se utiliza el método `serialize()` para crear un string en formato URL-encoded con los datos del formulario y que pueda recibirlos correctamente el script en el lado del servidor.

Véase el ejemplo de página web que realiza la suma de dos números:

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8" />
 <title> Suma dos números con AJAX y JQuery </title>
 <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/
jquery.min.js"></script>
 <script>
 $(document).ready(function(){
 $("#btnSuma").click(function(){
 $.post("suma.php",
 $("#formulario").serialize(),
 function(data,status){
 $('#respSuma').html(data);
 });
 });
 });
 </script>
</head>
<body>
 <form method="post" id="formulario">
 Introduzca dos valores para calcular la suma:

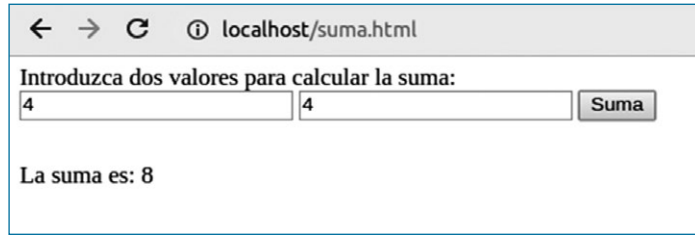
 <input type="number" name="valor1" placeholder="valor1"
autofocus/>
 <input type="number" name="valor2" placeholder="valor2"/>
 <input type="button" id="btnSuma" value="Suma" />
 </form>

 <div id="respSuma"></div>
</body>
</html>
```

La página php (suma.php), como se puede apreciar en el código siguiente, es sumamente simple. La única función que realiza es la suma de los dos parámetros recibidos por POST:

```
<?php
 $valor1 = $_POST['valor1'];
 $valor2 = $_POST['valor2'];
 $suma = $valor1+$valor2;
 echo "La suma es: ".$suma;
?>
```

En la figura 5.7, el aspecto de la página web una vez ejecutada en el navegador.



**Figura 5.7**

Aspecto del ejemplo anterior en un navegador.

#### RECUERDA

- ✓ Load(), post() y get() son funciones reducidas de la llamada jQuery ajax(). La llamada post() del ejercicio expuesto en este apartado se podría haber sustituido por el siguiente código:

```
$.ajax({
 type: «POST»,
 url: url,
 data: $("#formulario").serialize(),
 success: function(data)
 {
 $('#respSuma').html(data);
 }
});
```

Como se puede observar, la llamada post() utilizada en el ejercicio es más intuitiva y genera menos líneas de código.

#### Actividad propuesta 5.4



¿Serías capaz de conseguir que el ejercicio expuesto en este apartado pudiese realizar la suma o la resta?

## 5.6. Práctica guiada: encuestas interactivas con AJAX

En este ejemplo, se va a crear una encuesta interactiva utilizando AJAX. Este tipo de encuestas es muy común en muchas páginas web. Esta encuesta hará una pregunta al usuario y mostrará el resultado total de todas las preguntas realizadas hasta el momento.

La página web constará de tres partes:

- Un fichero de texto (*resultados.txt*). Donde se almacenan los resultados.
- Una página html (*encuesta.html*). Donde se genera la encuesta.
- Una página php (*encuesta\_voto.php*). Recoge el voto del usuario y calcula los tantos por ciento teniendo en cuenta los datos anteriores, además de publicar el resultado.

#### 1. El fichero de texto (*resultados.txt*)

Es donde se van a almacenar los resultados de la encuesta. Se va a llamar *resultados.txt* y se va a iniciar con la siguiente línea de texto:

```
0||0||0||0
```

Cada uno de esos campos separados por || serán los votos que obtendrá cada una de las opciones. Conforme los usuarios vayan votando, el contenido del fichero irá cambiando:

```
19||3||6||0
```

#### 2. La página web (*encuesta.html*)

La página web para la encuesta es sumamente sencilla. Consta de cuatro input de tipo radio que, al ser seleccionados, ejecutarán la función `getvoto()`. Esta función realiza una llamada AJAX a la página web php *encuesta\_voto.php* pasándole como parámetro por GET el valor del voto emitido por el usuario.

```
<!DOCTYPE html>
<html>
<head>
<script>
function getvoto(int) {
 xmlhttp=new XMLHttpRequest();
 xmlhttp.onreadystatechange=function() {
 if (this.readyState==4 && this.status==200) {
 document.getElementById("encuesta").innerHTML=this.responseText;
 }
 }
 xmlhttp.open("GET","encuesta_voto.php?voto="+int,true);
 xmlhttp.send();
}
</script>
</head>
<body>
<div id="encuesta">
<h3>¿Qué equipo crees que va a ganar la liga este añ
de;o?</h3>
<form>
```

```

Real Madrid:
<input type="radio" name="voto" value="0" onclick="getvoto(this.
value)">

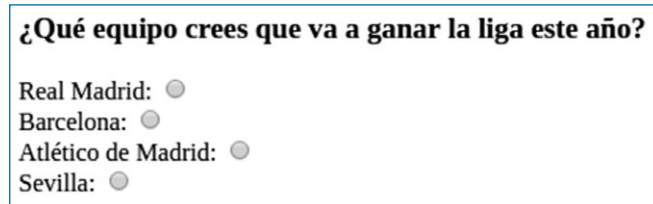
Barcelona:
<input type="radio" name="voto" value="1" onclick="getvoto(this.
value)">

Atlético de Madrid:
<input type="radio" name="voto" value="2" onclick="getvoto(this.
value)">

Sevilla:
<input type="radio" name="voto" value="3" onclick="getvoto(this.
value)">
</form>
</div>
</body>
</html>

```

El aspecto de la página web anterior será el que se muestra en la figura 5.8.



**Figura 5.8**

Aspecto de la página web anterior.

### 3. La página php (*encuesta\_voto.php*)

En esta página php, se reciben los votos, se calculan los resultados y se le presentan al usuario. El código está comentado para una mejor comprensión.

```

<?php
//Se abre el fichero deonde están almacenados los datos
$fichero = "resultados.txt";
$contenido = file($fichero);
//colocamos el contenido en un array y lo almacenamos en cuatro va-
riables por equipos
$array = explode("||", $contenido[0]);
$real = $array[0];
$bar = $array[1];
$atl = $array[2];
$sev = $array[3];

```

```

//extraemos el voto de los participantes
$voto = $_GET['voto'];

//actualizamos los votos añadiendo el recibido a los anteriores
if ($voto == 0) {
 $real = $real + 1;
}
if ($voto == 1) {
 $bar = $bar + 1;
}
if ($voto == 2) {
 $atl = $atl + 1;
}
if ($voto == 3) {
 $sev = $sev + 1;
}
//creamos la cadena que se va a insertar en el fichero
$insertvoto = $real."||".$bar."||".$atl."||".$sev;
//se abre el fichero como escritura y se escriben los votos actualizados
$fp = fopen($fichero,"w");
fputs($fp,$insertvoto);
fclose($fp);

// se calcula el % del voto de cada uno de los equipos
$denominador=(int)$real+(int)$bar+(int)$atl+(int)$sev;
$tantoMadrid=100*round($real/$denominador,2);
$tantoBarcelona=100*round($bar/$denominador,2);
$tantoAtletico=100*round($atl/$denominador,2);
$tantoSevilla=100*round($sev/$denominador,2);
?>
<h2>Resultado:</h2>
<table>
 <tr>
 <td>Real Madrid:</td>
 <td>
 ' height='20'> <?php echo($tantoMadrid); ?>%
 </td>
 </tr>
 <tr>
 <td>Barcelona:</td>
 <td>
 ' height='20'> <?php echo($tantoBarcelona); ?>%
 </td>
 </tr>
 <tr>
 <td>Atlético de Madrid:</td>

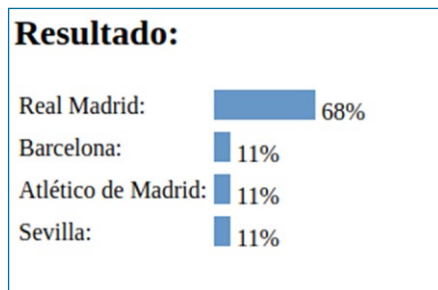
```

```

 <td>
 ' hei-
 ght='20'> <?php echo($tantoAtletico); ?>%
 </td>
 </tr>
 <tr>
 <td>Sevilla:</td>
 <td>
 ' hei-
 ght='20'> <?php echo($tantoSevilla); ?>%
 </td>
 </tr>
</table>

```

En la figura 5.9, se observa el aspecto del resultado una vez emitido el voto por parte del usuario.



**Figura 5.9**

Aspecto de la página web una vez emitido el voto.

Para mostrar el resultado con una barra, se ha utilizado una imagen de 30 píxeles de ancho y 200 píxeles de alto de color azul. Esa imagen tendrá la anchura del tanto por ciento de votos obtenido por cada equipo. De esta manera, el resultado quedará más vistoso.

### Actividad propuesta 5.5



Añade un equipo a la encuesta y verifica que su programa funciona correctamente en un navegador.

### Resumen

- Los web worker aparecen en HTML5.
- Para crear un web worker, se utiliza la sentencia `w = new Worker("trabajador.js");` donde *trabajador.js* es el fichero JavaScript que ejecutará el web worker.

- JSON es el acrónimo de JavaScript Object Notation, que es una sintaxis utilizada para almacenar e intercambiar datos.
- La sintaxis JSON en JavaScript sigue cinco reglas:
  1. Es un subconjunto de la sintaxis de JavaScript.
  2. Los datos aparecen como pares nombre/valor.
  3. Los datos se separan con comas.
  4. Las llaves { } contienen objetos.
  5. Los corchetes [ ] contienen arrays.
- JSON y XML son parecidos, pero JSON es más útil a la hora de programar porque es más corto, más rápido y más sencillo.
- Instalar un servidor JSON permite recuperar de una manera sencilla o incluso insertar datos desde una aplicación web.
- AJAX es el acrónimo de Asynchronous JavaScript and XML. Con esta técnica, se pueden crear páginas dinámicas de forma rápida que consultan de una forma asíncrona fuentes de datos externas a la página.
- AJAX se basa en el objeto XMLHttpRequest para intercambiar datos con el servidor de forma asíncrona.
- Los pasos de una petición AJAX son los siguientes:
  1. `var xmlhttp = new XMLHttpRequest();` Para crear un objeto del tipo XMLHttpRequest.
  2. `xmlhttp.open();` Para llamar al servicio o página web donde se van a recibir los datos.
  3. `xmlhttp.send();` Para enviar la petición.
  4. `xmlhttp.onreadystatechange = function() {...};`
- AJAX se suele utilizar con datos en formato JSON y XML.
- AJAX se integra también en jQuery con las funciones `load()`, `get()` y `post()`.

## Ejercicios propuestos



1. Modifica la página web `suma.html` en la que se sumaban dos números utilizando AJAX y jQuery con la función `post()`. Cambia la función `post()` por la función más genérica `ajax()`, aunque el código sea algo más extenso.
2. Repite el ejercicio anterior, pero utiliza GET en vez de POST.
3. Transforma el siguiente código XML en formato JSON:

```
<departamento>
 <trabajador>
 <nombre>Pedro</nombre> <apellidos>Ruíz Aranda</apellidos>
 </trabajador>
</trabajador>
```

```

 <nombre>Isaac</nombre> <apellidos>Sánchez Alcalde</ape-
 llidos>
 </trabajador>
 <trabajador>
 <nombre>Lars</nombre> <apellidos>Arnstron</apellidos>
 </trabajador>
</departamento>

```

4. Crea un webworker que vaya mostrando en la página web los números pares de forma incremental cada segundo.
5. Elabora una ventana parecida a la siguiente que valide mediante AJAX los datos de usuario y password introducidos. Si se introduce el usuario admin y la password 1234, el sistema dirá USUARIO VÁLIDO y, en caso contrario, responderá USUARIO NO VÁLIDO.

Introduzca usuario y password:

usuario

password

Log In

**Figura 5.10**  
Ejemplo de aplicación.

Introduzca usuario y password:

admin

\*\*\*\*

Log In

USUARIO VALIDO

**Figura 5.11**  
Acceso inválido.

6. Partiendo del ejercicio resuelto de la encuesta del apartado 5.6, prueba que funcionan los script anteriores, pero cambia la pregunta para que tenga 6 equipos.
7. ¿Qué es la API Fetch? Indica si puede ser una alternativa a las formas de comunicación asíncrona vistas hasta el momento.
8. Diseña una página web que calcule la cuota mensual de un préstamo. Para ello, tendrás que solicitar el capital, el interés y el número de meses necesarios para devolver dicho préstamo. El resultado tendrá que ser parecido a la figura 5.13.

Cálculo de la cuota de un préstamo:

capital solicitado

interés anual

Plazo (meses)

Calcular

Su cuota mensual será de: Euros.

**Figura 5.12**  
Aspecto de la página web del ejercicio.

Cálculo de la cuota de un préstamo:

100

5

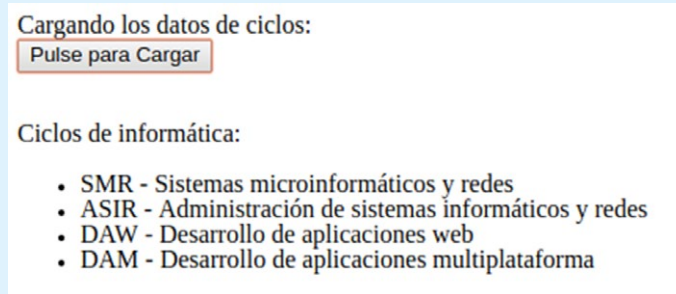
10

Calcular

Su cuota mensual será de: 10.230595941059 Euros.

**Figura 5.13**  
Aspecto de la página web del ejercicio tras realizar un cálculo.

9. Investiga si existe alguna función que permita cargar en un campo datos codificados en formato JSON utilizando la petición GET HTTP.
10. Teniendo los siguientes datos en formato JSON fichero ciclos.json, cárgalos en una lista en la página web.

**Figura 5.14**

Aspecto de la página web pretendida de carga.

## ACTIVIDADES DE AUTOEVALUACIÓN

1. ¿Cómo se denomina un proceso JavaScript que se ejecuta en segundo plano?:
  - ☐ a) Proceso sincrónico.
  - ☐ b) Proceso event-driven.
  - ☐ c) Web worker.
2. Los web workers funcionan a partir de:
  - ☐ a) HTML5.
  - ☐ b) JavaScript ES6.
  - ☐ c) EmacsScript
3. ¿Cuál de las siguientes opciones hay que cumplir para crear un web worker?:
  - ☐ a) Tener una función startWorker().
  - ☐ b) Crear un objeto del tipo Worker.
  - ☐ c) Crear un objeto del tipo webWorker.
4. En JSON, ¿qué contienen las llaves {}?:
  - ☐ a) Objetos.
  - ☐ b) Arrays.
  - ☐ c) Pares nombre/valor.

5. ¿Qué sintaxis utiliza JSON?:
- ☐ a) JavaScript.
  - ☐ b) XML.
  - ☐ c) Ni JavaScript ni XML.
6. ¿Cuál es el significado del acrónimo AJAX?:
- ☐ a) Asynchronous JSON and XML.
  - ☐ b) Asynchronous JavaScript and XML.
  - ☐ c) Asynchronous JavaScript and XML.
7. ¿Cuál de las siguientes empresas usó por primera vez AJAX?:
- ☐ a) Facebook.
  - ☐ b) Twitter.
  - ☐ c) Google.
8. Cuando se hace una XMLHttpRequest(), los datos estarán disponibles si:
- ☐ a) this.readyState == 4 || this.status == 200.
  - ☐ b) this.readyState == 4 && this.status == 200.
  - ☐ c) this.status == 4 && this.readyState == 200.
9. ¿Qué método se utiliza para interpretar una información en formato JSON?:
- ☐ a) JSON.parseJSON().
  - ☐ b) JSON.parse().
  - ☐ c) JSON.getData().
10. Cuando se procesa una petición de datos XML, ¿dónde se almacena el contenido?:
- ☐ a) XMLHttpRequest.responseXML.
  - ☐ b) XMLHttpRequest.XMLdata.
  - ☐ c) XMLHttpRequest.getXML().

### SOLUCIONES:

1. ☐ a ☐ b ☒ c

2. ☒ a ☐ b ☐ c

3. ☐ a ☒ b ☐ c

4. ☒ a ☐ b ☐ c

5. ☒ a ☐ b ☐ c

6. ☐ a ☐ b ☒ c

7. ☐ a ☐ b ☒ c

8. ☐ a ☒ b ☐ c

9. ☐ a ☒ b ☐ c

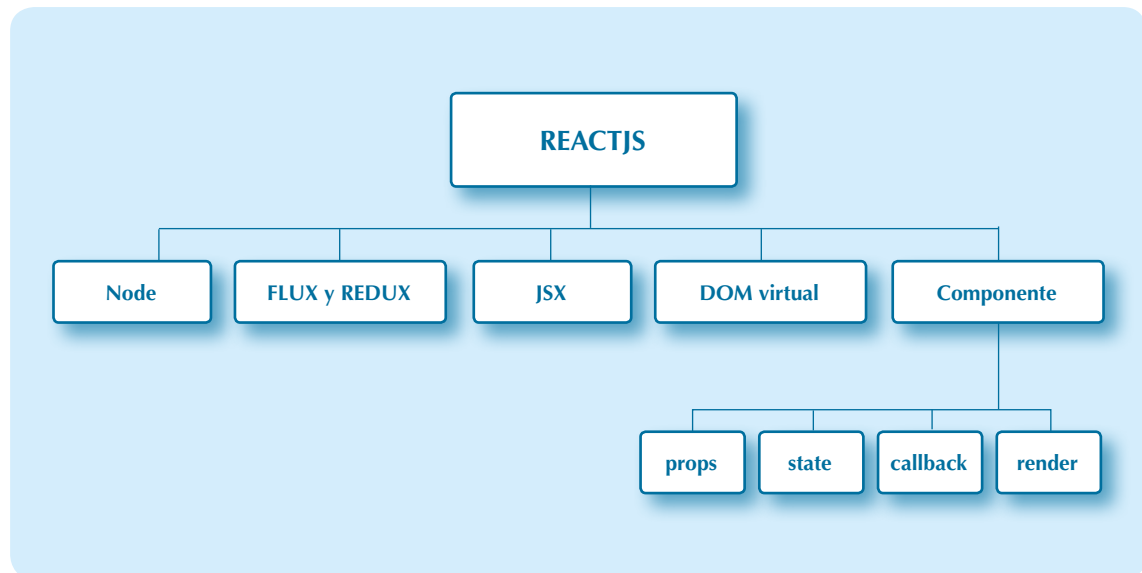
10. ☒ a ☐ b ☐ c

# ReactJS: un framework de JavaScript

## Objetivos

- ✓ Aprender la programación orientada a componentes.
- ✓ Comprender el concepto de *componente* con su state y sus props.
- ✓ Trabajar otro lenguaje de programación como es JSX.
- ✓ Conocer el concepto de *callback* o *feedback* de los componentes hijos hacia los padres.
- ✓ Utilizar y conocer el concepto de *DOM virtual*.
- ✓ Entender el framework o librería ReactJS, que es actualmente el más utilizado en la programación front-end.
- ✓ Crear una aplicación sencilla con ReactJS en la que poder tener una visión lo más amplia de la utilización de este framework.

## Mapa conceptual



## Glosario

**Callback.** Llamada que hace un componente hijo a un método de un componente padre. Es la manera que tiene un componente hijo de comunicarse con su componente padre.

**Componente.** La programación orientada a componentes es un tipo de programación en la que los sistemas se descomponen en componentes funcionales o presentacionales (relativos a la interfaz). El nivel de abstracción es más alto que el de los objetos.

**Elemento.** Pequeña porción de código en una aplicación React.

**JSX.** Extensión de JavaScript utilizada para programar componentes React.

**Props.** Sistema de comunicación entre componentes que utiliza React. Los componentes padre envían información a los componentes hijo a través de las props.

**Renderizar.** Proceso por el cual el componente se dibuja en el interfaz (navegador).

**State.** Estado de un componente donde se almacena la información relativa a este. Al igual que los objetos tienen atributos, un componente tiene estado.

**Virtual DOM.** DOM virtual utilizado por React. Al utilizar un DOM virtual los cambios en la interfaz son más ágiles y rápidos.

## 6.1. Introducción a React

ReactJS es un framework que permite a los programadores realizar aplicaciones web utilizando programación declarativa. Las aplicaciones React están formadas por componentes y React es capaz de renderizar dichos componentes de una manera sencilla, rápida y eficiente siempre que alguno de ellos sufra una modificación.

Como se ha explicado anteriormente, React utiliza programación orientada a componentes (que no objetos). Los componentes gestionan su propio estado y, agrupando una serie de componentes, los programadores son capaces de crear una interfaz de usuario.

Al ser los componentes elementos autónomos, su reutilización es muy alta y sencilla.

### FUNDAMENTAL

*Node.js*

Para trabajar con React, es preciso instalar en el equipo Node.js. La instalación de Node es sumamente sencilla. En Ubuntu, bastaría con ejecutar los siguientes comandos:

1. `$ sudo apt install nodejs`
2. `$ sudo apt install npm`

El primero para instalar Node y el segundo para instalar el gestor de paquetes de node (npm - node package manager).

ReactJS utiliza una sintaxis parecida al XML llamada *JSX*. Lo normal en ReactJS es programar en JSX, pero no es absolutamente necesario.

A lo largo de este capítulo, se aprenderá a utilizar ReactJS creando un sudoku interactivo, para lo cual va a utilizarse el código del ejercicio resuelto del sudoku del capítulo 3 de este libro.

### Recursos web

*www*

En el primer código QR, encontrarás un enlace de Wikipedia con más información sobre React. A través del segundo código QR, podrás acceder a un enlace de Netguru para obtener más información sobre programación declarativa e imperativa.

1.



2.



React fue diseñado por Jordan Walke y su primera versión se utilizó en las newsfeed de Facebook en el 2011. Se hizo open-source en la conferencia JSConf en los Estados Unidos en mayo del 2013. Este framework se diseñó para que fuese rápido, simple y escalable.

Por lo tanto, ReactJS es:

- Una *librería JavaScript* para crear interfaces de usuario.
- Utiliza *programación declarativa* y no imperativa como puede ser Java, por ejemplo.
- Utiliza *programación component-based* (basada en componentes).
- *Technology stack agnostic*. Esto quiere decir que interactúa bien con cualquier arquitectura. No obliga a utilizar una arquitectura determinada.

### 6.1.1. Creación de la primera aplicación React

En este apartado, se va a crear la primera aplicación React. En este caso, se va a emplear la utilidad `create-react-app`, que crea el esqueleto de una aplicación React. Este esqueleto podrá ser utilizado para modificarlo y dar forma a la aplicación que se quiera desarrollar.

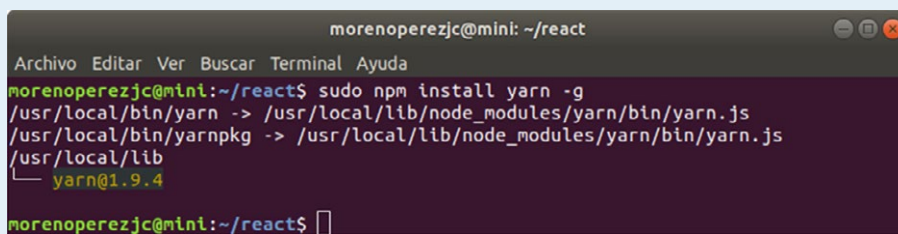
#### 🎯 FUNDAMENTAL

*También se puede instalar YARN*

Yarn es una herramienta para gestionar paquetes de aplicaciones Node.js. Es fiable, rápido y seguro y, por lo tanto, es muy utilizado en Node.js. Se puede instalar en Ubuntu de una forma sencilla y rápida.

Existen varios métodos para instalar Yarn, pero uno muy directo sería instalarlo utilizando NPM, para ello basta con ejecutar el siguiente comando (para instalar Yarn de forma global y no solamente para el proyecto actual, hay que utilizar el parámetro `-g`):

```
sudo npm install yarn -g
```



```
morenoperezjc@mini: ~/react
Archivo Editar Ver Buscar Terminal Ayuda
morenoperezjc@mini:~/react$ sudo npm install yarn -g
/usr/local/bin/yarn -> /usr/local/lib/node_modules/yarn/bin/yarn.js
/usr/local/bin/yarnpkg -> /usr/local/lib/node_modules/yarn/bin/yarn.js
/usr/local/lib
└─ yarn@1.9.4
morenoperezjc@mini:~/react$
```

**Figura 6.1**  
Instalación de Yarn.

Una vez ejecutado este comando, Yarn estará instalado correctamente.

El primer paso es instalar la aplicación `create-react-app` mediante NPM (Node Package Manager).

En Ubuntu, se puede instalar mediante los siguientes comandos:

```
sudo npm install -g create-react-app
```

O

```
sudo npm install -g create-react-app@1.5.2
```

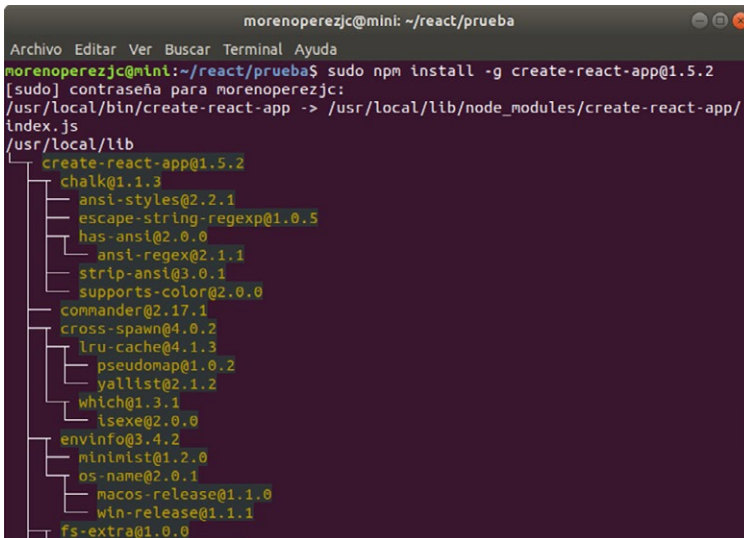
El segundo comando sirve para instalar una versión concreta de create-react-app.

Para crear la primera aplicación, bastará con ejecutar create-react-app. En Ubuntu, se ejecutaría con el siguiente comando:

```
create-react-app conversor
```

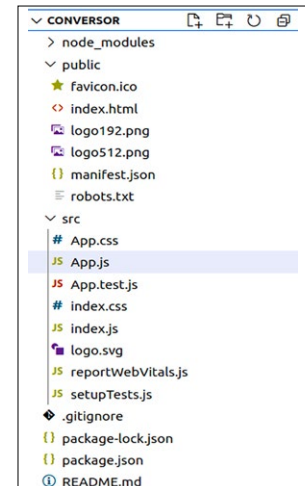
Donde conversor es el nombre de la aplicación que ha de crearse.

El comando anterior creará un esqueleto de una aplicación básica en React con el contenido que se indica en la figura 6.3.



**Figura 6.2**

Instalando create-react-app.



**Figura 6.3**

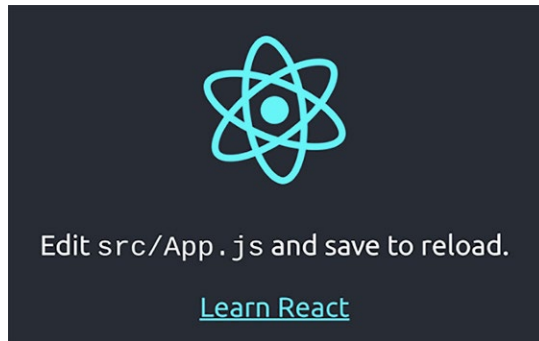
Contenido de la primera aplicación ReactJS. Visual Studio Code.

## 6.1.2. Ejecución de la aplicación React

El siguiente paso es ejecutar la aplicación. En Ubuntu, para ejecutar una aplicación, basta con estar en el directorio donde se ubique esta y ejecutar el comando:

```
npm start o yarn start
```

Una vez ejecutado el comando, se abrirá el navegador por defecto y se ejecutará la aplicación.



**Figura 6.4**  
Aspecto de la primera aplicación en React.

## 6.2. Análisis del contenido de la primera aplicación

En este apartado, se analizará el contenido (ficheros) de la primera aplicación con React creada en el apartado 6.1.1. Los archivos más importantes de la aplicación son los siguientes:

- El archivo HTML *public/index.html*.
- El archivo de script *src/App.js*.
- El fichero de script *src/index.js*.

### RECUERDA

- ✓ Los componentes React comienzan con letra mayúscula y el resto del nombre estará escrito en minúsculas.  
Las etiquetas HTML deberán estar en minúsculas en el código JSX.

### 6.2.1. El fichero *index.html*

La siguiente línea es la más interesante de este fichero:

```
<div id="root"></div>
```

Como se puede observar, hay un elemento de tipo `<div>` con `id="root"`. Justo en este elemento, se renderizará la aplicación React.

### 6.2.2. El fichero *index.js*

En este fichero se importan los distintos elementos de los que consta la aplicación y que serán necesarios para la misma. Crea el enlace entre la aplicación y el elemento root definido en *index.html* mediante la siguiente línea:

```
ReactDOM.render(
 <React.StrictMode>
 <App />
```

```

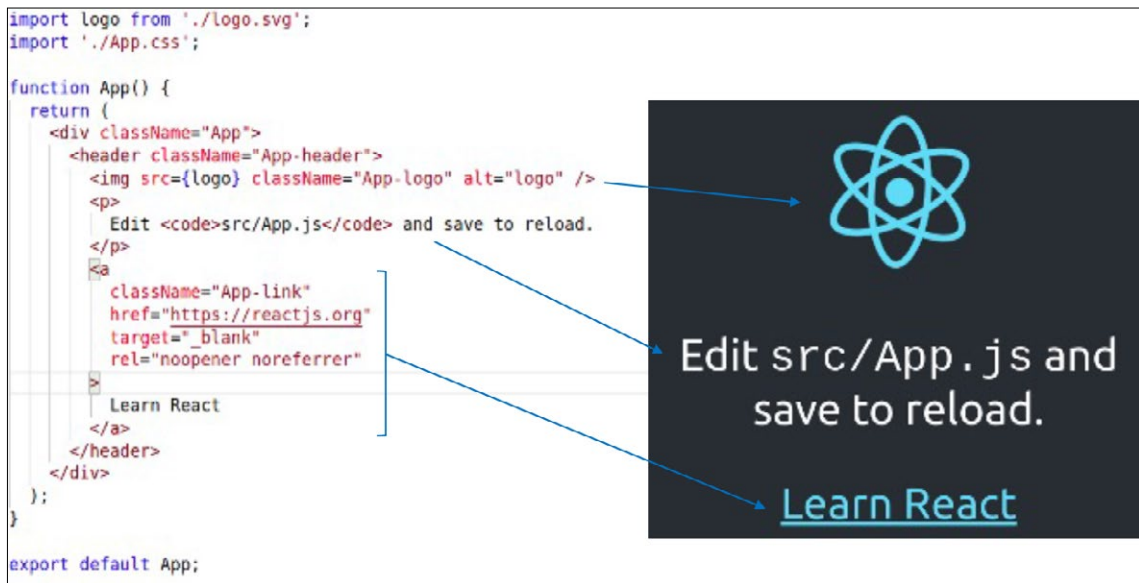
</React.StrictMode>,
document.getElementById('root')
);

```

Este orden renderiza la aplicación en el elemento de index.html con el id="root".

### 6.2.3. El fichero src/App.js

Este archivo es donde residirá la lógica de la aplicación. Interpretando el contenido se puede adivinar lo que se visualizará en el navegador.



**Figura 6.5**

Aspecto de la primera aplicación en React.

Como se puede observar, en la sentencia return del componente App se muestra el componente en la interfaz (navegador). El código dentro de la sentencia return es básicamente código HTML.



**SABÍAS QUE...**

El método render() en un componente React es el encargado de mostrar el componente en la interfaz web.

Por último, para que este componente se pueda referenciar desde otros ficheros externos hay que exportarlo mediante la siguiente sentencia:

```
export default App;
```



## TOMA NOTA

Las aplicaciones React están creadas en base a pequeñas porciones de código. Por ejemplo:

```

```

En estos elementos se utiliza `className` en vez de `class` para no confundirse con las clases. Como se puede observar, para que un elemento sea eficiente, los objetos javascript que se utilicen tienen que ser sencillos y fáciles de crear.

Recuerde que los *components* o componentes están generados en base a *elements* o elementos.

### 6.3. Introducción a JSX

JSX es una extensión de JavaScript. Aunque JSX parece un lenguaje de plantillas, se puede utilizar JavaScript sin problemas para darle una mayor potencia.

JSX genera elementos React (muchos elementos HTML tienen una correspondencia con un elemento React).

Una de las ventajas que tiene JSX es que está íntimamente ligado con el manejo de eventos, con cómo el estado de los componentes cambia durante su vida y cómo se formatean los datos para ser mostrados en la interfaz.

Los componentes de React contienen la lógica y la parte de interfaz en el mismo componente.



## PARA SABER MÁS

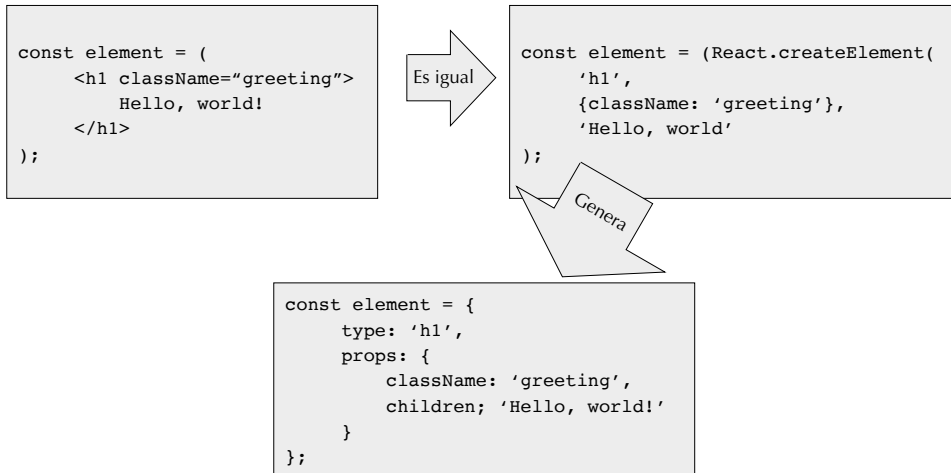
#### TypeScript y JSX

JSX es diferente a TypeScript. TypeScript, es un superconjunto de JavaScript que se utiliza para desarrollar aplicaciones que luego se compilan a JavaScript. Es posible añadir TypeScript a tu proyecto añadiendo las librerías correspondientes a tu aplicación React.

Es importante tener en cuenta que React no necesita utilizar JSX, pero los desarrolladores lo utilizan porque encuentran sumamente útil tener el HTML dentro del código JavaScript, puesto que está íntimamente ligado. Además, permite a React ser más útil a la hora de depurar y mostrar los mensajes de advertencia o error.

En la figura 6.4, se puede observar que los dos códigos de los elementos de la parte superior son similares para React. El código de la parte superior izquierda es JSX, mientras que el de la parte superior derecha es código JavaScript haciendo una llamada a un método de React.

Ambos códigos generarán el elemento que se describe en la parte baja de la figura.



**Figura 6.5**  
JavaScript frente a ES6.

**Recurso web**

Para más información sobre JSX, puedes consultar la página web de React a través del siguiente código QR:



## 6.4. Componentes React

Cuando se desarrolla una aplicación React, hay que tener en cuenta lo siguiente sobre los componentes:

- Las interfaces de usuario se forman por componentes que son independientes unos de otros.
- Los componentes son piezas reutilizables, como las piezas de LEGO.
- Un componente devolverá una serie de elementos que aparecerán en la ventana del navegador.
- En React, se pueden definir diferentes clases o tipos de componentes.



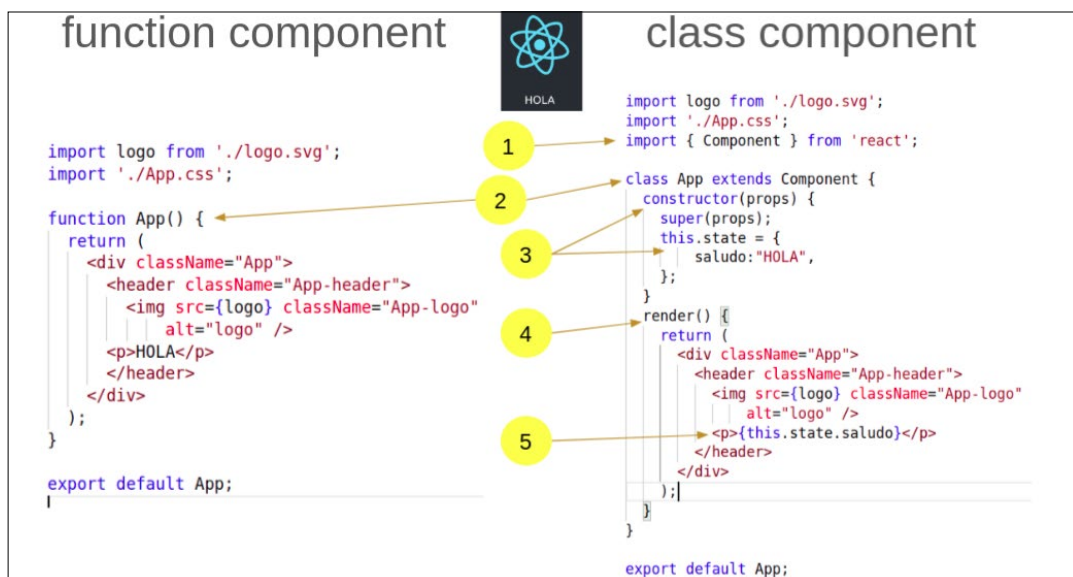
## TOMA NOTA

- Los componentes realizados por el programador deben tener la primera letra en mayúsculas obligatoriamente. Esos componentes serán compilados con el método `React.createElement(.....)`.
- Las etiquetas que empiezan por minúsculas se tratarán como etiquetas del DOM.
- También hay que tener en cuenta que existen componentes predefinidos.

### 6.4.1. Class components y function components

Cuando se crean interfaces en ReactJS, existe la posibilidad de crear un class component o un function component. Primero se verá la diferencia de código entre uno y otro y luego se explicará cuándo es mejor utilizar uno u otro, junto con sus ventajas e inconvenientes.

A continuación, se ha realizado una pequeña modificación al primer componente creado en apartados anteriores para que muestre solo el logo y se ha creado primero como function component y luego como class component:



**Figura 6.6**

Function component vs. class component.

#### RECUERDA

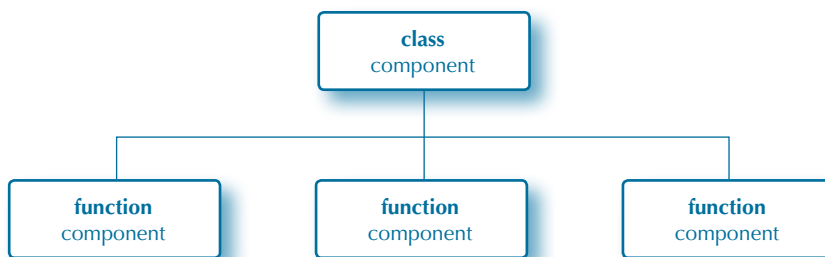
- ✓ A los class components se les denomina *stateful components*, mientras que a los function component se les denomina *stateless components* al carecer de estado estos últimos.

Ambos componentes son similares en comportamiento, pero estructuralmente, como se puede observar, tienen diferencias. A simple vista se han seleccionado las cinco principales entre un function y un class component. Están enumerados y corresponden con los círculos marcados en la figura 6.6:

1. En primer lugar, en un class component se deberá importar la librería Component, la cual es parte de React, la principal de este framework.
2. En el encabezado del componente se observa claramente que uno comienza por class y el otro es una función (function) clásica de JavaScript. El class component será del tipo Component por eso lo extiende (extends).
3. El class component tiene constructor y dentro de él se puede especificar el estado (state), que va a ser muy importante en las aplicaciones puesto que va a almacenar el estado o información de las mismas.
4. Los class component tienen un método que se denomina render(), que es el que dibuja el componente en el interfaz. Hay que tener en cuenta que un componente puede tener a su vez distintos componentes.
5. Por último, en los class components, al almacenar la información dentro del state será posible referenciarla mediante this.state. Más adelante se explicará cómo se modifica el state (estado) del componente porque tiene una manera específica de hacerse.

**SABÍAS QUE...**

El método render() en un componente React es el encargado de mostrar el componente en la interfaz web. La estructura visual de la aplicación está en el método render(). El método render es el que devuelve el contenido HTML, el que dibuja el componente de la aplicación. Es uno de los más importantes, puesto que va a definir o mostrar la funcionalidad del componente.

**Figura 6.7**

Esquema de componentes de una aplicación.

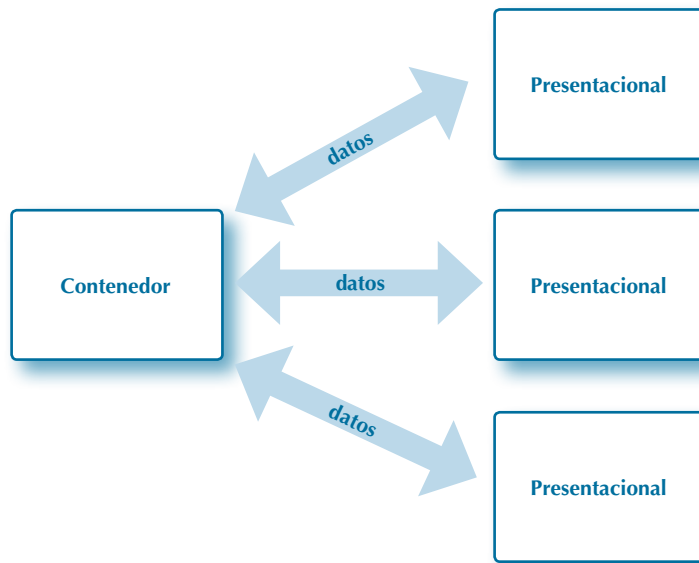
Por lo visto anteriormente sobre los class components y los function components, no es extraño ver en muchas aplicaciones pequeñas, como la mostrada en la figura 6.7, que el componente principal sea un class component para almacenar la información de la aplicación, mientras que los componentes hijo sean function component dado que solamente muestran dicha información. Esto no tiene por qué ser estrictamente así, pero normalmente componentes principales suelen ser de tipo component para poder gestionar el estado de la aplicación.

### 6.4.2. Componentes contenedores y presentacionales

Los componentes en React se clasifican dependiendo de su uso (o de cómo se utilizan).

Aunque existen muchos tipos de uso, lo más común es utilizar componentes contenedores y presentacionales:

- Los componentes *presentacionales*:
  - Están orientados a renderizar la vista. Se preocupan de que tengan un aspecto visual agradable. Utilizan estilos predefinidos para mejorar la presentación.
  - Renderizan la vista dependiendo de los datos que se les proporcione.
  - No hace falta que gestionen su propio estado. No hace falta tampoco que manejen datos, solamente los necesarios para realizar la presentación de la interfaz.



**Figura 6.8**  
Componentes contenedores y presentacionales.

- Los componentes *contenedores*:
  - Son los responsables de que la aplicación funcione correctamente. Pueden dedicarse a realizar consultas a bases de datos, actualizar el estado de los componentes, etc.
  - Realizan llamadas a los componentes presentacionales para mostrar la interfaz al usuario. También les envían datos a estos componentes para que los muestren en el navegador.
  - Generalmente, estos componentes presentacionales están embebidos en forma de `div`s en estos contenedores.
  - Son los responsables de comunicarse con las bases de datos u otros orígenes de datos.



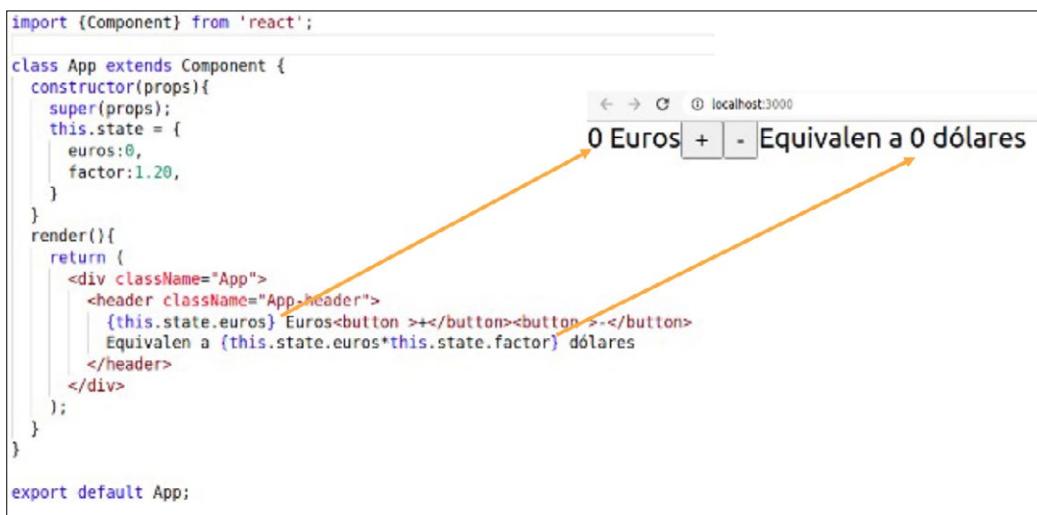
Flux y Redux son las tecnologías que utiliza Facebook para realizar sus interfaces con ReactJS. ¿Qué diferencias hay entre FLUX y REDUX?

- Flux es un patrón de diseño, mientras que Redux, que es más complejo, es una librería. En React se comenzó utilizando Flux, pero ahora se utiliza Redux al tener más ventajas que el primero.
- Flux es el nombre que se le da al patrón de diseño observer (observador), el cual se modificó levemente para utilizarlo en React.
- Tanto Flux como Redux tienen acciones que serían los eventos o lo que desencadenan los eventos. De todas formas, cuando se utiliza un middleware como Redux, las acciones también pueden ser funciones o promesas (promises).
- En Flux el almacenamiento de los componentes puede tener varias ubicaciones, mientras que en Redux el almacenamiento está centralizado. Esta es una de las claves por las que al trabajar con React se utiliza Redux en vez de Flux, por la centralización de la información de la aplicación que hace Redux, lo que redundará en aplicaciones más seguras.

En definitiva, este tipo de aplicaciones se basan en una aplicación que tiene un estado inicial y va a ir modificándose con base en los eventos acaecidos.

### 6.4.3. Creando el componente principal. Conversor euro-dólares

A continuación, se muestra la pequeña aplicación, la cual cuenta solamente con un componente principal App que muestra la conversión de euros a dólares de una cantidad determinada:



**Figura 6.9**

Ejemplo de un componente. Aplicación conversor.

En la figura 6.9 se muestra el código y la apariencia de este código en el navegador. En el ejemplo de código anterior se puede observar cómo se hace uso del state:

```
this.state = {
 euros:0,
 factor:1.20,
}
```

En la declaración del state hay dos atributos, euros y factor. El primero registrará el valor en euros con el que se está trabajando y el segundo es el cambio euro-dólar, que para nuestro ejercicio hemos estimado que será siempre de 1,20 dólares por euro.

Por último, con las siguientes sentencias:

```
{this.state.euros} y {this.state.euros*this.state.factor}
```

Se referencian los datos del state para mostrar el valor en euros y dólares dentro de la visualización de la interfaz. Es importante el uso de las llaves para que se interpreten como código y no como texto.

## 6.5. Los componentes. El state

Como ya se adelantó en un apartado anterior, el state representa el estado del componente y las props (o propiedades), que se explicarán posteriormente, son los parámetros que le pasa un componente padre a su componente subordinado o hijo.

### 6.5.1. El state y los métodos

Sobre el state hay que tener en cuenta las tres siguientes reglas:

1. Cada componente tiene su información local registrada en su estado (state). Esa información es controlada solamente por el componente, no puede ser accedida directamente por otro componente.
2. Ese estado se le puede pasar a los componentes hijos o subordinados como props. De esa forma un ancestro (ancestor) puede pasar información a toda su descendencia (descendant).
3. Solamente los class components pueden tener estado local (state).

Se muestra una evolución del programa conversor con actualización del state:

```
import React, { Component } from 'react';
class App extends Component {
 constructor(props) {
 super(props);
 this.state = {

 euros: 0,
```

```

 factor: 1.20,
 };
}
aumentar(){
 var auxeuro = this.state.euros+1;
 this.setState({euros:auxeuro});
}
disminuir(){
 var auxeuro = this.state.euros-1;
 if (auxeuro>=0) this.setState({euros:auxeuro});
}
render() {
 return (
 <div className="App">

 {this.state.euros} euros <button onClick={() => this.aumentar()}>+</bu-
 tton><button onClick={() => this.disminuir()}>-</button>
 Equivalen a {Math.round(this.state.euros * this.state.factor*100)/100}
 dólares

 </div>
);
}
}
export default App;

```

El aspecto de la aplicación es ahora el siguiente:

8 euros   Equivalen a 9.6 dólares

**Figura 6.10**  
Ejemplo de un componente.  
Aplicación conversor.

#### RECUERDA

- ✓ Cuando modificas el estado con `setState()` la interfaz se renderizará automáticamente. Si no utilizas `setState()` y modificas el estado directamente (no recomendado), la interfaz no se renderizará de forma automática.

Como se puede ver, se han incluido dos métodos, `aumentar()` y `disminuir()`, los cuales se ejecutarán cuando se haga click sobre los botones.

## © FUNDAMENTAL

El estado (state) solamente puede ser modificado utilizando el método `setState()`. Un ejemplo de este método sería el siguiente:

```
aumentar(){
 var auxeuro = this.state.euros+1;
 this.setState({euros:auxeuro});
}
```

Como se puede observar, primero se hace una copia del state, se modifica y luego se vuelve a actualizar con el método `setState()`. ReactJS recomienda no actualizar directamente el *state* del componente. Por ejemplo:

```
this.state.euros += 1;
```

La sentencia anterior no debe hacerse, dado que el modificar el *state* directamente implica no renderizar de nuevo el interfaz, entre otras cosas.

### 6.5.2. Los componentes hijo y la comunicación padre-hijo: las props

JSX pasa información de un componente a otro mediante props, las cuales son tratadas como un objeto individual. Puede accederse a esta información dentro del componente mediante sus props y se puede pasar más de una propiedad (prop).

WWW

#### Recurso web

A través del código QR adjunto accederás a un enlace de React en el que se muestra cómo se renderizan los componentes y cómo se utilizan correctamente las props.



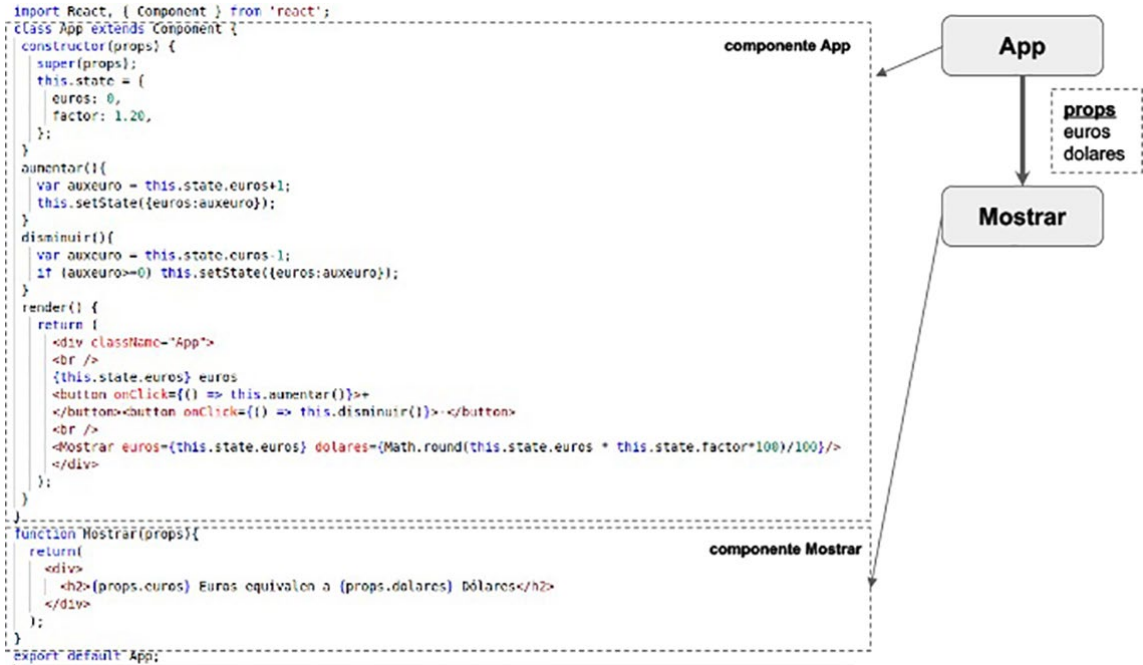
Un componente no puede modificar las props. Si se quiere modificar la información que viene vía props, lo más lógico es copiar el contenido en el state del componente y luego modificar lo que haga falta.



#### TOMA NOTA

Por props puedes enviar tanto variables como funciones. El envío de funciones es muy útil cuando quieres comunicar el hijo con el padre.

A continuación, se muestra una pequeña aplicación que es la evolución de la aplicación conversor que ahora tiene dos componentes. App que será el componente padre y Mostrar que es el componente hijo. En la figura 6.11 aparece el código y alguna información extra para entender la aplicación:



**Figura 6.11**  
Mejorando la aplicación conversor con un componente hijo Mostrar.

#### RECUERDA

- ✓ Aunque en la aplicación anterior se han incluido dos componentes en un mismo fichero para una mejor comprensión, generalmente cada componente irá codificado en un fichero aparte. Por ejemplo, el componente Mostrar se ubicaría en el fichero `MostrarComponent.js`. Recuerda que el nombre de los componentes siempre empieza por mayúsculas.

Como se puede observar en la imagen anterior, el componente App le pasa vía props dos parámetros (*euros* y *dolares*). El componente Mostrar lo podrá referenciar invocando *props.euros* y *props.dolares*.

También hay que tener en cuenta que en el evento *onClick* del botón se le pasa el método *disminuir()* de la siguiente manera:

```
onClick={ () => this.disminuir() }
```

Si en vez de realizar la llamada anterior, se hubiese hecho de la siguiente forma, muy típica en JavaScript:

```
onClick={this.disminuir() }
```

no funcionará. Más adelante, cuando se trabajen los callback, se explicará esta cuestión en profundidad.

### Actividad propuesta 6.1



Instala las developer tools. Necesitas herramientas para conocer el state y las props de los componentes o mucha otra información interesante de tus aplicaciones React. En el siguiente código encontrarás extensiones para Firefox y para Chrome, las cuales van a permitir depurar y mejorar las aplicaciones de una forma sencilla y rápida.

Instálalas en el navegador y comprueba cómo funcionan.



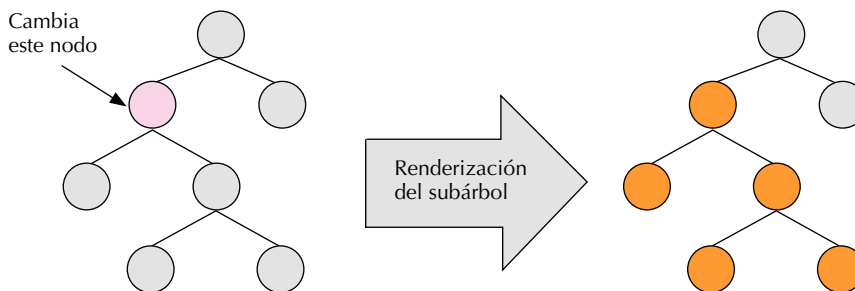
## 6.6. El DOM virtual de React

Como ya se ha indicado, el DOM (*Document Object Model*) es un objeto del navegador. Al igual que el navegador, React utiliza un objeto que mantiene un DOM virtual, que es una representación ligera del DOM del navegador.

React tiene mapeados los objetos del DOM a objetos virtuales suyos. El DOM virtual se almacena como un árbol en memoria donde cada nodo será un objeto que representará a un elemento. Al residir en memoria, este DOM virtual es muy rápido y las modificaciones del mismo son mucho más ágiles que las del DOM del navegador.

### 6.6.1. Renderización y cambios

React tiene un algoritmo que detecta los nodos que cambian y se encarga de re-renderizarlos (mostrarlos de nuevo en el navegador). Además, si cambia un nodo y se renderiza, los nodos descendientes tienen que renderizarse de nuevo, puesto que un ancestro ha cambiado y dichos cambios podrían afectarlos.



**Figura 6.12**  
Cambios ante una renderización.

Por ejemplo, en la figura 6.12 se puede observar cómo React ha detectado que un nodo del DOM virtual ha cambiado. React realizará un recorrido hacia abajo, renderizando todos los objetos del DOM virtual descendientes del nodo modificado.

## 6.7. Los callbacks

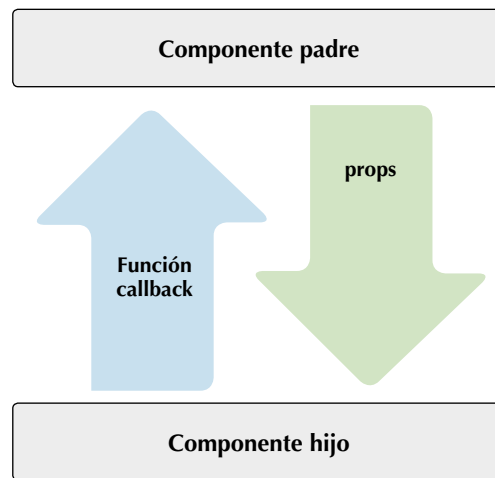
Imagínese un caso frecuente en el que un componente necesita enviar una información a otro componente subordinado y necesita algún feedback (cierta información de vuelta).

Póngase un caso concreto, un componente ficha de paciente que tiene otros componentes subordinados, como diagnósticos del paciente, medicamentos tomados por el paciente, vacunas. El componente ficha necesitará tener constancia de cualquier cambio en los componentes subordinados, como, por ejemplo, la introducción de un nuevo medicamento. El componente ficha será el encargado de:

- Actualizar los cambios o la ficha completa del paciente en la base de datos. O es el responsable de llamar al componente que haga este trabajo.
- Enviar a los componentes subordinados la información (diagnósticos, medicamentos, vacunas) para que lo visualicen.

Este envío de vuelta de información (del hijo al padre) se denomina *callback* y es una función que se le pasa al componente subordinado (hijo) para que la ejecute con los datos del mismo (hijo). Esta función modificará el state del componente padre (figura 6.14).

De una forma gráfica:



**Figura 6.13**  
Comunicación entre componentes padre y componentes hijo.

### Recurso web

Para más información acerca de cómo funcionan los callbacks, puedes consultar un artículo de Jason Arnold mediante el código QR adjunto.



www

A continuación, se va a modificar la aplicación con la que se ha trabajado para añadirle un componente extra llamado Botonera, el cual va a recibir varias props, entre otras, una función que se va a invocar (callback) cuando el usuario presione el botón. Aquí se muestra el código de la aplicación:

```
import React, { Component } from 'react';
class App extends Component {
 constructor(props) {
 super(props);
 this.state = {
 euros: 0,
 factor: 1.20,
 };
 }
 aumentar(){
 var auxeuro = this.state.euros+1;
 this.setState({euros:auxeuro});
 }
 disminuir(){
 var auxeuro = this.state.euros-1;
 if (auxeuro>=0) this.setState({euros:auxeuro});
 }
 render() {
 return (
 <div className="App">

 {this.state.euros} euros
 <Botonera f1={() => this.aumentar()} texto1={"+"} f2={() => this.disminuir()} texto2={"-"} />
 Equivalen a {Math.round(this.state.euros * this.state.factor*100)/100}
 dólares

 </div>
);
 }
}
function Botonera(props){
 return(

 <button onClick={() => props.f1()}>{props.texto1}</button>
 <button onClick={() => props.f2()}>{props.texto2}</button>

);
}
export default App;
```

Como se puede observar en el código, el componente App renderiza al componente hijo, y en esa llamada le pasa vía props cuatro parámetros, los dos textos de los botones y dos funciones *callback*, `aumentar()` y `disminuir()`, que se llamarán en el componente hijo Botonera `f1()` y `f2()`. Cuando se llame a la función `f1()` se estará ejecutando en el padre la función `aumentar()`, y lo mismo ocurrirá en el caso de `f2()` y `disminuir()`.

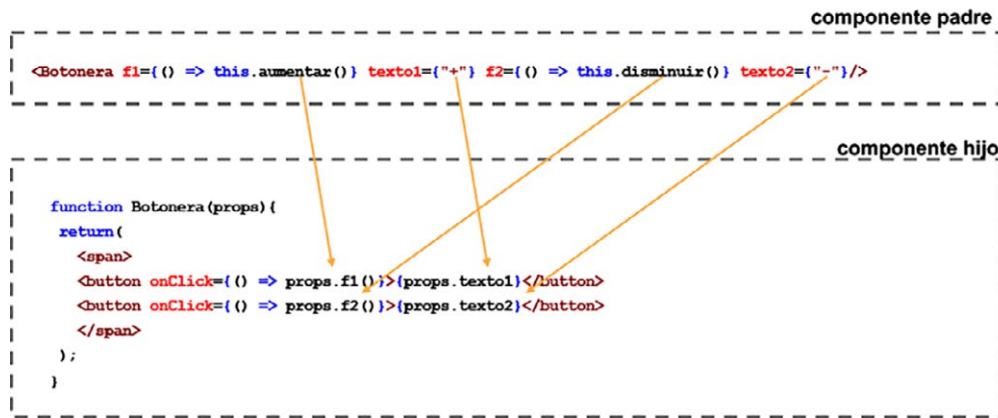


Figura 6.14

Comunicación entre el componente App y el componente hijo Botonera.

La forma más común de pasar las funciones callback como parámetros es la siguiente:

```
f1={() => this.aumentar()}
```

No obstante, la siguiente forma también funcionaría:

```
f1={this.aumentar.bind(this)}
```

Con la llamada anterior, se enlaza (*bind*) la función *aumentar* al parámetro enviado.

#### TOMA NOTA



Al enlazar o pasar una función mediante una función flecha estás creando una nueva función cada vez que el componente se renderiza. Utilizar la función flecha es la forma más utilizada y sencilla de enviar una función callback vía props.

Al enlazar o utilizar una función flecha te aseguras que el método que se va a invocar es realmente el del componente padre.

## 6.8. Prácticas guiadas

A continuación, se muestran dos prácticas que utilizan dos de las técnicas más empleadas en React, las props y los callbacks.

### 6.8.1. Los reyes godos

Se quiere realizar una pequeña aplicación que muestre información sobre los reyes godos que dominaron la península ibérica. Se espera que la aplicación tenga este aspecto:

## Leovigildo

Fue el rey visigodo más importante y admirado de la historia

## Atanagildo

Luchó contra Agila I para ser el rey lo cual supuso una crisis económica para el reino

## Suintila

Derrotó a los vascones y dominó toda la península

## Recaredo

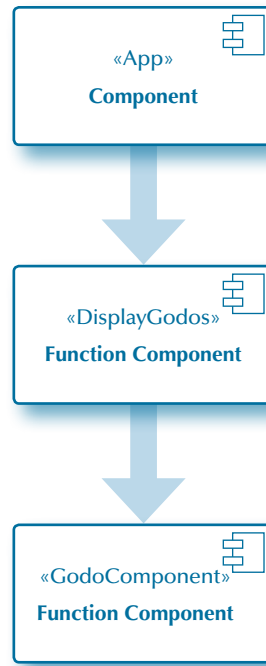
Hijo de Leovigildo que se convirtió al catolicismo.

**Figura 6.15**  
Aspecto del ejercicio.

Para ello, se proporcionará al programador un array en formato JSON, que podrá incorporar a su aplicación de la siguiente manera:

```
const GODOS = [
 {
 id:0,
 nombre:"Leovigildo",
 texto:"Fue el rey visigodo más importante y admirado de la historia"
 },
 {
 id:1,
 nombre:"Atanagildo",
 texto:"Luchó contra Agila I para ser el rey lo cual supuso una crisis
económica para el reino"
 },
 {
 id:2,
 nombre:"Suintila",
 texto:"Derrotó a los vascones y dominó toda la península"
 },
 {
 id:3,
 nombre:"Recaredo",
 texto:"Hijo de Leovigildo que se convirtió al catolicismo."
 }
];
```

Para realizar este ejercicio se propone utilizar una jerarquía de componente como esta:



**Figura 6.16**  
Jerarquía de componentes  
de la aplicación.

Se presenta la solución comentada del ejercicio:

Para el componente principal que se llamará App se ha elegido que sea un class component, puesto que se le va a asignar en el state un atributo con la lista de los reyes godos. A estas alturas el lector ya sabe transformar un function component en un class component. El código del componente será el siguiente:

```

class App extends Component {
 constructor(props) {
 super(props);
 this.state = {
 godos: GODOS,
 };
 }
 render(){
 return (
 <div className="App">
 <header className="App-header">
 <DisplayGodos reyes={this.state.godos}/>
 </header>
 </div>
);
 }
}

```

Como se puede observar, el componente App hace una llamada al componente DisplayGodos, pasándole por props la lista de reyes godos.

El componente DisplayGodos, dado que no tiene que almacenar ningún dato, puede ser un function component. Los function component son más ligeros que los class components y se renderizan antes en el navegador. El código del componente es el siguiente:

```
function DisplayGodos(props){
 const losreyes = props.reyes;
 const lista = losreyes.map((rey)=>
 <GodoComponent key={rey.id.toString()}
 titulo={rey.nombre}
 texto={rey.texto}
 />);
 return(

 {lista}

);
}
```

Como se puede observar, en el componente se almacenan dos datos, uno con la lista de los reyes pasada por props y otro en el que se va a generar un array de componentes. Para generar el array se utiliza la función map(), como se puede ver, y creará tantos componentes GodoComponent como elementos se tengan en el array de reyes godos.

Obsérvese que GodoComponent tiene una prop especial llamada key. Esta prop es necesaria cuando se genera un número determinado de componentes iguales (en este caso, GodoComponent). De esa forma, ReactJS podrá diferenciar a uno de otro por su key.

Al listado de componentes se le pasan dos props, el título o nombre del rey y el texto que se desea que aparezca en el navegador.

No es necesario hacer una copia siempre de las props, pero a veces es conveniente si se piensa que en un futuro la variable dentro del componente puede sufrir algún cambio. Como el lector sabe, no se pueden modificar las props dentro de un componente.

Por último, hay que crear el componente GodoComponent y se ha elegido un function component (función) al ser un componente presentacional. El componente se ha codificado de la siguiente manera:

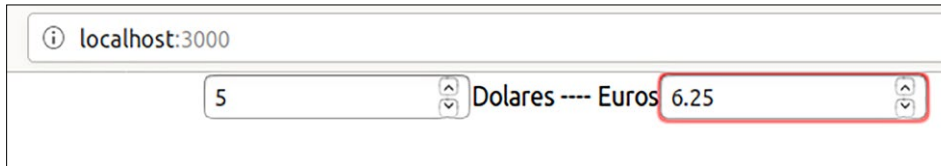
```
const GodoComponent = (props) => {
 return (
 <div>
 <h1>{props.titulo}</h1>
 <p>{props.texto}</p>
 </div>
);
};
```

Como se puede vislumbrar, se ha cambiado la palabra *function* por una función flecha (ya se vio en anteriores capítulos cómo realizar este cambio). GodoComponent lo único que hace es visualizar en el navegador el nombre y texto de un rey godo.

### 6.8.2. Conversor de dólares a euros y viceversa

Se quiere realizar un conversor de euros a dólares y viceversa. El cambio actual es 1 euro = 0,80 dólares.

Se quiere que tenga este aspecto:



**Figura 6.17**

Aspecto del ejercicio para crear un conversor de dólares a euros y viceversa.

Cuando se introduzca una cifra en dólares, se muestran en euros, y viceversa.

Del mismo modo que en el ejercicio anterior, se han unificado todos los componentes en un único archivo. Como se dijo anteriormente, cada componente se suele ubicar en un archivo separado.

En este caso, la aplicación es parecida a la anterior pero en el componente subordinado Moneda se enlaza (bind) el evento onChange con la función handleChange(), la cual hace una llamada al método convertir enviado vía props. Este método o función tiene dos parámetros. El primero será “dolar” o “euro” para conocer el campo que ha sido modificado y el segundo el valor actual de dicho campo. Este valor se recoge en el atributo evt.target.value de la función handleChange(). El objeto evt recoge el evento producido, el objeto target recoge el objeto que ha producido dicho evento y en value se recogerá el value (número) del input utilizado.

```
import React, { Component } from 'react';
import logo from './logo.svg';
import './App.css';

class App extends Component {
 constructor(props) {
 super(props);
 this.state = {
 dolares: 0,
 euros: 0,
 };
 }

 convertir(moneda, valor){
 if (moneda=="dolar"){
 this.setState({dolares:valor});
 this.setState({euros:valor/0.8});
 }
 if (moneda=="euro"){
 this.setState({dolares:valor*0.8});
 this.setState({euros:valor});
 }
 }
}
```

```

 }
 }

 render() {
 return (
 <div className="App">
 <Moneda moneda="dolar" value={this.state.dolares} convertir={moneda, -
 valor} => this.convertir(moneda, valor)} />
 Dólares ---- Euros
 <Moneda moneda="euro" value={this.state.euros} convertir={moneda, va-
 lor} => this.convertir(moneda, valor)} />

 </div>
);
 }
}

class Moneda extends Component {
 handleChange(evt) {
 this.props.convertir(this.props.moneda, evt.target.value);
 }
 render() {
 return (
 <input type="number"
 value = {this.props.value}
 onChange={this.handleChange.bind(this)}
 />
);
 }
}

export default App;

```

### Actividad propuesta 6.2



Introduce el código expuesto en este apartado para comprobar cómo funciona el conversor en tu instalación Node. Una vez verificado, añade una divisa más: el yen japonés.

## Resumen

- Las aplicaciones React están formadas por componentes.
- ReactJS utiliza una sintaxis parecida al XML llamada JSX.
- Lo normal en ReactJS es programar en JSX, pero no es absolutamente necesario.

- Yarn, al igual que NPM, es una herramienta para gestionar paquetes de aplicaciones Node.js.
- create-react-app permite crear el esqueleto de una aplicación básica.
- El método render() en un componente React es el encargado de mostrar el componente en la interfaz web.
- JSX es una extensión de JavaScript.
- Las interfaces de usuario se formarán por componentes los cuales son independientes unos de otros.
- Los componentes son piezas reutilizables, como las piezas de LEGO.
- Un componente devolverá una serie de elementos que aparecerán en la ventana del navegador.
- En React se pueden definir diferentes clases o tipos de componentes.
- Los componentes realizados por el programador tienen que tener la primera letra en mayúsculas obligatoriamente.
- Los componentes en React se pueden clasificar dependiendo de su uso en presentacionales o contenedores.
- El state representa el estado del componente y las props (o propiedades) son los parámetros que le pasa un componente padre a su componente subordinado o hijo.
- Solamente los class components pueden tener estado local (state).
- React utiliza un objeto que mantiene un DOM virtual, esto es, una representación ligera del DOM del navegador.
- React tiene un algoritmo que detecta los nodos que cambian y se encarga de re-renderizarlos.
- El envío de vuelta de información se denomina callback y es una función que se le pasa al componente subordinado (hijo) para que la ejecute con los datos del mismo.

## Ejercicios propuestos



1. *Aplicación básica ReactJS.* Crea una aplicación que tenga el siguiente aspecto:



**Figura 6.18**  
Aspecto en un navegador  
del ejercicio 1.

2. *Los class component.* Convierte el componente del ejercicio anterior (function component) en un class component, de tal manera que el ejercicio tenga el mismo aspecto visual.
3. *Añade un campo de texto.* El objetivo de este ejercicio es añadir un campo de texto al ejercicio anterior en el cual se puedan añadir los regalos o deseos que tenga el usuario. El aspecto de nuestra aplicación tendrá que ser el siguiente:



**Figura 6.19**  
Aspecto en un navegador  
del ejercicio 3 finalizado.

Para que sea más fácil realizar el ejercicio, se proporcionan dos componentes, `Desire` y `DesireList`, los cuales son un campo de texto para introducir el deseo y una lista para mostrar todos los deseos del usuario.

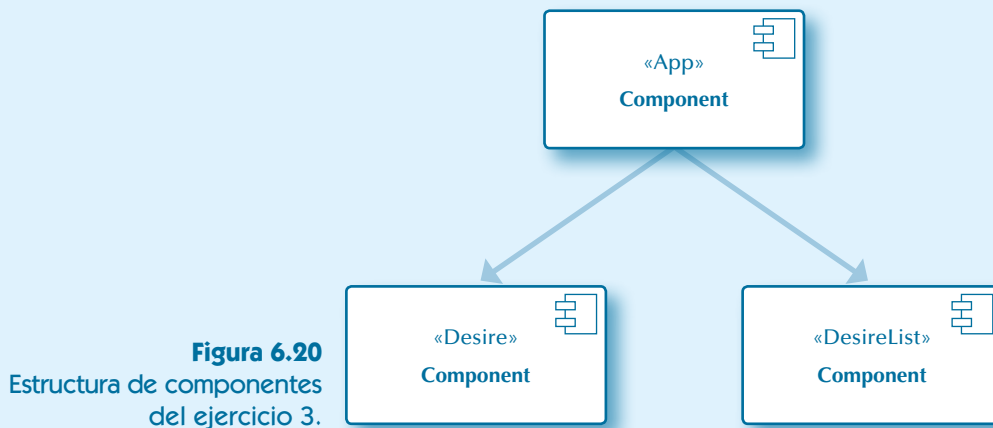
```
class DesireList extends Component {
 render() {
 return(

);
 }
}

class Desire extends Component {
 render() {
 return(
 <form>
 <input type="text" placeholder="Escribe tu deseo" name="deseo"
 />
 </form>);
 }
}
```

Incorpora estos componentes al mismo archivo App.js. No es la manera habitual de proceder el incluir varios componentes en un archivo, pero es más útil para aprender, teniendo en cuenta que nuestros componentes son muy sencillos.

Aprovecha para invocar los componentes en el sitio adecuado del código del componente App. La estructura de componentes de la aplicación será ahora la siguiente:



4. *Añadir un constructor a un componente.* Añade un constructor al componente App del ejercicio anterior que tenga un atributo llamado deseos, el cual se detalla a continuación:

```
deseos: ["GAMBAS", "JAMON"],
```

5. Una vez realizado el ejercicio 4, pasa por props del componente App al componente DesireList el atributo deseos que creaste previamente. Modifica también el componente DesireList para que muestre este parámetro o prop en el navegador dentro de la etiqueta <ul>.

El aspecto de la aplicación debería ser el siguiente:



**Figura 6.21**  
Aspecto en un navegador del ejercicio 5 finalizado.

6. *Incluir nuevos deseos en la aplicación.* Llegó el momento de hacer que a partir del ejercicio 5 se puedan incluir nuevos deseos en la aplicación. Para ello, deberás incluir el siguiente método o función en el código del componente App:

```
handleAniadirDeseo (event) {
 event.preventDefault();
 var aux = [];
 if (this.state.deseos && this.state.deseos !== "null" && this.state.deseos !== "undefined"){
 aux = this.state.deseos.slice();
 }
 aux.push(event.target.deseo.value);
 this.setState({deseos: aux});
}
```

El código anterior lo que hace con `preventDefault()` es evitar que el navegador ejecute la acción por defecto (date cuenta que el input de tipo text estaba dentro de una etiqueta form) y se refresque la página, perdiendo la información suministrada. Una vez desactivada la acción por defecto, se hace una copia del atributo deseos almacenado en el state para, posteriormente, añadirle el nuevo deseo y actualizarlo con el método `setState`.

Además de incluir el código anterior, deberás modificar la llamada al componente `Desire` utilizando el código siguiente:

```
<Desire onAddDeseo={this.handleAniadirDeseo.bind(this)} />
```

Y también modificar el formulario del componente `Desire` para que cuando se pulse la tecla intro se ejecute la función enviada vía props:

```
<form onSubmit={this.props.onAddDeseo}>
```

El aspecto de la aplicación será parecido al siguiente:

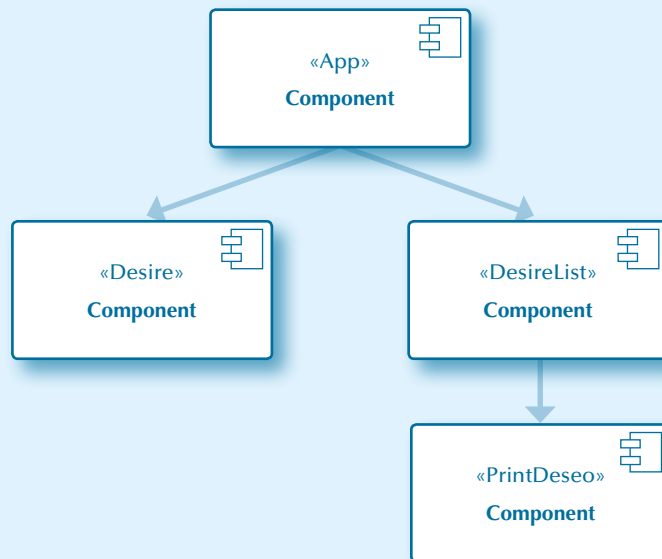


**Figura 6.22**  
Aspecto en un navegador del ejercicio 6 finalizado.

7. *Mejorando el componente DesireList.* Como se puede observar, el aspecto de la aplicación al mostrar los deseos no es el adecuado. Los deseos aparecen uno tras otro sin espacios, por lo tanto, se propone al lector colocar cada deseo en un `<li>` independiente. Para este menester se proporciona el siguiente componente:

```
function PrintDeseo(props){
 return(
 {props.deseo}
);
}
```

Tendrás que modificar el componente DesireList para crear tantos componentes PrintDeseo como deseos haya introducido el usuario.



**Figura 6.23**  
Estructura de componentes  
del ejercicio 7.

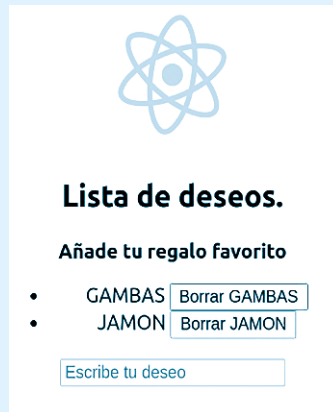
*Ayuda:* utiliza la función `map()` para recorrer el array de deseos. El aspecto de la aplicación debería ser similar al siguiente:



**Figura 6.24**  
Aspecto en un navegador  
del ejercicio 7 finalizado.

8. *Eliminando deseos.* Actualmente, tras realizar el ejercicio 7, se pueden añadir deseos en la aplicación, pero no eliminarlos. Sería necesario incluir un botón para que el usuario pueda eliminar los deseos que estime oportunos.

El aspecto que se le quiere dar a la aplicación es el siguiente:



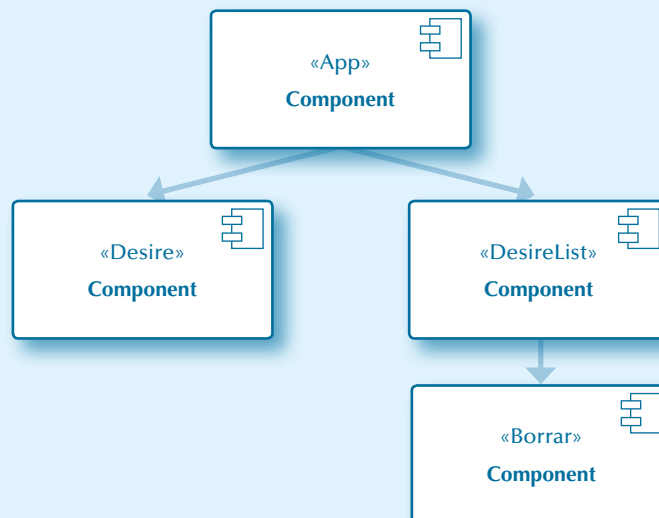
**Figura 6.25**

Aspecto en un navegador del ejercicio 8 finalizado.

Cuando el usuario pulse el botón de borrar, desaparecerá tanto el botón como el deseo correspondiente.

Para realizar el ejercicio tendrás que realizar las siguientes acciones:

1. Realizar un método `quitar()` en el componente `App` que, dado un elemento que se le pase por parámetro, lo elimine del array de deseos del state.
2. Crear un componente `Borrar` el cual sea el botón. Cuando se presione sobre el mismo, este llamará al método `quitar` anterior pasado por props.
3. Modificar la llamada a `DesireList` para pasarle también el método `quitar` como función callback.
4. Modificar el componente `DesireList` para incluir no solo el deseo, sino también un componente `Borrar` por cada deseo impreso.



**Figura 6.26**

Estructura de componentes propuesta para el ejercicio 8.

9. *Instalar ReactStrap en un proyecto.* Es el momento de introducir Bootstrap a un proyecto. Bootstrap da mucha más versatilidad, calidad y definición a una aplicación web.

## Recurso web

www

Reactstrap es una librería que permite utilizar Bootstrap en tu proyecto ReactJS. Muchos proyectos recurren a ella porque al utilizar Bootstrap el desarrollo es más rápido y el resultado más fiable, homogéneo y vistoso. La instalación puede ser a través de npm, como con el resto de librerías; en la web [reactstrap.github.io](https://reactstrap.github.io) puedes ver cómo se instala y se verifica su funcionamiento.

Para incluir Bootstrap en el proyecto, tienes que seguir los siguientes pasos:

Paso 1. Crea la aplicación. En el caso de este ejemplo, se le ha puesto el nombre `holamundoreact`.

```
create-react-app holamundoreact
```

Paso 2. Instala Bootstrap, Reactstrap y sus dependencias con los siguientes comandos:

```
npm install --save bootstrap
```

```
npm install --save reactstrap react react-dom
```

Paso 3. Importa Bootstrap en el fichero `src/index.js` del proyecto creado con la siguiente línea:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

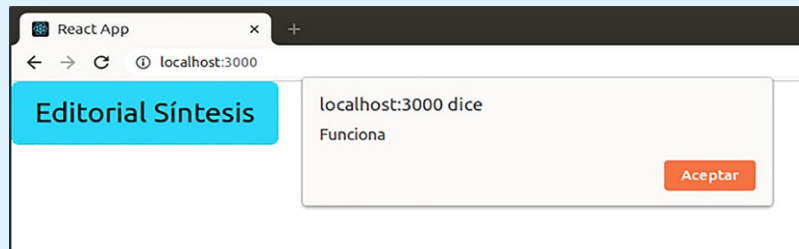
Paso 4. Utiliza el siguiente código en el archivo `src/App.js`

```
import React from 'react';
import { Button } from 'reactstrap';

function App() {
 return (
 <Button color="info" onClick={()=>alert("Funciona")}>Editorial
 Síntesis</Button>
);
}

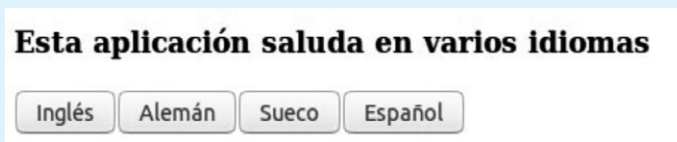
export default App;
```

Si el código funciona y, tras pulsar click sobre el botón, muestra el siguiente mensaje de alerta, eso significa que lo has conseguido.



**Figura 6.27**  
Resultado del ejercicio 9.

10. *Web que saluda en varios idiomas.* Realiza una página web que salude en cuatro idiomas, como la siguiente:



**Figura 6.28**  
Aspecto del ejercicio 10.

Se proporciona parte del componente Saluda:

```
class Saluda extends React.Component {

 constructor(){
 super();
 this.state = {
 saludo: "Esta aplicación saluda en varios idiomas"
 };
 }
 render() {
 return (
 <div>
 <h3> {this.state.saludo}</h3>
 <button onClick={this.mensajeIngles}>Inglés</button>
 <button onClick={this.mensajeAleman}>Alemán</button>
 <button onClick={this.mensajeSueco}>Sueco</button>
 <button onClick={this.mensajeEspanol}>Español</button>
 </div>
)
 }
}
```

Finaliza el componente y la página web.

## ACTIVIDADES DE AUTOEVALUACIÓN

1. ¿Cuál de las siguientes props es correcta?:
  - ☐ a) `incFallo={this.incFallo()}`
  - ☐ b) `incFallo={() => this.incFallo()}`
  - ☐ c) `incFallo={() = this.incFallo()}`
2. Para enlazar el evento `onBlur` con el método `handleBlur()` se utiliza la siguiente sentencia:
  - ☐ a) `onBlur={this.handleBlur.bind(this)}`
  - ☐ b) `onBlur={bind(this.handleBlur)}`
  - ☐ c) `bind(onBlur,this.handleBlur())`
3. ¿Cuál de las siguientes órdenes es la correcta?:
  - ☐ a) `<Casilla posicion={i}> </Casilla>`
  - ☐ b) `<Casilla posicion=this.props.i> </Casilla>`
  - ☐ c) `<Casilla posicion={i} />`
4. Para importar funciones de un archivo JavaScript del proyecto se utilizará la sentencia:
  - ☐ a) `import { aleatorio(),getSudoku(), generaSudoku() } from './sudoku.js';`
  - ☐ b) `import { aleatorio,getSudoku, generaSudoku } from './sudoku.js';`
  - ☐ c) `import { #aleatorio, #getSudoku, #generaSudoku } from './sudoku.js';`
5. En un constructor siempre tiene que existir la llamada:
  - ☐ a) `super(props);`
  - ☐ b) `super(this);`
  - ☐ c) `parent(props);`
6. ¿Cuál de las siguientes afirmaciones es correcta?:
  - ☐ a) Flux y Redux son librerías.
  - ☐ b) Flux es un patrón de diseño mientras que Redux es una librería.
  - ☐ c) Flux y Redux son framework.
7. ¿Cuál es la afirmación verdadera?:
  - ☐ a) JSX es una extensión de JavaScript
  - ☐ b) JSX es una extensión de HTML
  - ☐ c) JSX es una extensión de XML
8. ReactJS es:
  - ☐ a) Technology program agnostic
  - ☐ b) Technology component based agnostic
  - ☐ c) Technology stack agnostic
9. La línea `<div id="root"></div>` está en el fichero:
  - ☐ a) `public/index.html`
  - ☐ b) `src/App.js`
  - ☐ c) `src/index.js`

10. ¿Cuál de las siguientes sentencias es la correcta?:

- ☐ a) `class Tablero extends Component {...}`
- ☐ b) `class Tablero extends React.Component {...}`
- ☐ c) `class Tablero extends React.DOM.Component {...}`

#### SOLUCIONES:

1. ☐ a ☒ b ☐ c

2. ☒ a ☐ b ☐ c

3. ☐ a ☐ b ☒ c

4. ☐ a ☒ b ☐ c

5. ☒ a ☐ b ☐ c

6. ☐ a ☒ b ☐ c

7. ☒ a ☐ b ☐ c

8. ☐ a ☐ b ☒ c

9. ☒ a ☐ b ☐ c

10. ☐ a ☒ b ☐ c